

人工智能 面试宝典

一本搞定面试难题

基础 强化 提升

- 综合素质，专业知识
- 常见问题，详细解答
- 全面实用，技巧精湛
- 一本在手，面试无忧



黑马程序员
www.itheima.com

传智播客旗下
高端IT教育品牌

为莘莘学子改变命运而讲课，为千万学生少走弯路而著书

目录

一、面试软技能-面试基础指导.....	1
1. 简历怎么写.....	1
2. 面试前针对项目撰写完成项目文档.....	3
3. 面试前.....	5
4. 面试中.....	5
5. 面试后.....	6
二、PYTHON语言.....	7
1. PYTHON基础.....	7
1.1 Python与其他语言（如Java/C++）的核心区别是什么？.....	7
1.2 列表和元组的核心区别是什么？.....	7
1.3 在异常处理中，else和finally块分别何时执行？举例说明其适用场景。.....	7
1.4 编写程序在D盘根目录下创建一个文本文件test.txt，并向其中写入字符串hello world.....	7
1.5 写出下面代码的优化版本，提高运行效率.....	8
1.6 写出下面代码的执行结果.....	9
1.7 写出下面代码的运行结果.....	9
1.8 写出下面代码的运行结果.....	10
1.9 编写函数，判断一个数字是否为素数，是则返回字符串YES，否则返回字符串NO.....	11
1.10 编写函数，模拟Python内置函数sorted()。.....	12
1.11 编写程序，实现分段函数计算，如下表所示。.....	12
1.12 请写一段Python代码实现删除一个list里面的重复元素.....	14
1.13 解释 Python 中的三元表达式.....	15
1.14 Python的可变类型和不可变类型？.....	15
2. PYTHON进阶.....	15
2.1 解释装饰器的作用及典型应用场景.....	15
2.2 __new__和__init__的区别是什么？.....	15
2.3 深拷贝与浅拷贝的区别是什么？.....	15
2.4 什么是闭包？Python中如何实现？.....	15
2.5 Python迭代器是什么？什么场景用到迭代器？举一个例子？.....	16



2.6 Python生成器是什么？什么场景用到生成器？举一个例子？	16
2.7 Python多线程与多进程的区别.....	17
2.8 装饰器的实质是什么？	18
2.9 Python下多线程的限制以及多进程中传递参数的方式.....	18
2.10 Python是如何进行内存管理的？	18
2.11 Python里面如何拷贝一个对象？	18
2.12 Python里面search()和match()的区别？	19
2.13 lambda表达式格式以及应用场景？	19
2.14 *args和**kwarg作用.....	19
三、数据处理与统计分析	20
1. MYSQL	20
1.1 SQL基础练习.....	20
1.2 SQL高级练习.....	23
2. PYTHON数据分析	25
2.1 将下面的字典创建为DataFrame	25
2.2 提取含有字符串"Python"的行.....	26
2.3 输出df的所有列名.....	26
2.4 修改第二列列名为'popularity'	26
2.5 统计grammar列中每种编程语言出现的次数.....	26
2.6 将空值用上下值的平均值填充.....	26
2.7 提取popularity列中值大于3的行.....	27
2.8 按照grammar列进行去重.....	27
2.9 计算popularity列平均值.....	27
2.10 将grammar列转换为list.....	27
2.11 将DataFrame保存为EXCEL	27
2.12 看数据行列数.....	27
2.13 提取popularity列值大于3小于7的行.....	28
2.14 交换两列位置.....	28
2.15 提取popularity列最大值所在行.....	28
2.16 查看最后5行数据.....	28
2.17 删除最后一行数据.....	29



2.18 添加一行数据['Perl',6.6]	29
2.19 对数据按照"popularity"列值的大小进行排序.....	29
2.20 统计grammar列每个字符串的长度.....	29
2.21 读取本地EXCEL数据.....	29
2.22 查看df数据前5行.....	30
2.23 将salary列数据转换为最大值与最小值的平均值.....	30
2.24 将数据根据学历进行分组并计算平均薪资.....	30
2.25 将createTime列时间转换为月-日.....	30
2.26 查看索引、数据类型和内存信息.....	31
2.27 查看数值型列的汇总统计.....	31
2.28 新增一列根据salary将数据分为三组.....	31
2.29 按照salary列对数据降序排列.....	32
2.30 取出第33行数据.....	32
2.31 计算salary列的中位数.....	32
2.32 绘制薪资水平频率分布直方图.....	32
2.33 绘制薪资水平密度曲线.....	33
2.34 删除最后一列categories	33
2.35 将df的第一列与第二列合并为新的一列.....	33
2.36 将education列与salary列合并为新的一列.....	33
2.37 计算salary最大值与最小值之差.....	34
2.38 将第一行与最后一行拼接.....	34
2.39 将第8行数据添加至末尾.....	34
2.40 查看每列的数据类型.....	34
2.41 将createTime列设置为索引.....	35
2.42 生成一个和df长度相同的随机数dataframe	35
2.43 将上一题生成的dataframe与df合并.....	35
2.44 生成新的一列new为salary列减去之前生成随机数列.....	35
2.45 检查数据中是否含有任何缺失值.....	35
2.46 将salary列类型转换为浮点数.....	35
2.47 计算salary大于10000的次数.....	36
2.48 查看每种学历出现的次数.....	36



2.49 查看education列共有几种学历.....	36
2.50 提取salary与new列的和大于60000的最后3行.....	36
2.51 使用绝对路径读取本地Excel数据.....	37
2.52 查看数据前三行.....	37
2.53 查看每列数据缺失值情况.....	37
2.54 提取日期列含有空值的行.....	38
2.55 输出每列缺失值具体行数.....	38
2.56 删除所有存在缺失值的行.....	39
2.57 绘制收盘价的折线图.....	39
2.58 同时绘制开盘价与收盘价.....	40
2.59 绘制涨跌幅的直方图.....	40
2.60 让直方图更细致.....	41
2.61 以data的列名创建一个dataframe.....	41
2.62 打印所有换手率不是数字的行.....	41
2.63 打印所有换手率为-的行.....	42
2.64 重置data的行号.....	42
2.65 删除所有换手率为非数字的行.....	42
2.66 绘制换手率的密度曲线.....	43
2.67 计算前一天与后一天收盘价的差值.....	43
2.68 计算前一天与后一天收盘价变化率.....	43
2.69 设置日期为索引.....	43
2.70 以5个数据作为一个数据滑动窗口，在这个5个数据上取均值(收盘价).....	44
2.71 以5个数据作为一个数据滑动窗口，计算这五个数据总和(收盘价).....	44
2.72 将收盘价5日均线、20日均线与原始数据绘制在同一个图上.....	44
2.73 按周为采样规则，取一周收盘价最大值.....	45
2.74 绘制重采样数据与原始数据.....	45
2.75 将数据往后移动5天.....	45
2.76 将数据向前移动5天.....	45
2.77 使用expanding函数计算开盘价的移动窗口均值.....	46
2.78 绘制上一题的移动均值与原始数据折线图.....	46
2.79 计算布林指标.....	46



2.80 计算布林线并绘制.....	47
2.81 导入并查看pandas与numpy版本.....	47
2.82 使用numpy生成20个0-100随机数.....	47
2.83 使用numpy生成20个0-100固定步长的数.....	48
2.84 使用numpy生成20个指定分布(如标准正态分布)的数.....	48
2.85 将df1, df2, df3按照行合并为新DataFrame.....	48
2.86 将df1, df2, df3按照列合并为新DataFrame.....	48
2.87 查看df所有数据的最小值、25%分位数、中位数、75%分位数、最大值.....	49
2.88 修改列名为col1,col2,col3.....	49
2.89 提取第一列中不在第二列出现的数字.....	49
2.90 提取第一列和第二列出现频率最高的三个数字.....	49
2.91 提取第一列中可以整除5的数字位置.....	49
2.92 计算第一列数字前一个与后一个的差值.....	49
2.93 将col1,col2,col3三列顺序颠倒.....	50
2.94 提取第一列位置在1,10,15的数字.....	50
2.95 查找第一列的局部最大值位置.....	50
2.96 按行计算df的每一行均值.....	50
2.97 对第二列计算移动平均值.....	50
2.98 将数据按照第三列值的大小升序排列.....	51
2.99 将第一列大于50的数字修改为高.....	51
2.100 计算第一列与第二列之间的欧式距离.....	51
2.101 从csv文件中读取指定数据.....	51
2.102 从csv文件中读取指定数据.....	52
2.103 从dataframe提取数据.....	52
2.104 将数据取消使用科学计数法.....	52
2.105 将上一题的数据转换为百分数.....	53
2.106 查找上一题数据中第3大值的行号.....	54
2.107 反转df的行.....	54
2.108 按照多列对数据进行合并.....	54
2.109 按照多列对数据进行合并.....	54
2.110 再次读取数据1并显示所有的列.....	55



2.111 查找secondType与thirdType值相等的行号.....	55
2.112 查找薪资大于平均薪资的第三个数据.....	55
2.113 将上一题数据的salary列开根号.....	55
2.114 将上一题数据的linestaion列按_拆分.....	55
2.115 查看上一题数据中一共有多少列.....	56
2.116 提取industryField列以'数据'开头的行.....	56
2.117 按列制作数据透视表.....	56
2.118 同时对salary、score两列进行计算.....	56
2.119 对不同列执行不同的计算.....	56
2.120 计算并提取平均薪资最高的区.....	57
四、机器学习	59
1.1 机器学习中特征的理解.....	59
1.2 机器学习中，有哪些特征工程方法？	60
1.3 机器学习中的正负样本.....	61
1.4 线性分类器与非线性分类器的区别及优劣.....	61
1.5 如何解决过拟合问题.....	61
1.6 L1和L2正则的区别，如何选择L1和L2正则.....	62
1.7 有监督学习和无监督学习的区别.....	62
1.8 有了解过哪些机器学习的算法？	62
1.9 你对LR是怎么理解的？ LR的损失函数是怎样的？	64
1.10 二分类模型如何做多分类？	64
1.11 KMeans算法和KNN算法区别？	65
1.12 请问GBDT和XGBoost的区别是什么？	65
1.13 LR和SVM的联系与区别？	65
1.14 LR与线性回归的区别与联系？	66
1.15 为什么XGBoost要用泰勒展开，优势在哪里？	66
1.16 集成学习介绍下有哪些，讲讲f的随机是什么？.....	67
1.17 讲讲分类评估方式都有什么？.....	67
1.18 F1公式讲一讲？.....	67
1.19 介绍梯度优化的各种算法，详细的发展历史，优化原因？.....	67
1.20 用过xgboost吗，原理你了解吗？他是怎么做并行的？.....	68

五、深度学习	69
1.1 谈谈深度学习中的归一化问题	69
1.2 LSTM结构推导，为什么比RNN好?	70
1.3 Sigmoid、Tanh、ReLu这三个激活函数有什么缺点或不足	70
1.4 为什么引入非线性激励函数?	71
1.5 请问人工神经网络中为什么ReLu要好过于tanh和sigmoid ?	72
1.6 为什么LSTM模型中既存在sigmoid又存在tanh两种激活函数，而不是选择统一一种sigmoid或者tanh? 这样做的目的是什么?	73
1.7 如何解决RNN梯度爆炸和弥散的问题?	73
1.8 什么样的资料集不适合用深度学习?	74
1.9 广义线性模型是怎被应用在深度学习中?	74
1.10 如何解决梯度消失和梯度膨胀	76
1.11 深度学习有哪些熟悉的优化算法?	76
1.12 Adam怎么更新的? (Adam怎么优化?)	76
1.13 讲讲学习率衰减?	77
1.14 pytorch里怎么从CPU迁移到GPU?	77
1.15 激活函数sigmoid的梯度最大点在哪儿，等于多少?	77
1.16 batch_size对训练的影响过拟合，怎么处理?	77
1.17 gelu函数和relu函数的区别?	78
六、自然语言处理	79
1. NLP基础	79
1.1 hmm原理、主要问题、解决方法?	79
1.2 fasttext和word2vec差异?	79
1.3 介绍一下Glove?	80
1.4 attention为什么使用多头?	80
1.5 交叉熵和对数似然损失的区别和联系?	81
1.6 交叉熵和KL散度的区别和联系?	81
1.7 rnn怎么产生梯度爆炸的，lstm怎么解决梯度消失的	81
1.8 过拟合的解决方法?	82
1.9 Bert为啥能解一词多意?	82
1.10 Textcnn会不会? 详细介绍一下细节?	82



1.11 标签平滑是怎么做的，有什么作用？	83
1.12 NER中加入CRF的原因是什么？	83
1.13 transformer为什么用layer norm不用batch norm，解释下，什么情况可以用batch norm？	83
1.14 fasttext结构是怎么样的？怎么迁移学习的，具体过程？fasttext对比lstm优点？为什么有这些优点？具体讲下层次softmax和ngram (lstm都有哪些结构)？	84
1.15 层次softmax为什么用霍夫曼树，其他树行不行？	84
1.16 维特比算法时间复杂度，维特比算法原理？	85
1.17 CRF和HMM的优缺点，模型的结构上有什么不同吗，模型的输入输出是什么样子呢？	85
1.18 说一下层次softmax和负采样？	86
1.19 Add&norm层的作用？	87
1.20 rouge指标是怎么计算的？	87
1.21 介绍一下BERT？BERT和BiLSTM 有什么区别？BERT 和 Attention是什么关系？解释Q, K, V分别的作用？	87
1.22 词向量是什么，有哪些方式生成词向量，句子的词向量是什么	88
1.23 说一下前缀树？	88
1.24 讲讲迁移学习，特征迁移和微调迁移？	89
1.25 bert和transformer的区别？	90
1.26 bert怎么解决超级长的文本 如果不截断呢？了解XLnet么？	90
1.27 Word2vec是怎么计算的，点乘还是加权求和？	90
1.28 霍夫曼树如何训练Word2vec的，介绍一下？	90
1.29 有论文说bert不加crf效果和加了crf效果差不多，有没有看相关论文？你自己预想只用bert和 bert+crf来做ner会有什么样的差别？	91
2. 投满分项目面试题	91
2.1 了解那些激活函数？详细讲一下你用到的激活函数GeLU	91
2.2 项目的构建过程？	92
2.3 Bert模型的构建？	93
2.4 了解哪些模型压缩的方法？详细讲讲一下模型量化	93
2.5 了解那些机器学习算法？详细讲一下你用到的随机森林的算法原理	96
2.6 讲一个你认为最能体现你各方面能力的项目？重点讲解一下做项目过程中的优化思路	96
2.7 你们模型部署是用什么框架的？具体讲讲模型部署过程	97
2.8 你们的数据是哪来的？数据内容是什么？	97
2.9 项目中算法的损失函数是什么？优化器？	97



2.10 项目中的评估指标有哪些？都是怎么计算的？	98
2.11 你们模型服务封装是用什么框架的？具体讲讲模型部署过程.....	99
2.12 KL散度损失和交叉熵损失有什么区别？	100
2.13 fit(),transform(),fit_transform()有什么区别？	101
3. 知识图谱项目面试题	102
3.1 医疗数据来自于哪里，原始数据什么格式？	102
3.2 使用了什么标注工具，如何分配的标注任务？	102
3.3 使用了什么标注规则，详细解释一下.....	102
3.4 如何实现数据预处理，中间经过了哪些步骤？	102
3.5 对比了NER哪些方法，最后选择了哪一套模型组合，为什么这么选？	103
3.6 有哪些方法可以实现关系抽取，比较一下他们的原理.....	103
3.7 如何推导CRF损失函数？	103
3.8 CRF的路径分数是怎么计算的？	104
3.9 实现关系抽取过程中，都遇到了哪些棘手问题？	104
3.10 比较pipeline和joint方法的不同.....	104
3.11 在哪些场景用过基于规则方法，核心是什么，用了哪些工具？	104
3.12 基于pipeline实现关系抽取，如何组织数据的，中间经过了哪些过程？	104
3.13 如何部署上线的，用了哪些工具？	104
3.14 如何定义的知识图谱的本体和实例？	104
七、大语言模型	105
1. 大语言模型基础	105
1.1 目前主流的开源模型体系有哪些？	105
1.2 prefix LM 和 causal LM 区别是什么？	106
1.3 为何现在的大模型大部分是Decoder only结构.....	107
1.4 LLMs复读机问题.....	108
1.5 如何让大模型处理更长的文本？	109
2. 大语言模型架构	109
2.1 讲讲对Attention的理解？	109
2.2 Attention的计算步骤是什么？	110
2.3 Attention机制和传统的Seq2Seq模型有什么区别？	110
2.4 transformer中multi-head attention中每个head为什么要进行降维？	111



2.5 Encoder编码器与Decoder掩码有什么区别？	111
2.6 为什么BERT选择mask掉15%这个比例的词，可以是其他比例吗？	111
2.7 BERT非线性的来源在哪里？	112
2.8 为什么要进行LN呢？	112
3. 训练数据集	113
3.1 SFT（有监督微调）的数据集格式？	113
3.2 RM（奖励模型）的数据格式？	113
3.3 PPO（强化学习）的数据格式？	114
4. 有监督微调	115
4.1 微调方法是啥？如何微调？	115
4.2 微调和参数高效微调之间的区别是什么？	115
4.3 PEFT 有什么优点？	116
4.4 多种不同的高效微调方法对比	116
4.5 当前高效微调技术存在的一些问题	117
5. 推理	117
5.1 为什么大模型推理时显存涨的那么多还一直占着？	117
5.2 大模型在GPU和CPU上推理速度如何	118
5.3 推理速度上，INT8和FP16比起来怎么样？	118
5.4 大模型有推理能力吗？	118
5.5 大模型生成时的参数怎么设置？	119
5.6 有哪些省内存的大语言模型训练/微调/推理方法？	119
6. 强化学习	120
6.1 简单介绍强化学习？	120
6.2 简单介绍一下 RLHF？	121
6.3 奖励模型需要和基础模型一致吗？	123
6.4 RLHF 在实践过程中存在哪些不足？	123
6.5 什么是 LLM Agent？	124
6.6 LLM Agent 有什么关键能力？	124
6.7 怎样构建基于 LLM 的 Agents？	124
7. 大语言模型评估	125
7.1 大模型怎么评测？	125



7.2 大模型的 honest 原则是如何实现的？模型如何判断回答的知识是训练过的已知的知识，怎么训练这种能力？	125
7.3 如何衡量大模型水平？	125
7.4 大模型评估方法有哪些？	126
7.5 什么是大模型幻觉？	126
7.6 为什么需要解决LLM的幻觉问题？	126
7.7 幻觉一定是有害的吗？	126
7.8 幻觉有哪些不同类型？	127
7.9 为什么LLM会产生幻觉？	127
7.10 如何度量幻觉？	127
8. 大语言模型应用	128
8.1 什么是 LangChain?	128
8.2 LangChain 包含哪些核心模块	128
8.3 LangChain 如何调用 LLMs 生成回复？	129
8.4 LangChain 如何修改 提示模板？	129
八、数据结构与算法	130
1. 背景知识	130
1.1 python数据结构	130
1.2 插件	131
2. 二 热身	132
2.1 两数之和	132
3. 排序	132
3.1 经典排序	132
3.2 148. 排序链表	137
4. 滑动窗口算法	137
4.1 3. 无重复字符的最长子串	137
4.2 76. 最小覆盖子串	137
4.3 438. 找到字符串中所有字母异位词	138
4.4 567. 字符串的排列	138
5. 动态规划	138
5.1 509. 斐波那契数	138

5.2 322. 零钱兑换.....	138
5.3 416. 分割等和子集.....	138
6. 买卖股票最佳时机.....	140
6.1 188. 买卖股票的最佳时机 IV.....	140
6.2 121. 买卖股票的最佳时机.....	140
6.3 122. 买卖股票的最佳时机 II.....	140
6.4 123. 买卖股票的最佳时机 III.....	140
6.5 309. 最佳买卖股票时机含冷冻期.....	140
6.6 714. 买卖股票的最佳时机含手续费.....	140
7. 回溯算法.....	140
7.1 46. 全排列.....	140
7.2 51. N 皇后.....	141
8. BFS算法.....	141
8.1 111. 二叉树的最小深度.....	141
8.2 752. 打开转盘锁.....	142
9. 其它算法题目.....	142
九、计算机视觉.....	142
1. 图像分类.....	142
1.1 GoogLeNet采用了模块化的结构（Inception结构），Inception历经了V1、V2、V3、V4等多个版本的发展，不断趋于完善，请说出3条以上V1-V4演进中的变化.....	142
1.2 请分别描述开运算，闭运算操作的特点及功能.....	143
1.3 卷积神经网络的组成.....	144
1.4 为什么 CNN 对像素级别的分类很难？.....	146
1.5 什么是扩张卷积？.....	146
1.6 什么是空间金字塔池化（Spatial Pyramid Pooling）.....	147
1.7 1×1 卷积核的作用是什么？.....	149
1.8 窄卷积和宽卷积分别是什么？.....	149
1.9 FCN 网络结构是什么？.....	150
1.10 如何进行卷积神经网络进行参数设置？.....	151
1.11 CNN为什么要进行局部连接？.....	152
1.12 什么是权值共享，有什么用？.....	152



1.13 为什么很多做人脸的项目会最后加入一个 Local Connected Conv (即参数不共享) ?	154
1.14 为什么出现棋盘现象?	155
1.15 有哪些方法可以避免棋盘效应?	156
2. 目标检测	156
2.1 从R-CNN到Faster R-CNN谈谈物体检测	156
2.2 目标检测two-stage模型有哪些	156
2.3 说一下NMS 的处理思想	157
2.4 Soft nms为什么可以提升模型效果	157
2.5 描述一下IOU的作用	158
2.6 roi pooling 实现怎样的映射	159
2.7 SPP具体是指什么	160
2.8 说一下RPN的作用	161
2.9 说一下faster rcnn的损失函数	162
2.10 Faster rcnn怎么筛选正负anchor	162
2.11 为什么使用focal loss	163
2.12 目标检测one-stage模型有哪些	164
2.13 SSD如何生成目标框	164
2.14 Yolo v1如何生成目标框	165
2.15 语义分割模型	166
2.16 说一下DeepLab v1 v2 v3的区别	166
2.17 介绍一下Unet	167
2.18 Canny边缘检测的流程	167
2.19 对霍夫变换的理解	168
2.20 说一下对HOG特征的理解	168
2.21 常用的图像局部特征和全局特征	169
2.22 图像的角点是什么? 为什么使用角点作为特征?	169
2.23 常用的颜色空间	170
2.24 图像的插值方法有哪些?	170
2.25 常用的滤波方法, 写出均值滤波的实现	174
2.26 Graph-cut的基本原理和应用	176
2.27 写一个图像的resize方法	176

2.28 图像中直方图的应用.....	178
2.29 SIFT/SURF的特征提取方法，是如何保持尺度不变性的？	179
十、面试时注意事项和人事面	179
1. 请介绍一下你自己.....	179
2. 为什么来XX（北京或深圳）找工作？	180
3. 你为什么离开原来的公司？	180
4. 你最大的缺点是什么？	181
5. 你对薪资的要求？	181
6. 什么时候能入职？	182
7. 介绍一个你认为最熟悉的项目（项目经理）	182
8. 如果公司录用你，你将怎样开展工作？	182
9. 你还有什么问题问我吗？	182
10. 社保问题	183
11. 上家公司薪资构成，及到手多少钱？	183
12. 你有没有在工作中遇到过你无论如何都难以解决的问题？你的态度如何？	183
13. 你对加班的看法？	184
14. 你朋友对你的评价？	184
15. 谈谈你对跳槽的看法？	184
16. 工作中你难以和同事、上司相处，你该怎么办？	184
17. 假设你在某单位工作，成绩比较突出，得到领导的肯定，但同时你发现同事越来越孤立你，你怎么看这个问题？你准备怎么办？	184
18. 最能概括你自己的三个词是什么？	185
19. 你的业余爱好是什么？	185
20. 说说你对行业、技术发展趋势的看法	185
21. 说你的家庭？	185
22. 就你申请的这个职位，你认为你还欠缺什么？	185
23. 你欣赏哪种性+格的人？	186
24. 你为什么愿意到我们公司来工作？	186
25. 你和别人发生过争执吗？你是怎样解决的？	186
26. 你做过的哪件事最令自己感到骄傲？	186
27. 对这项工作，你有哪些可预见的困难？	186

28. 与上级意见不一致，你将怎么办？	187
29. 你希望与什么样的上级共事？	187
30. 为什么我们要在众多的面试者中选择你？	187
31. 怎样看待学历和能力？	187
32. 工作中学习到了些什么？	188
33. 你希望你的上级是什么样子的？	188
34. 除了本公司外，还应聘了哪些公司？	188
35. 面试注意事项	188

一、面试软技能-面试基础指导

1. 简历怎么写

面试时候技能点写法：

建议：不建议“熟练使用python语言，熟悉机器学习算法，熟悉深度学习模型”，建议：了解分类、回归、聚类等机器学习算法的原理，并能够结合实际业务加以运用；掌握RNN、CNN、Transformer等深度学习模型架构和原理，并具备企业实际项目应用经验

面试之前建议：

1) 需要大家讲简短的自我介绍(不要讲以前Java转到AI、或者是机械转AI等)

我叫什么

我来自于什么

我之前在哪里工作

我目前来到贵公司希望应聘什么职位

(看到贵公司工作氛围比较好，符合我的工作环境预期)

2) 项目讲解都要书面化

项目名称：基于Neo4j图数据库实现医疗智能对话问答系统项目描述：

简要介绍下项目背景，突出为什么做，以及做之后的优势 数据来源是什么：如何获取的数据（自训练还是人工标注）

开发并搭建了什么类型的模型，该模型支持什么任务，模型指标是什么

基于第一个版本模型的某些弊端，提出模型优化改进方法：如修改参数、清洗数据、甚至是换新的模型，改进后模型的指标对比

达成业务目标（如：通过智能客服对话，减少人工成本，提升服务质量）

3) 项目职责(简历中写)

不要出现太多的参与，只允许有1项参与

多状态少动作--多写数据分析的指标任务，少写数据采集，数据导入导出

日常任务：

(1) 训练和测试数据的搜集和清洗

(2) 基于业务分析，开发和搭建模型



- (3) 模型迭代优化，优化过程包括：dif数据修正、模型结构更改等等
- (4) 开发模型API接口，便于项目功能的服务对外使用
- (5) 模型实现A/B测试，并部署服务

2. 面试前针对项目撰写完成项目文档

建议大家准备完成自我介绍：

自我介绍（参考）

面试官,您好！

我叫XXX，21年毕业于XXXX学校，主修专业为机械工程，本人在校期间，对人工智能方向非常感兴趣，因此在大三期间，申请进入了我校**老师的实验室，该老师的研究方向主要为：机器人与人工智能，在实验室这段时间，本人对机器学习算法以及NLP算法知识进行了系统学习和了解。21年毕业后，本人就职于**科技有限公司，主要岗位职责为：负责

文本数据分析、中文分词、文本分类、NER以及知识图谱等相关NLP基础任务，主要参与项目：

- 1) 基于Neo4j图数据库搭建了一套辅助医疗诊断的智能问答系统；
- 2) 基于公司商业产品数据，实现了产品自动分类的过程。本人在工作中，态度比较端正，能够积极负责，对待同事也能够友好的相处，本人平时爱好主要为听音乐和阅读，那么以上就是本人的基本情况介绍，感谢面试官的聆听。

项目介绍

那我给您介绍一下最近的这个项目。

这里应该写出是在什么时间什么地点基于什么场景做的这个项目，在开始介绍下面

该项目是基于NLP技术的一款聊天机器人平台，其目的在于通过人工智能技术让机器像人类一样能够进行智能回复，解决现实中的各种问题。公司基于客户需求，为了方便病人快速了解所得疾病，基于Neo4j图数据库搭建了一套辅助医疗诊断的智能问答系统。

本项目工作主要分为两个模块，分别为离线模块和在线模块，其中离线模块包括：1、基于BiLSTM+CRF模型实现非结构化文本NER的抽取，共支持3种实体类型，其中模型指标为F1=91%，2、基于BERT+RNN完成命名实体审核任务，模型指标ACC=92%，3、设计schema，将实体以及对应关系导入到Neo4j图数据库中。以上是离线部分。在线部分为：负责主逻辑函数的开发，以及基于BERT模型实现句子主题相关服务的搭建，模型指标为ACC=96%，线上最大支持200qps。

个人主要职责：

- 1、负责非结构非结构化数据的处理：如脏数据清洗，正则过滤特殊符号等；
- 2、负责模型的设计以及训练工作，并采用不同策略进行模型的优化，如：参数调节，结果后处理等；

3、负责对话主逻辑服务的开发，借助 werobot服务实现智能对话。

在这里应该准备几个技术上遇到的难题，以及给出解决办法(可以将最后的遇到的问题在这里讲，根据情况做调整)

技术点

训练数据集以及测试集的大小是多少，是如何获得的（全是人工标注的吗）？

一般数据量大小可以自己拟定，这里拟定训练集为20万条，测试集为1w；测试集人为标定

训练集的获取方式：

第一种回答方式：数据部门直接提供（已经打上标签），不需要我们负责；

第二种回答方式：半监督（自训练）方式

针对第二种半监督（自训练）方式的实现过程（一般思路）：

- 1) 首先人工标注1w条数据集（人工标注的数据我们一般认为准确率为100%），将标注好的数据集，分为训练集8000和测试集2000；
- 2) 利用8000训练集训练我们的基础模型如：bilstm+crf，根据训练好的模型去预测测试集，对比和人工标注的结果dif，如果准确率为96%，可以忽略dif，我们认为该模型效果不错，直接使用训练好的模型，接下面第3步；
- 3) 利用第二步训练好的模型，去预测新一批数据（该部分数据为非标注数据），如新选取1w条，此时经过模型预测后，1w条新的数据已打上标签；
- 4) 将第3步得到新的1w条数据和之前的8000人工标注数据合并，在此送入模型中训练，训练好的模型去预测2000条测试集，此时准确率可能为94%，如果我们定义模型准确率指标为95%，那么此时训练集中引入噪声，存在脏数据（也就是实体类别有标错的数据），可以根据2000条测试集的dif数据，人工筛选实体数据错误类型，对应查找训练集的数据，然后修改，修改完后重复4步骤，发现模型准确率为95%，即达标。

在做NER抽取任务时，为什么BiLSTM模型后面还要加一个CRF？BiLSTM也能直接解析NER结果的

在NER抽取任务中，BiLSTM模型可以用于对输入序列进行建模和特征提取，得到每个词对应的特征表示。然而，由于NER任务是一个序列标注任务，它需要考虑词之间的关系，以及如何将这些词标注为实体类别。BiLSTM模型可以得到每个位置上的词语的特征表示，但是它并不能直接捕捉到整个序列上的概率分布，也不能直接对整个序列进行标注。而这些都是CRF模型擅长的。CRF（条件随机场）是一个经典的序列标注模型，可以直接对整个序列进行建模，并且可以根据上下文信息、标注标签之间的相关性等因素来进行标注，从而得到更加准确的结果。因此，在NER抽取任务中，通常会将BiLSTM和CRF结合起来使用，BiLSTM用于提取特征表示，而CRF用于对整个序列进行标注，以得到最终的NER结果。

3. 面试前

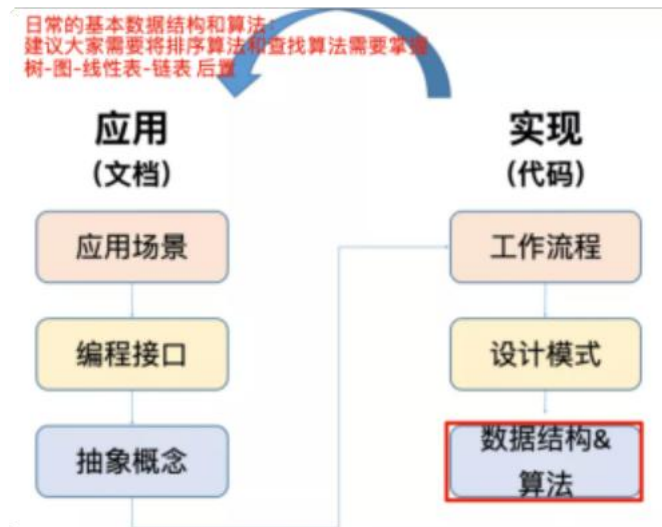
- 1) Boss上搜索对应职位与HR及技术进行简单沟通，寻求符合匹配项的职位；
- 2) 沟通中根据自己做过的项目简述核心要点，更详细部分可以进一步约面试详聊
- 3) 切记：不要针对一个公司准备一个简历，要使用我们的简历和项目寻求合适公司
- 4) 语言表达清楚：思维逻辑清晰，表达流畅
- 5) 所述内容不犯错
 - (1) 不说之前公司或者自己的坏话
 - (2) 往自己擅长的方面说
 - (3) 如果对于熟悉的内容多聊聊业务+技术；没听过，可以说后续很快能够学会。

学会查看岗位职责，包含NLP岗位核心技术，比如文本翻译、语音识别、文本分类等

4. 面试中

- (1) 投递简历当天没有收到面试邀约
 - ① 当天投递，需要等待3天左右时间，需要查看简历
 - ② 建议大家去Boss直聘跟Boss聊天进一步预约面
- (2) 讲解项目
 - ① 清晰讲解项目业务场景描述+项目架构+自己负责部分+优化及坑
- (3) 讲解知识
 - ① 知识点是什么+原理是什么+工作中怎么用+优化是什么
 - ② 如何学习一门新技术或技术栈

Github的开源社区+技术博客+相关论文阅读



- ① 应用主要分为三个部分，首先应该了解它的应用场景，比如窗口的一些使用场景。然后，进一步地我们去了解它的编程接口，最后再深入了解它的一些抽象概念。因为一个框架或一项技术，肯定有它的编程接口和抽象概念来组成它的编程模型。我们可以通过查看文档的方式来熟悉它的应用。在对应用这三个部分有了初步的了解后，我们就可以通过阅读代码的方式去了解它的一些实现了。
- ② 实现部分也分三个阶段，首先从工作流程开始，可以通过 API 层面不断的下钻来了解它的工作流程。接下来是它整体的设计模式，通常对一些框架来说，如果能构建一个比较成熟的生态，一定是在设计模式上有一些独特的地方，使其有一个比较好的扩展性。最后是它的数据结构和算法，因为为了能够处理海量数据并达到高性能，它的数据结构和算法一定有独到之处。我们可以做些深入了解。

(4) 面试中关于技术选型的演变

比如：在进行NER任务时，第一个版本用的是BiLSTM+CRF，但是第二个版本用了BERT+CRF，这里选用BERT的主要原因在于其更强的特征表达能力

5. 面试后

- (1) 面试官面试后会给一定反馈，及时询问反馈
- (2) 如果对岗位和公司非常感兴趣可以多问几次，可以说明自己的意愿
- (3) 切记：面试后需要找老师沟通存在什么问题

二、python语言

1. Python基础

1.1 Python与其他语言（如Java/C++）的核心区别是什么？

Python是动态类型的解释型语言，语法简洁，支持多种编程范式（面向对象、函数式、过程式）。与Java相比，Python无需编译且语法更简洁；与C++相比，Python开发效率高但运行速度较慢，且通过解释器执行而非直接编译为机器码。

1.2 列表和元组的核心区别是什么？

列表可变，用[]定义，适用于动态数据集合（如增删元素）；

元组不可变，用()定义，适用于固定数据（如字典键或配置项），且内存占用更小。

1.3 在异常处理中，else和finally块分别何时执行？举例说明其适用场景。

else在无异常时执行（如资源初始化成功后的操作），finally无论是否异常均执行（如释放文件锁）。

1.4 编写程序在D盘根目录下创建一个文本文件test.txt，并向其中写入字符串hello world

答：

```
1 fp = open(r'D:\test.txt', 'a+')
2 print('hello world', file=fp)
3 fp.close()
```

Python

复制代码

1.5 写出下面代码的优化版本，提高运行效率

▼ Python [复制代码](#)

```
1 x = list(range(500))
2 for item in x:
3     t = 5**5
4     print(item+t)
```

答:

▼ Python [复制代码](#)

```
1 x = list(range(500))
2 t = 5**5
3 for item in x:
4     print(item+t)
```



1.6 写出下面代码的执行结果

Python

复制代码

```
1 def Join(List, sep=None):  
2     return (sep or ',').join(List)  
3 print(Join(['a', 'b', 'c']))  
4 print(Join(['a', 'b', 'c'], '.'))
```

答:

Python

复制代码

```
1 a,b,c  
2 a:b:c
```

1.7 写出下面代码的运行结果

Python

复制代码

```
1 def Sum(a, b=3, c=5):  
2     return sum([a, b, c])  
3 print(Sum(a=8, c=2))  
4 print(Sum(8))  
5 print(Sum(8, 2))
```

答:



Python

复制代码

```
1 13
2 16
3 15
```

1.8 写出下面代码的运行结果



Python

复制代码

```
1 def Sum(*p):
2     return sum(p)
3 print(Sum(3, 5, 8))
4 print(Sum(8))
5 print(Sum(8, 2, 10))
```

答:



Python

复制代码

```
1 16
2 8
3 20
```



1.9 编写函数，判断一个数字是否为素数，是则返回字符串YES，否则返回字符串NO

答

Python

 复制代码

```
6 def IsPrime(v):  
    if v <= 1:  
        return 'No'  
    if v == 2:  
        return 'Yes'  
    if v % 2 == 0:  
        return 'No'  
    n = math.isqrt(v) + 1  
    for i in range(3, n, 2):  
        if v % i == 0:  
            return 'No'  
    return 'Yes'
```

1.10 编写函数，模拟Python内置函数sorted().

Python 复制代码

```
1 def Sorted(v):
2     t = v[:]
3     r = []
4     while t:
5         tt = min(t)
6         r.append(tt)
7         t.remove(tt)
8     return r
```

1.11 编写程序，实现分段函数计算，如下表所示.

x	y
$x < 0$	0
$0 \leq x < 5$	x
$5 \leq x < 10$	$3x - 5$
$10 \leq x < 20$	$0.5x - 2$
$20 \leq x$	0



答：



Python

复制代码

```
1  x = input('Please input x:')
2  x = eval(x)
3  if x<0 or x>=20:
4      print(0)
5  elif 0<=x<5:
6      print(x)
7  elif 5<=x<10:
8      print(3*x-5)
9  elif 10<=x<20:
10 print(0.5*x-2)
```

1.12 请写一段Python代码实现删除一个list里面的重复元素

Python |  复制代码

```
1  >>> l = [1,1,2,3,4,5,4]
2  >>> list(set(l)) [1, 2, 3, 4, 5]
3  或者
4  d = {}for x in mylist:
5      d[x] = 1
6  mylist = list(d.keys())
7  利用map()函数，把用户输入的不规范的英文名字，变为首字母大写，其他小写的规范名字输入：
   [ 'adam', 'LISA', 'barT'], 输出： ['Adam', 'Lisa', 'Bart']:
8  def normalize(name):
9      return name[0].upper()+name[1:].lower()
10 def normalizeList(inputlist):
11     return list(map(normalize, inputlist))
```

1.13 解释 Python 中的三元表达式

与 C++不同，在 Python 中我们不需要使用?符号，而是

使用如下语法： [on true] if [expression]else [on false]

如果 [expression] 为真，则 [on true] 部分被执行。如果表示为假则 [on false] 部分被执行

1.14 Python的可变类型和不可变类型？

(1) 不可变类型：指一旦创建，其值就不能被修改的类型。当对不可变类型的数据进行操作时，实际上是创建了一个新的对象。Python 中常见的不可变类型有：数字 (Number)：包括整数 (int)、浮点数 (float)、复数 (complex)；

(2) 可变类型：指在创建后，其值可以被修改的类型。对可变类型的数据进行操作时，是在原对象上进行修改，不会创建新的对象。Python 中常见的可变类型有：列表、字典、集合

2. Python进阶

2.1 解释装饰器的作用及典型应用场景

装饰器用于在不修改原函数代码的情况下扩展功能（如日志记录、性能统计、权限校验）。通过闭包实现，接收函数作为参数并返回增强后的函数。例如缓存装饰器可优化递归性能。

2.2 __new__ 和 __init__ 的区别是什么？

__new__是类方法，负责创建实例并返回；

__init__是实例方法，负责初始化实例属性。重写__new__常用于单例模式或不可变对象定制。

2.3 深拷贝与浅拷贝的区别是什么？

浅拷贝 (copy.copy) 仅复制对象外层引用，若对象包含可变元素（如嵌套列表），修改拷贝对象会影响原对象。深拷贝 (copy.deepcopy) 递归复制所有层级元素，完全独立于原对象。

2.4 什么是闭包？Python中如何实现？

闭包是嵌套函数捕获外层作用域变量的机制。例如计数器函数可通过闭包保留计数状态，而无需全局变量。

2.5 Python迭代器是什么？什么场景用到迭代器？举一个例子？

在Python中，迭代器 (Iterator) 是一个可以迭代访问序列元素的对象。迭代器对象实现了两个方法：__iter__() 和 __next__()。__iter__()方法返回迭代器对象本身，__next__()方法返回下一个元素。

迭代器常用于遍历序列、集合、字典等容器类型数据。它的优点是可以惰性计算 (lazy evaluation)，即只有在需要时才会计算，避免了一次性加载所有数据的开销，同时也可以节省内存空间。

在Python中，使用迭代器通常有以下场景：

(1) 遍历大量数据集：当需要处理大量的数据集时，使用迭代器可以避免一次性加载所有数据，节省内存空间。

(2) 实现自定义迭代器：当需要遍历自定义数据结构时，可以通过实现迭代器对象的__iter__()和__next__()方法实现自定义迭代器；

(3) 实现惰性计算：当需要进行惰性计算时，可以使用迭代器来实现，例如通过filter()、map()等高阶函数返回一个迭代器对象来进行惰性计算。

下面是一个使用迭代器遍历列表的例子：

```
1 my_list = [1, 2, 3, 4, 5]
2 my_iterator = iter(my_list)
3 while True:
4     try:
5         item = next(my_iterator)
6         print(item)
7     except StopIteration:
8         break
```

Python

复制代码

2.6 Python生成器是什么？什么场景用到生成器？举一个例子？

在Python中，生成器 (Generator) 是一种特殊的迭代器，它使用生成器函数来生成序列中的元素，而不是在内存中一次性生成所有元素。

生成器函数是使用yield关键字定义的函数，每次调用生成器函数时，它会返回一个迭代器对象，调用next()方法时，它会从上次暂停的位置继续执行，直到遇到下一个yield语句，然后返回一个值，并再次暂停。因

此，生成器可以惰性地生成序列中的元素，并在需要时逐个生成元素，避免了一次性生成所有元素所带来的内存消耗。

使用生成器的场景包括：

- (1) 生成大量的数据集：当需要生成大量数据时，使用生成器可以避免一次性占用大量内存空间；
- (2) 实现自定义的迭代器：当需要自定义迭代器对象时，可以使用生成器函数来实现，避免了繁琐的迭代器对象的定义；
- (3) 实现惰性计算：当需要进行惰性计算时，可以使用生成器来实现，例如通过filter()、map()等高阶函数返回一个生成器对象来进行惰性计算。

下面是一个使用生成器函数生成斐波那契数列的例子：

Python 复制代码

```
1 def fibonacci(n):
2     a, b = 0, 1
3     for i in range(n):
4         yield a
5         a, b = b, a + b
6 fib = fibonacci(10)
7 for num in fib:
8     print(num)
```

在这个例子中，我们定义了一个生成器函数fibonacci()，它的参数n表示需要生成的斐波那契数列的长度。在函数中，我们使用yield语句返回斐波那契数列中的每一个元素，这样每次调用next()函数时，它会返回下一个元素，并在下次调用时从上次暂停的位置继续执行。最后，我们使用for循环遍历生成器对象，并打印出每个元素。

2.7 Python多线程与多进程的区别

在UNIX平台上，当某个进程终结之后，该进程需要被其父进程调用wait，否则进程成为僵尸进程（Zombie）。所以，有必要对每个Process对象调用join()方法（实际上等同于wait）。对于多线程来说，由于只有一个进程，所以不存在此必要性。

多进程应该避免共享资源。在多线程中，我们可以比较容易地共享资源，比如使用全局变量或者传递参数。在多进程情况下，由于每个进程有自己独立的内存空间，以上方法并不合适。此时我们可以通过共享内存和Manager的方法来共享资源。但这样做提高了程序的复杂度，并因为同步的需要而降低了程序的效率。

2.8 装饰器的实质是什么？

或者说为什么装饰器要写2层嵌套函数，里层函数完全就已经实现了装饰的功能为什么不直接用里层函数名作为装饰器名称？

答：装饰器是要把原来的函数装饰成新的函数，并且返回这个函数本身的高阶函数

2.9 Python下多线程的限制以及多进程中传递参数的方式

python多线程有个全局解释器锁（global interpreter lock），这个锁的意思是任一时间只能有一个线程使用解释器，跟单cpu跑多个程序一个意思，大家都是轮着用的，这叫“并发”，不是“并行”。

多进程间共享数据，可以使用 multiprocessing.Value 和 multiprocessing.Array

2.10 Python是如何进行内存管理的？

Python引用了一个内存池(memory pool)机制，即Pymalloc机制(malloc:n.分配内存)，用于管理对小块内存的申请和释放内存池（memory pool）的概念：

当创建大量消耗小内存的对象时，频繁调用new/malloc会导致大量的内存碎片，致使效率降低。内存池的概念就是预先在内存中申请一定数量的，大小相等 的内存块留作备用，当有新的内存需求时，就先从内存池中分配内存给这个需求，不够了之后再申请新的内存。这样做最显著的优势就是能够减少内存碎片，提升效率。

内存池的实现方式有很多，性能和适用范围也不一样。python中的内存管理机制——Pymalloc：

python中的内存管理机制都有两套实现，一套是针对小对象，就是大小小于256bits时,pymalloc会在内存池中申请内存空间；当大于256bits，则会直接执行new/malloc的行为来申请内存空间。

关于释放内存方面，当一个对象的引用计数变为0时，python就会调用它的析构函数。在析构时，也采用了内存池机制，从内存池来的内存会被归还到内存池中，以避免频繁地释放动作。

2.11 Python里面如何拷贝一个对象？

标准库中的copy模块提供了两个方法来实现拷贝.一个方法是copy,它返回和参数包含内容一样的对象,使用deepcopy方法,对象中的属性也被复制

2.12 Python里面search()和match()的区别?

match()函数只检测re是不是在string的开始位置匹配, search()会扫描整个string查找匹配,也就是说match()只有在0位置匹配成功的话才有返回,如果不是开始位置匹配成功的话,match()就返回none。

2.13 lambda表达式格式以及应用场景?

lambda函数就是可以接受任意多个参数（包括可选参数）并且返回单个表达式值得函数。语法: lambda [arg1 [,arg2, argn]]:expression

应用:

- 1) lambda函数比较轻便,即用即仍,适合完成只在一处使用的简单功能。
- 2) 匿名函数,一般用来给filter, map这样的函数式编程服务
- 3) 作为回调函数,传递给某些应用,比如消息处理。

2.14 *args和**kwarg作用

*args代表位置参数,它会接收任意多个参数并把这些参数作为元组传递给函数。

**kwargs代表的关键字参数,允许你使用没有事先定义的参数名。位置参数一定要放在关键字参数的前面。

作用: 使用*args和**kwargs可以非常方便的定义函数,同时可以加强扩展性,以便日后的代码维护。

三、数据处理与统计分析

1. Mysql

1.1 SQL基础练习

创建数据表

```
CREATE TABLE employee
```

```
(  
    id INT,  
    NAME VARCHAR(20),  
    sex VARCHAR(10),  
    salary FLOAT  
);  
INSERT INTO employee VALUES(1,'zhangsan','male',2000),(2,'lisi','male',1000),(3,'xiaohong','female',4000);
```

```
CREATE TABLE student(
```

```
    id INT,  
    NAME VARCHAR(20),  
    chinese INT,  
    english INT,  
    math INT  
);  
INSERT INTO student(id, NAME, chinese, english, math)  
VALUES (1, '张小明', 89, 78, 90),(2, '李进', 67, 53, 95),(3, '王五', 87, 78, 77),(4, '李一', 88, 98, 92),(5, '李来财', 82, 84, 67),(6, '张进宝', 55, 85, 45),(7, '黄蓉', 75, 65, 30),(7, '黄蓉', 75, 65, 30);
```

部门表

```
CREATE TABLE DEPT(
```

```
    DEPTNO INT PRIMARY KEY, -- 部门编号  
    DNAME VARCHAR(14), -- 部门名称  
    LOC VARCHAR(13) -- 部门地址  
);
```

```
INSERT INTO DEPT VALUES (10,'ACCOUNTING','NEW YORK'),(20,'RESEARCH','DALLAS'),(30,'SALES','
```

```
CHICAGO'),(40,'OPERATIONS','BOSTON');
```

DQL数据查询

```
-- 统计每个学生的总分。

-- 在所有学生总分数上加10分特长分。

-- 使用别名表示学生分数。
-- 查询英语成绩大于90分的同学

-- 查询总分大于200分的所有同学

-- 查询英语分数在 80 - 90之间的同学。
-- 查询英语分数不在 80 - 90之间的同学。

-- 查询数学分数为89,90,91的同学。

-- 查询所有姓李的学生英语成绩。

-- 查询数学分90并且语文分89的同学。

-- 查询英语80或者总分200的同学
```

```
CREATE TABLE emp(
    empno      INT,
    ename      VARCHAR(50),
    job        VARCHAR(50),
    mgr        INT, -- 上级领导编号
    hiredate   DATE, -- 入职日期
    sal        INT,
    comm       INT, -- 奖金
    deptno     INT -- 部门编号
);
```

```
INSERT INTO emp VALUES(7369,'SMITH','CLERK',7902,'1980-12-17',800,NULL,20),(7499,'ALLEN','SALESMAN',7698,'1981-02-20',1600,300,30),(7521,'WARD','SALESMAN',7698,'1981-02-22',1250,500,30),(7566,'JONES','MANAGER',7839,'1981-04-02',2975,NULL,20),(7654,'MARTIN','SALESMAN',7698,'1981-09-28',12
```

```
50,1400,30),(7698,'BLAKE','MANAGER',7839,'1981-05-01',2850,NULL,30),(7782,'CLARK','MANAGER',7839,
'1981-06-09',2450,NULL,10),(7788,'SCOTT','ANALYST',7566,'1987-04-19',3000,NULL,20),(7839,'KING','PR
ESIDENT',NULL,'1981-11-17',5000,NULL,10),(7844,'TURNER','SALESMAN',7698,'1981-09-08',1500,0,30),(78
76,'ADAMS','CLERK',7788,'1987-05-23',1100,NULL,20),(7900,'JAMES','CLERK',7698,'1981-12-03',950,NULL,
30),(7902,'FORD','ANALYST',7566,'1981-12-03',3000,NULL,20),(7934,'MILLER','CLERK',7782,'1982-01-23',1
300,NULL,10);
```

- 1、按员工编号升序排列不在10号部门工作的员工信息
- 2、查询姓名第二个字母不是"A"且薪水大于800元的员工信息，按年薪降序排列
- 3、求每个部门的平均薪水
- 4、求各个部门的最高薪水
- 5、求每个部门每个岗位的最高薪水
- 6、求平均薪水大于2000的部门编号
- 7、将部门平均薪水大于1500的部门编号列出来，按部门平均薪水降序排列

-- 工资等级表

```
CREATE TABLE SALGRADE
```

```
(
```

```
    GRADE INT, -- 工资等级
```

```
    LOSAL DOUBLE, -- 最低工资
```

```
    HISAL DOUBLE ); -- 最高工资
```

```
INSERT INTO SALGRADE VALUES (1,700,1200),(2,1201,1400),(3,1401,2000),(4,2001,3000),(5,3001,9999);
```

-- 练习:

- 0、求最高薪水的员工信息
- 求大于平均薪水的员工信息
- 1、查询部门和所属的员工
- 2、工资水平多于smith的员工信息。
- 3、返回员工和所属上级的姓名。

- 4、返回雇员的雇佣日期早于其领导雇佣日期的员工及其领导姓名。
- 5、返回从事clerk工作的员工姓名和所在部门名称。
- 6、返回部门号,部门名称及其本部门的最低工资
- 7、返回销售部(sales)所有员工的姓名。
- 8、返回与SCOTT从事相同工作的员工。

1.2 SQL高级练习

创建数据表

```
-- 1.学生表
-- sid 学生编号,sname 学生姓名,sage 出生年月,ssex 学生性别
create table if not exists student
(
    sid    varchar(10),
    sname  varchar(50),
    sage   varchar(10),
    ssex   char(2)
);
-- 2.课程表
-- cid -- 课程编号,cname 课程名称,tid 教师编号
create table if not exists course
(
    cid    varchar(10),
    cname  varchar(50),
    tid    varchar(10)
);
-- 3.教师表
-- tid 教师编号,tname 教师姓名
create table if not exists teacher
(
    tid    varchar(10),
    tname  varchar(50)
);
-- 4.成绩表
```

```
-- sid 学生编号,cid 课程编号,score 分数
```

```
create table if not exists sc
```

```
(  
    sid    varchar(10),  
    cid    varchar(10),  
    score  double  
);
```

```
-- 创建测试数据
```

```
insert into student
```

```
values ('01', '赵雷', '1990-01-01', '男'),('02', '钱电', '1990-12-21', '男'),('03', '孙风', '1990-05-20', '男'),('04', '李云', '1990-08-06', '男'),('05', '周梅', '1991-12-01', '女'),('06', '吴兰', '1992-03-01', '女'),('07', '郑竹', '1989-07-01', '女'),('08', '王菊', '1990-01-20', '女');
```

```
insert into course
```

```
values ('01', '语文', '02'),('02', '数学', '01'),('03', '英语', '03');
```

```
insert into teacher
```

```
values ('01', '张三'),('02', '李四'),('03', '王五');
```

```
insert into sc
```

```
values ('01', '01', 80),('01', '02', 90),('01', '03', 99),('02', '01', 70),('02', '02', 60),('02', '03', 80),('03', '01', 80),('03', '02', 80),('03', '03', 80),('04', '01', 50),('04', '02', 30),('04', '03', 20),('05', '01', 76),('05', '02', 87),('06', '01', 31),('06', '03', 34),('07', '02', 89),('07', '03', 98);
```

DQL数据查询

```
# 1.查询"01"课程比"02"课程成绩高的学生的信息及课程分数
```

```
# 2.查询同时存在"01"课程和"02"课程的情况，有可能存在"01"课程但可能不存在"02"课程的学生，查询"01"课程比"02"课程成绩高的学生的信息及课程分数。
```

```
# 3.查询平均成绩大于等于60分的同学的学生编号和学生姓名和平均成绩
```

```
# 4.查询平均成绩小于60分的同学的学生编号和学生姓名和平均成绩
```

```
# 5.查询所有同学的学生编号、学生姓名、选课总数、所有课程的总成绩
```

```
# 6.查询"李"姓老师的数量（两种方法）
```

- # 7.查询没学过"张三"老师授课的同学的信息
- # 8.查询学过编号为"01"并且也学过编号为"02"的课程的同学的信息
- # 9. 查询没学过"张三"老师讲授的任一门课程的学生姓名
- # 10.查询两门及其以上不及格课程的同学的学号，姓名及其平均成绩
- # 11.查询任何一门课程成绩在70分以上的姓名、课程名称和分数；
- # 12.查询不及格的学生信息和课程名称
- # 13.查询课程编号为01且课程成绩在80分以上的学生的学号和姓名、成绩和课程名称
- # 14.检索至少选修两门课程的学生学号和学生姓名

2. Python数据分析

2.1 将下面的字典创建为DataFrame

```
data = {"grammer":["Python","C","Java","GO","R","SQL","PHP","Python"],  
        "score":[1,2,np.nan,4,5,6,7,10]}
```

期望结果

	grammer	score
0	Python	1.0
1	C	2.0
2	Java	NaN
3	GO	4.0
4	R	5.0
5	SQL	6.0
6	PHP	7.0
7	Python	10.0

答案：

2.2 提取含有字符串"Python"的行

期望结果

```
grammer  score
0 Python    1.0
7 Python   10.0
```

答案:

```
result=df[df['grammer'].str.contains("Python")]
```

2.3 输出df的所有列名

期望结果

```
Index(['grammer', 'score'], dtype='object')
```

答案

```
df.columns
```

2.4 修改第二列列名为'popularity'

答案

```
df.rename(columns={'score':'popularity'}, inplace = True)
```

2.5 统计grammer列中每种编程语言出现的次数

答案

```
df['grammer'].value_counts()
```

2.6 将空值用上下值的平均值填充

答案

```
df['popularity'] = df['popularity'].fillna(df['popularity'].interpolate())
```

2.7 提取popularity列中值大于3的行

答案

```
df[df['popularity'] > 3]
```

2.8 按照grammer列进行去重

答案

```
df.drop_duplicates(['grammer'])
```

2.9 计算popularity列平均值

答案

```
df['popularity'].mean()
```

2.10 将grammer列转换为list

答案

```
df['grammer'].to_list()
```

2.11 将DataFrame保存为EXCEL

答案

```
df.to_excel('filename.xlsx')
```

2.12 看数据行列数

答案

df.shape

2.13 提取popularity列值大于3小于7的行

答案

```
df[(df['popularity'] > 3) & (df['popularity'] < 7)]
```

2.14 交换两列位置

答案

'''

方法1

'''

```
temp = df['popularity']
df.drop(labels=['popularity'], axis=1,inplace = True)
df.insert(0, 'popularity', temp)
df
'''
```

方法2

```
cols = df.columns[[1,0]]
df = df[cols]
df
'''
```

2.15 提取popularity列最大值所在行

答案

```
df[df['popularity'] == df['popularity'].max()]
```

2.16 查看最后5行数据

答案

```
df.tail()
```

2.17 删除最后一行数据

答案

```
df.drop([len(df)-1],inplace=True)
```

2.18 添加一行数据['Perl',6.6]

答案

```
row={'grammar':'Perl','popularity':6.6}  
df = df.append(row,ignore_index=True)
```

2.19 对数据按照"popularity"列值的大小进行排序

答案

```
df.sort_values("popularity",inplace=True)
```

2.20 统计grammar列每个字符串的长度

答案

```
df['grammar'].map(lambda x: len(x))
```

2.21 读取本地EXCEL数据

答案

```
df = pd.read_excel('pandas120.xlsx')
```

2.22 查看df数据前5行

答案

```
df.head()
```

2.23 将salary列数据转换为最大值与最小值的平均值

答案

#为什么不能直接使用max, min函数，因为我们的数据中是20k-35k这种字符串，所以需要用正则表达式提取数字

```
import re
for i in range(len(df)):
    str1 = df.ix[i,2]
    k = re.findall(r"\d+\.\d*",str1)
    salary = ((int(k[0]) + int(k[1]))/2)*1000
    df.ix[i,2] = salary
df
```

2.24 将数据根据学历进行分组并计算平均薪资

期望输出

```
education salary
不限 19600.000000
大专 10000.000000
本科 19361.344538
硕士 20642.857143
```

答案

```
df.groupby('education').mean()
```

2.25 将createTime列时间转换为月-日

答案

```
for i in range(len(df)):
    df.ix[i,0] = df.ix[i,0].to_pydatetime().strftime("%m-%d")
df.head()
```

2.26 查看索引、数据类型和内存信息

期望输出

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 135 entries, 0 to 134
Data columns (total 4 columns):
createTime 135 non-null object
education 135 non-null object
salary 135 non-null int64
categories 135 non-null category
dtypes: category(1), int64(1), object(2)
memory usage: 3.5+ KB
```

答案

```
df.info()
```

2.27 查看数值型列的汇总统计

答案

```
df.describe()
```

2.28 新增一列根据salary将数据分为三组

期望输出

答案

```
bins = [0,5000, 20000, 50000]
group_names = ['低', '中', '高']
df['categories'] = pd.cut(df['salary'], bins, labels=group_names)
```

2.29 按照salary列对数据降序排列

答案

```
df.sort_values('salary', ascending=False)
```

2.30 取出第33行数据

答案

```
df.loc[32]
```

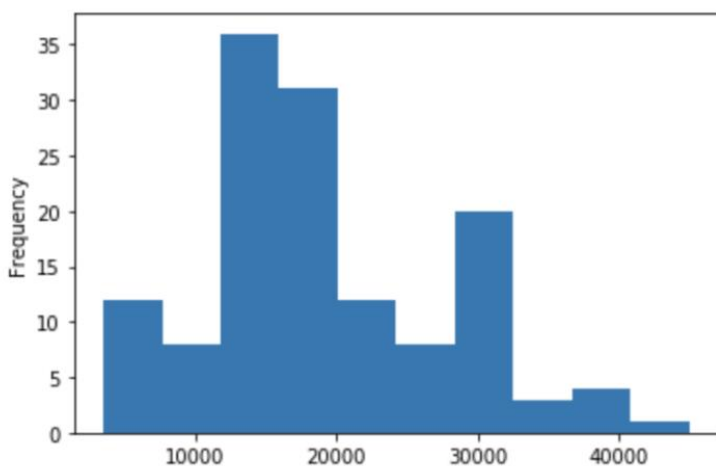
2.31 计算salary列的中位数

答案

```
np.median(df['salary'])
```

2.32 绘制薪资水平频率分布直方图

期望输出

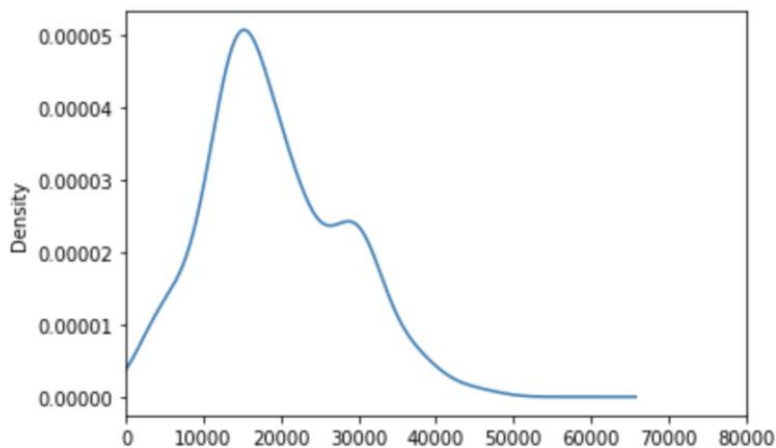


答案

```
df.salary.plot(kind='hist')
```

2.33 绘制薪资水平密度曲线

期望输出



答案

```
df.salary.plot(kind='kde',xlim=(0,80000))
```

2.34 删除最后一列categories

答案

```
del df['categories']
```

2.35 将df的第一列与第二列合并为新的一列

答案

```
df['test'] = df['education']+df['createTime']
```

2.36 将education列与salary列合并为新的一列

备注：salary为int类型，操作与35题有所不同

答案

```
df["test1"] = df["salary"].map(str) + df['education']
```

2.37 计算salary最大值与最小值之差

答案

```
df[['salary']].apply(lambda x: x.max() - x.min())
```

2.38 将第一行与最后一行拼接

答案

```
pd.concat([df[:1], df[-2:-1]])
```

2.39 将第8行数据添加至末尾

答案

```
df.append(df.iloc[7])
```

2.40 查看每列的数据类型

期望结果

createTime object

education object

salary int64

test object

test1 object

dtype: object

答案

```
df.dtypes
```

2.41 将createTime列设置为索引

答案

```
df.set_index("createTime")
```

2.42 生成一个和df长度相同的随机数dataframe

答案

```
df1 = pd.DataFrame(pd.Series(np.random.randint(1, 10, 135)))
```

2.43 将上一题生成的dataframe与df合并

答案

```
df= pd.concat([df,df1],axis=1)
```

2.44 生成新的一列new为salary列减去之前生成随机数列

答案

```
df["new"] = df["salary"] - df[0]
```

2.45 检查数据中是否含有任何缺失值

答案

```
df.isnull().values.any()
```

2.46 将salary列类型转换为浮点数

答案

```
df['salary'].astype(np.float64)
```

2.47 计算salary大于10000的次数

答案

```
len(df[df['salary']>10000])
```

2.48 查看每种学历出现的次数

期望输出

本科 119

硕士 7

不限 5

大专 4

Name: education, dtype: int64

答案

```
df.education.value_counts()
```

2.49 查看education列共有几种学历

答案

```
df['education'].nunique()
```

2.50 提取salary与new列的和大于60000的最后3行

期望输出

	createTime	education	salary	test	test1	0	new
92	03-16	本科	35000	本科03-16	35000本科	6	34994
101	03-16	本科	37500	本科03-16	37500本科	5	37495
131	03-16	硕士	37500	硕士03-16	37500硕士	6	37494

答案

```
df1 = df[['salary','new']]
rowsums = df1.apply(np.sum, axis=1)
res = df.iloc[np.where(rowsums > 60000)[0][-3:], :]
```

2.51 使用绝对路径读取本地Excel数据

答案

```
data = pd.read_excel('600000.SH.xls')
```

2.52 查看数据前三行

期望结果

	代码	简称	日期	前收盘价(元)	开盘价(元)	最高价(元)	最低价(元)	收盘价(元)	成交量(股)	成交金额(元)	涨跌(元)	涨跌幅(%)	均价(元)	换手率(%)	A股流通市值(元)	总市值(元)
0	600000.SH	浦发银行	2016-01-04	16.1356	16.1444	16.1444	15.4997	15.7205	42240610	754425783	-0.4151	-2.5725	17.8602	0.2264	3.320318e+11	3.320318e+11
1	600000.SH	浦发银行	2016-01-05	15.7205	15.4644	15.9501	15.3672	15.8618	58054793	1034181474	0.1413	0.8989	17.8139	0.3112	3.350163e+11	3.350163e+11
2	600000.SH	浦发银行	2016-01-06	15.8618	15.8088	16.0208	15.6234	15.9855	46772653	838667398	0.1236	0.7795	17.9307	0.2507	3.376278e+11	3.376278e+11

答案

```
data.head(3)
```

2.53 查看每列数据缺失值情况

期望结果

```
代码 1
简称 2
日期 2
前收盘价(元) 2
开盘价(元) 2
最高价(元) 2
最低价(元) 2
```

收盘价(元) 2
成交量(股) 2
成交金额(元) 2
.....

答案

```
data.isnull().sum()
```

2.54 提取日期列含有空值的行

期望结果

	代码	简称	日期	前收盘价(元)	开盘价(元)	最高价(元)	最低价(元)	收盘价(元)	成交量(股)	成交金额(元)	涨跌(元)	涨跌幅(%)	均价(元)	换手率(%)	A股流通市值(元)	总市值(元)	A股流通股本(股)	市盈率
327	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
328	数据来源: Wind资讯	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

答案

```
data[data['日期'].isnull()]
```

2.55 输出每列缺失值具体行数

期望结果

列名: "代码", 第[327]行位置有缺失值
列名: "简称", 第[327, 328]行位置有缺失值
列名: "日期", 第[327, 328]行位置有缺失值
列名: "前收盘价(元)", 第[327, 328]行位置有缺失值
列名: "开盘价(元)", 第[327, 328]行位置有缺失值
列名: "最高价(元)", 第[327, 328]行位置有缺失值
列名: "最低价(元)", 第[327, 328]行位置有缺失值
列名: "收盘价(元)", 第[327, 328]行位置有缺失值
.....

答案

```
for columnname in data.columns:
```

```
if data[columnname].count() != len(data):  
    loc = data[columnname][data[columnname].isnull().values==True].index.tolist()  
    print('列名: "{}", 第{}行位置有缺失值'.format(columnname,loc))
```

2.56 删除所有存在缺失值的行

答案

```
data.dropna(axis=0, how='any', inplace=True)
```

备注

axis: 0-行操作（默认），1-列操作

how: any-只要有空值就删除（默认），all-全部为空值才删除

inplace: False-返回新的数据集（默认），True-在原数据集上操作

2.57 绘制收盘价的折线图

期望结果



答案

```
data['收盘价(元)'].plot()
```

2.58 同时绘制开盘价与收盘价

期望结果

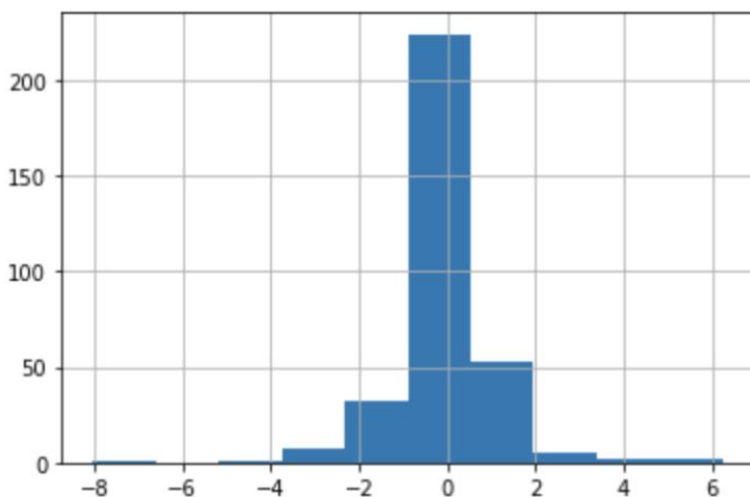


答案

```
data[['收盘价(元)', '开盘价(元)']].plot()
```

2.59 绘制涨跌幅的直方图

期望结果

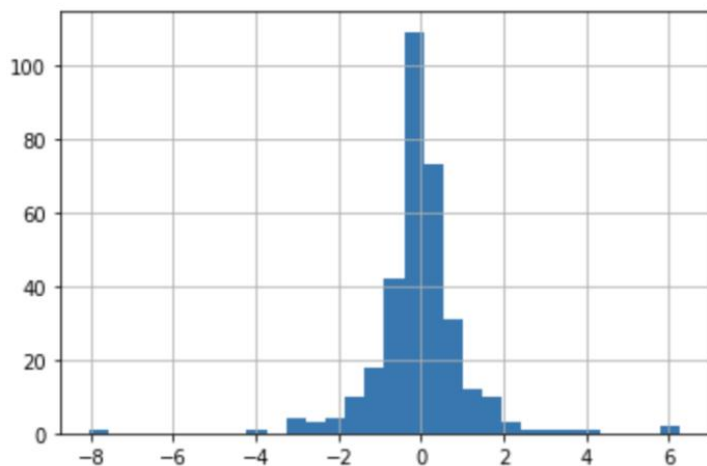


答案

```
data['涨跌幅(%)'].hist()
```

2.60 让直方图更细致

期望结果



答案

```
data['涨跌幅(%)'].hist(bins = 30)
```

2.61 以data的列名创建一个dataframe

答案

```
temp = pd.DataFrame(columns = data.columns.to_list())
```

2.62 打印所有换手率不是数字的行

期望结果

代码	简称	日期	前收盘价(元)	开盘价(元)	最高价(元)	最低价(元)	收盘价(元)	成交量(股)	成交金额(元)	涨跌(元)	涨跌幅(%)	均价(元)	换手率(%)	A股流通市值(元)	总市值(元)	A股流通股本(股)	市盈率
26	600000.SH	浦发银行 2016-02-16	16.2946	16.2946	16.2946	16.2946	16.2946	--	--	0.0	0.0	--	--	3.441565e+11	3.441565e+11	1.865347e+10	6.801
27	600000.SH	浦发银行 2016-02-17	16.2946	16.2946	16.2946	16.2946	16.2946	--	--	0.0	0.0	--	--	3.441565e+11	3.441565e+11	1.865347e+10	6.801
28	600000.SH	浦发银行 2016-02-18	16.2946	16.2946	16.2946	16.2946	16.2946	--	--	0.0	0.0	--	--	3.441565e+11	3.441565e+11	1.865347e+10	6.801

答案

```
for i in range(len(data)):
    if type(data.iloc[i,13]) != float:
        temp = temp.append(data.loc[i])
```

temp

2.63 打印所有换手率为--的行

答案

```
data[data['换手率(%)'].isin(['--'])]
```

2.64 重置data的行号

答案

```
data = data.reset_index()
```

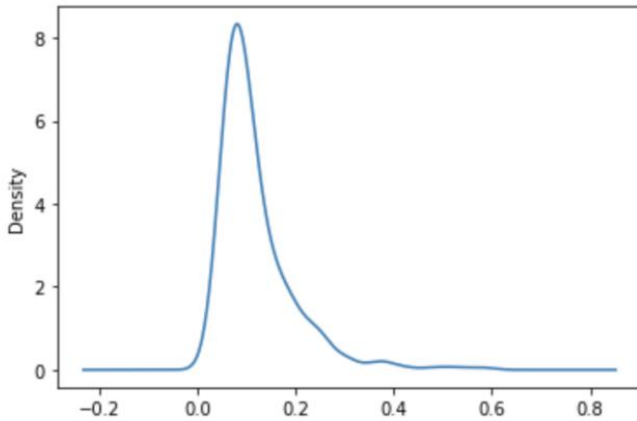
2.65 删除所有换手率为非数字的行

答案

```
k = []
for i in range(len(data)):
    if type(data.iloc[i,13]) != float:
        k.append(i)
data.drop(labels=k,inplace=True)
```

2.66 绘制换手率的密度曲线

期望结果



答案

```
data['换手率(%)'].plot(kind='kde')
```

2.67 计算前一天与后一天收盘价的差值

答案

```
data['收盘价(元)'].diff()
```

2.68 计算前一天与后一天收盘价变化率

答案

```
data['收盘价(元)'].pct_change()
```

2.69 设置日期为索引

答案

```
data.set_index('日期')
```

2.70 以5个数据作为一个数据滑动窗口，在这个5个数据上取均值(收盘价)

答案

```
data['收盘价(元)'].rolling(5).mean()
```

2.71 以5个数据作为一个数据滑动窗口，计算这五个数据总和(收盘价)

答案

```
data['收盘价(元)'].rolling(5).sum()
```

2.72 将收盘价5日均线、20日均线与原始数据绘制在同一个图上

期望结果



答案

```
data['收盘价(元)'].plot()  
data['收盘价(元)'].rolling(5).mean().plot()  
data['收盘价(元)'].rolling(20).mean().plot()
```

2.73 按周为采样规则，取一周收盘价最大值

答案

```
data['收盘价(元)'].resample('W').max()
```

2.74 绘制重采样数据与原始数据

期望结果



答案

```
data['收盘价(元)'].plot()  
data['收盘价(元)'].resample('7D').max().plot()
```

2.75 将数据往后移动5天

答案

```
data.shift(5)
```

2.76 将数据向前移动5天

答案

```
data.shift(-5)
```

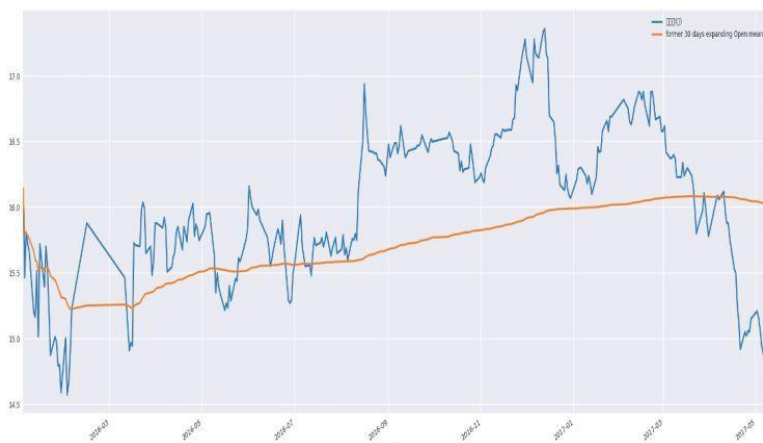
2.77 使用expanding函数计算开盘价的移动窗口均值

答案

```
data['开盘价(元)'].expanding(min_periods=1).mean()
```

2.78 绘制上一题的移动均值与原始数据折线图

期望结果



答案

```
data['expanding Open mean']=data['开盘价(元)'].expanding(min_periods=1).mean()
data[['开盘价(元)', 'expanding Open mean']].plot(figsize=(16, 6))
```

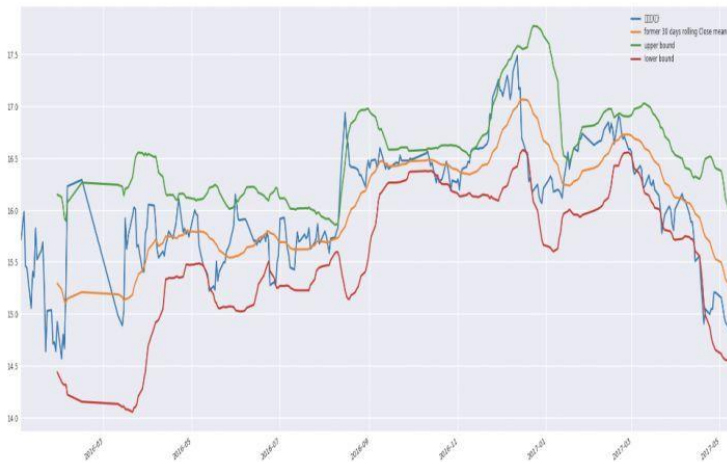
2.79 计算布林指标

答案

```
data['former 30 days rolling Close mean']=data['收盘价(元)'].rolling(20).mean()
data['upper bound']=data['former 30 days rolling Close mean']+2*data['收盘价(元)'].rolling(20).std()
#在这里我们取20天内的标准差
data['lower bound']=data['former 30 days rolling Close mean']-2*data['收盘价(元)'].rolling(20).std()
```

2.80 计算布林线并绘制

期望结果



答案

```
data[['收盘价(元)', 'former 30 days rolling Close mean', 'upper bound', 'lower bound']].plot(figsize=(16, 6))
```

2.81 导入并查看pandas与numpy版本

答案

```
import pandas as pd
import numpy as np
print(np.__version__)
print(pd.__version__)
```

2.82 使用numpy生成20个0-100随机数

答案

```
tem = np.random.randint(1,100,20)
df1 = pd.DataFrame(tem)
```

2.83 使用numpy生成20个0-100固定步长的数

答案

```
tem = np.arange(0,100,5)
df2 = pd.DataFrame(tem)
```

2.84 使用numpy生成20个指定分布(如标准正态分布)的数

答案

```
tem = np.random.normal(0, 1, 20)
df3 = pd.DataFrame(tem)
```

2.85 将df1, df2, df3按照行合并为新DataFrame

答案

```
df = pd.concat([df1,df2,df3],axis=0,ignore_index=True)
```

2.86 将df1, df2, df3按照列合并为新DataFrame

期望结果

```
0 1 2
0 95 0 0.022492
1 22 5 -1.209494
2 3 10 0.876127
3 21 15 -0.162149
4 51 20 -0.815424
5 30 25 -0.303792
.....
```

答案

```
df = pd.concat([df1,df2,df3],axis=1,ignore_index=True)
df
```

2.87 查看df所有数据的最小值、25%分位数、中位数、75%分位数、最大值

答案

```
print(np.percentile(df, q=[0, 25, 50, 75, 100]))
```

2.88 修改列名为col1,col2,col3

答案

```
df.columns = ['col1','col2','col3']
```

2.89 提取第一列中不在第二列出现的数字

答案

```
df['col1'][~df['col1'].isin(df['col2'])]
```

2.90 提取第一列和第二列出现频率最高的三个数字

答案

```
temp = df['col1'].append(df['col2'])  
temp.value_counts().index[:3]
```

2.91 提取第一列中可以整除5的数字位置

答案

```
np.argwhere(df['col1'] % 5 == 0)
```

2.92 计算第一列数字前一个与后一个的差值

答案

```
df['col1'].diff().tolist()
```

2.93 将col1,col2,clo3三列顺序颠倒

答案

```
df.ix[:, ::-1]
```

2.94 提取第一列位置在1,10,15的数字

答案

```
df['col1'].take([1,10,15])
```

2.95 查找第一列的局部最大值位置

备注

即比它前一个与后一个数字的都大的数字

答案

```
tem = np.diff(np.sign(np.diff(df['col1'])))  
np.where(tem == -2)[0] + 1
```

2.96 按行计算df的每一行均值

答案

```
df[['col1','col2','col3']].mean(axis=1)
```

2.97 对第二列计算移动平均值

备注

每次移动三个位置，不可以使用自定义函数

答案

```
np.convolve(df['col2'], np.ones(3)/3, mode='valid')
```

2.98 将数据按照第三列值的大小升序排列

答案

```
df.sort_values("col3",inplace=True)
```

2.99 将第一列大于50的数字修改为'高'

答案

```
df.col1[df['col1'] > 50]= '高'
```

2.100 计算第一列与第二列之间的欧式距离

备注

不可以使用自定义函数

答案

```
np.linalg.norm(df['col1']-df['col2'])
```

2.101 从CSV文件中读取指定数据

备注

从数据1中的前10行中读取positionName, salary两列

答案

```
df = pd.read_csv('数据1.csv',encoding='gbk', usecols=['positionName', 'salary'],nrows = 10)
```

2.102 从CSV文件中读取指定数据

备注

从数据2中读取数据并在读取数据时将薪资大于10000的为改为高

答案

```
df = pd.read_csv('数据2.csv',converters={'薪资水平': lambda x: '高' if float(x) > 10000 else '低'})
```

2.103 从dataframe提取数据

备注

从上一题数据中，对薪资水平列每隔20行进行一次抽样

期望结果

薪资水平	
0	高
20	高
40	高
60	高
80	高
100	高
120	高
140	高
160	高
180	高

答案

```
df.iloc[::20, :][['薪资水平']]
```

2.104 将数据取消使用科学计数法

输入

```
df = pd.DataFrame(np.random.random(10)**10, columns=['data'])
```

期望结果

data	
0	0.078
1	0.029
2	0.002
3	0.000
4	0.000
5	0.000
6	0.007
7	0.000
8	0.000
9	0.004

答案

```
df.round(3)
```

2.105 将上一题的数据转换为百分数

期望结果

data	
0	7.75%
1	2.94%
2	0.22%
3	0.00%
4	0.00%
5	0.00%
6	0.65%
7	0.01%
8	0.00%
9	0.38%

答案

```
df.style.format({'data': '{0:.2%}'.format})
```

2.106 查找上一题数据中第3大值的行号

答案

```
df['data'].argsort()[::-1][7]
```

2.107 反转df的行

答案

```
df.iloc[::-1, :]
```

2.108 按照多列对数据进行合并

输入

```
df1= pd.DataFrame({'key1': ['K0', 'K0', 'K1', 'K2'],  
'key2': ['K0', 'K1', 'K0', 'K1'],  
'A': ['A0', 'A1', 'A2', 'A3'],  
'B': ['B0', 'B1', 'B2', 'B3']})
```

```
df2= pd.DataFrame({'key1': ['K0', 'K1', 'K1', 'K2'],  
'key2': ['K0', 'K0', 'K0', 'K0'],  
'C': ['C0', 'C1', 'C2', 'C3'],  
'D': ['D0', 'D1', 'D2', 'D3']})
```

答案

```
pd.merge(df1, df2, on=['key1', 'key2'])
```

2.109 按照多列对数据进行合并

备注

只保存df1的数据

答案

```
pd.merge(df1, df2, how='left', on=['key1', 'key2'])
```

2.110 再次读取数据1并显示所有的列

备注

数据中由于列数较多中间列不显示

答案

```
df = pd.read_csv('数据1.csv',encoding='gbk')
pd.set_option("display.max.columns", None)
df
```

2.111 查找secondType与thirdType值相等的行号

答案

```
np.where(df.secondType == df.thirdType)
```

2.112 查找薪资大于平均薪资的第三个数据

答案

```
np.argwhere(df['salary'] > df['salary'].mean())[2]
```

2.113 将上一题数据的salary列开根号

答案

```
df[['salary']].apply(np.sqrt)
```

2.114 将上一题数据的linestaion列按_拆分

答案

```
df['split'] = df['linestaion'].str.split('_')
```

2.115 查看上一题数据中一共有多少列

答案

```
df.shape[1]
```

2.116 提取industryField列以'数据'开头的行

答案

```
df[df['industryField'].str.startswith('数据')]
```

2.117 按列制作数据透视表

答案

```
pd.pivot_table(df,values=["salary","score"],index="positionId")
```

2.118 同时对salary、score两列进行计算

答案

```
df[["salary","score"]].agg([np.sum,np.mean,np.min])
```

2.119 对不同列执行不同的计算

备注

对salary求平均，对score列求和

答案

```
df.agg({"salary":np.sum,"score":np.mean})
```

2.120 计算并提取平均薪资最高的区

答案

```
df[['district','salary']].groupby(by='district').mean().sort_values('salary',ascending=False).head(1)
```

3. Linux

3.1 Linux常用命令

答：find、df、tar、ps、top、netstat等。（尽量说一些高级命令）

3.2 Linux查看内存、磁盘存储、io 读写、端口占用、进程等命令

答：

- 1) 查看内存：top
- 2) 查看磁盘存储情况：df -h
- 3) 查看磁盘IO读写情况：iotop（需要安装一下：yum install iotop）、iotop -o（直接查看输出比较高的磁盘读写程序）
- 4) 查看端口占用情况：netstat -tunlp | grep 端口号
- 5) 查看进程：ps aux

3.3 使用Linux命令查询file1中空行所在的行号

3.4 有文件chengji.txt内容如下：

张三 40

李四 50

王五 60

使用Linux命令计算第二列的和并输出



Shell

 复制代码

```
1  答案:
2  [root@node1 datas]$ awk '/^$/ {print NR}' file1.txt
3  5
```



Shell

 复制代码

```
1  [root@node1 datas]$ cat chengji.txt | awk -F " " '{sum+=$2} END{print sum}'
2  150
```

3.5 Shell脚本里如何检查一个文件是否存在？如果不存在如何处理？



Shell

 复制代码

```
1  #!/bin/bash
2  if [ -f file.txt ]; then
3      echo "文件存在!"
4  else
5      echo "文件不存在!"
6  fi
```

3.6 用Shell写一个脚本，对文本中无序的一系列数字排序

```
1 [root@hadoop102 ~]# cat test.txt # 1-10
2 [root@hadoop102 ~]# sort -n test.txt|awk '{a+=$0;print $0}END{print "SUM"
  = "a}"
3 SUM=55
```

3.7 Shell脚本写出查找当前文件夹 (/home) 下所有的文本文件内容中包含有字符“shen”的文件名称

```
1 [root@hadoop102 datas]$ grep -r "shen" /home | cut -d ":" -f 1
2 /home/root/datas/sed.txt
3 /home/root/datas/cut.txt
```

四、机器学习

1.1 机器学习中特征的理解

def: 特征选择和降维

特征选择: 原有特征选择出子集, 不改变原来的特征空间

降维: 将原有的特征重组成为包含信息更多的特征, 改变了原有的特征空间

降维的主要方法

- Principal Component Analysis (主成分分析)

- Singular Value Decomposition (奇异值分解)

特征选择的方法

- Filter 方法：卡方检验、信息增益、相关系数
- Wrapper 方法：其主要思想是：将子集的选择看作是一个搜索寻优问题，生成不同的组合，对组合进行评价，再与其他的组合进行比较。这样就将子集的选择看作是一个是一个优化问题，这里有很多的优化算法可以解决，尤其是一些启发式的优化算法，如 GA, PSO, DE, ABC 等，详见“优化算法 —— 人工蜂群算法 (ABC)”，“优化算法 —— 粒子群算法 (PSO)”。
- Embedded 方法：其主要思想是：在模型既定的情况下学习出对提高模型准确性最好的属性。这句话并不是很好理解，其实是讲在确定模型的过程中，挑选出那些对模型的训练有重要意义的属性。
主要方法：正则化。岭回归就是在基本线性回归的过程中加入了正则项。

1.2 机器学习中，有哪些特征工程方法？

数据和特征决定了机器学习的上限，而模型和算法只是逼近这个上限而已

(1) 计算每一个特征与相应变量的相关性：工程上常用的手段有计算皮尔逊系数和互信息系数，皮尔逊系数只能衡量线性相关性而互信息系数能够很好地度量各种相关性，但是计算相对复杂一些，好在很多toolkit里边都包含了这个工具（如 sklearn的MINE），得到相关性之后就可以排序选择特征了；

(2) 构建单个特征的模型，通过模型的准确性为特征排序，借此来选择特征；

(3) 通过L1正则项来选择特征：L1正则方法具有稀疏解的特性，因此天然具备特征选择的特性，但是要注意，L1没有选到的特征不代表不重要，原因是两个具有高相关性的特征可能只保留了一个，如果要确定哪个特征重要应再通过L2

正则方法交叉检验*；

(4) 训练能够对特征打分的预选模型：RandomForest和Logistic Regression等都能对模型的特征打分，通过打分获得相关性后再训练最终模型；

(5) 通过特征组合后再来选择特征：如对用户id和用户特征最组合来获得较大的特征集再来选择特征，这种做法在推荐系统和广告系统中比较常见，这也是所谓亿级甚至十亿级特征的主要来源，原因是用户数据比较稀疏，组合特征能够同时兼顾全局模型和个性化模型，这个问题有机会可以展开讲。

(6) 通过深度学习来进行特征选择：目前这种手段正在随着深度学习的流行而成为一种手段，尤其是在计算机视觉领域，原因是深度学习具有自动学习特征的能力，这也是深度学习又叫unsupervised feature learning的原因。从深度学习模型中选择某一神经层的特征后就可以用来进行最终目标模型的训练了。

1.3 机器学习中的正负样本

在分类问题中，这个问题相对好理解一点，比如人脸识别中的例子，正样本很好理解，就是人脸的图片，负样本的选取就与问题场景相关，具体而言，如果你要进行教室中学生的人脸识别，那么负样本就是教室的窗子、墙等等，也就是说，不能是与你要研究的问题毫不相关的乱七八糟的场景图片，这样的负样本并没有意义。负样本可以根据背景生成，

有时候不需要寻找额外的负样本。一般3000-10000的正样本需要5,000,000-100,000,000的负本来学习，在互金领域一般在入模前将正负比例通过采样的方法调整到3) 1-5:1。

1.4 线性分类器与非线性分类器的区别及优劣

区别：所谓线性分类器即用一个超平面将正负样本分离开，表达式为 $y=wx$ 。这里强调的是平面。

而非线性的分类界面没有这个限制，可以是曲面，多个超平面的组合等。典型的线性分类器有感知机，LDA，逻辑斯特回归，SVM（线性核）；

典型的非线性分类器有朴素贝叶斯，kNN，决策树，SVM（非线性核）

优缺点：1.线性分类器判别简单、易实现、且需要的计算量和存储量小。

为解决比较复杂的线性不可分样本分类问题，提出非线性判别函数。：超曲面，非线性判别函数计算复杂，实际应用上受到较大的限制。在线性分类器的基础上，用分段线性分类器可以实现复杂的分类面。解决问题比较简便的方法是采用多个线性分界面将它们分段连接，用分段线性判别划分去逼近分界的超曲面。

如果一个问题是非线性问题并且它的类边界不能够用线性超平面估计得很好，那么非线性分类器通常会比线性分类器表现得更精准。如果一个问题是有线性的，那么最好使用简单的线性分类器来处理。

1.5 如何解决过拟合问题

解释过拟合：模型在训练集表现好，在真实数据表现不好，即模型的泛化能力不够。从另外一个方面来讲，模型在达到经验损失最小的时候，模型复杂度较高，结构风险没有达到最优。

解决：

学习方法上：限制机器的学习，使机器学习特征时学得不那么彻底，因此这样就可以降低机器学到局部特征和错误特征的几率，使得识别正确率得到优化。

数据上：要防止过拟合，做好特征的选取。训练数据的选取也是很关键的，良好的训练数据本身的局部特征应尽可能少，噪声也尽可能小。

1.6 L1和L2正则的区别，如何选择L1和L2正则

L0正则化的值是模型参数中非零参数的个数。

$$\|x\|_0 = \#(i) \text{ with } x_i \neq 0$$

也就是如果我们使用L0范数，即希望w的大部分元素都是0。（w是稀疏的）所以可以用于ML中做稀疏编码，特征选择。通过最小化L0范数，来寻找最少最优的稀疏特征项。但不幸的是，L0范数的最优化问题是一个NP hard问题，而且理论上证明，L1范数是L0范数的最优凸近似，因此通常使用L1范数来代替。

L1正则化表示各个参数绝对值之和。

$$\|x\|_1 = \sum_{i=1}^n |x_i|$$

L1范数的解通常是稀疏性的，倾向于选择数目较少的一些非常大的值或者数目较多的insignificant的小值。L2正则化标识各个参数的平方的和的开方值。

L2范数即欧氏距离：

$$\|x\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$$

L2范数越小，可以使得w的每个元素都很小，接近于0，但L1范数不同的是他不会让它等于0而是接近于0。

1.7 有监督学习和无监督学习的区别

有监督学习：对具有概念标记（分类）的训练样本进行学习，以尽可能对训练样本集外的数据进行标记（分类）预测。这里，所有的标记（分类）是已知的。因此，训练样本的歧义性低。监督学习中只要输入样本集，机器就可以从中推演出制定目标变量的可能结果。如同过滤推荐算法，通过对训练集进行监督学习，并对测试集进行预测，从而达到预测的目的。

无监督学习：对没有概念标记（分类）的训练样本进行学习，以发现训练样本集中的结构性知识。这里，所有的标记（分类）是未知的。因此，训练样本的歧义性高。聚类就是典型的无监督学习。

监督学习的典型例子就是决策树、神经网络以及疾病监测，而无监督学习就是很早之前的西洋双陆棋和聚类。

1.8 有了解过哪些机器学习的算法？

问题分析：

考官主要考察的是学员是否对于人工智能方面的感兴趣，是否有空闲时间了解过机器学习算法这方面的知

识，有则更好。

核心答案讲解：

学员需要根据自己的实际情况去回答，学员也可以自己课下空余的时间去掌握一两常用的机器学习算法，以此来增加自己的筹码。

决策树模型：

构建决策树三要素

1-特征选择

信息熵---信息增益---信息增益大的

$$H(x) = -\sum (p_i \log(p_i))$$

$$\text{Gain}(A) = \text{Info}(D) - \text{Info}_A(D)$$

43Gini：Gini系数是对信息熵关于 $\ln(x)$ 在 $x=0$ 处的一阶泰勒展开近似得到是信息Gini系数

2-决策树的构成

ID3算法：

算法输入：特征和样本构成的数据集

算法输出：ID3的决策树

算法步骤：

(1) 如果所有的属性都被处理完毕

(2) 计算各个节点的信息增益，选择最大的信息增益所代表的特征构建决策树(信息熵

-- $\sum(p_i \log(p_i))$)

(3) 如果上述的属性没有处理完毕，需要从剩余的属性中继续获取其他的信息增益较大的值对应的特征。

(4) 递归构建决策树算法模型

算法优化

选择信息增益率--C4.5算法

算法的停止迭代条件：

(1) 迭代次数

(2) 树的深度

(3) 树的叶子节点的个数

(4) 树的分支节点含有样本的个数

(5) 树的最大不纯度的下降

C4.5

Cart树---gini系数

3-决策树剪枝

先剪枝

后剪枝

问题扩展

在算法的推导过程中，会大量的使用到数学的推导公式，而数学思想对于程序员来说是必不可缺的一种思想，可以在讲解的时候，概括性的给学员讲一下数学的相关知识。

信息熵数学公式($\sum(p_i \cdot \log(p_i))$) + 信息增益数学公式(总体的信息熵 - 以A节点作为划分节点的信息熵)

结合项目中使用

此问题在项目中的是没有什么体现的，面试官就是想考察一下面试者本身具有的学习能力，以此来评估该面试者的可培养价值，以及潜力。但是机器学习算法在推荐系统项目中、以后数据挖掘工作中会起到至关重要的作用。

1.9 你对LR是怎么理解的？LR的损失函数是怎样的？

首先,LR是逻辑斯特回归，是分类算法。

LR的原理是在线性回归的基础上增加一个非线性函数，这里用的是sigmoid函数，得到的结果是以概率的形式输出类别的结果。

LR的损失函数是似然函数，似然函数的log对数的形式，得到是交叉熵损失，对应的是分类中最重要交叉熵损失函数。

交叉熵损失和其他的损失函数区别就在于交叉熵损失是定义在概率分布基础上的，它通常是被用于基于多项式的逻辑斯 特回归和神经网络中，还有用在EM算法中。对数损失是用来度量分类器的预测输出的概率分布和真是分布的差距，而不是比较离散类标签是否相等。

1.10 二分类模型如何做多分类？

决策树中ID3和C4.5算法都是可以解决多分类问题，而logisticModel、SVM只能解决二分类问题，如何将二分类问题拓

展到多分类上，使用常用的方法有两种：

Ovo-One vs one：通过将任意两个分类的组合形式来进行多分类

Ovr-One vs rest：C (n,2) f分类结果

<http://scikit-learn.org/stable/modules/classes.html>

1.11 KMeans算法和KNN算法区别？

KMeans算法是聚类的算法，KNN是一个分类算法

KNN是通过K值设定最近邻的个数，通过当前样本点到各个点之间的距离的求解，通过少数服从多数的投票原则划分当前样本点

KMeans通过K值设定有几个聚类中心，通过将各个样本点到聚类中心的距离，将离中心点最小距离的样本进行组合，从而将样本点分成不同的簇，通过均值的方法迭代聚类中心，直到迭代次数或mse误差或聚类中心不在改变停止迭代。

1.12 请问GBDT和XGBoost的区别是什么？

XGBoost类似于GBDT的优化版，不论是精度还是效率上都有了提升。与GBDT相比，具体的优点有：

损失函数是用泰勒展式二项逼近，而不是像GBDT里的就是一阶导数；对树的结构进行了正则化约束，防止模型过度复杂，降低了过拟合的可能性；节点分裂的方式不同，GBDT是用的基尼系数，XGBoost是经过优化推导后的分裂增益。

1.13 LR和SVM的联系与区别？

相同点：

(1) LR和SVM都可以处理分类问题，且一般都用于处理线性二分类问题（在改进的情况下可以处理多分类问题）

(2) 两个方法都可以增加不同的正则化项，如L1、L2等等。所以在很多实验中，两种算法的结果是很接近的。

区别：

(1) LR是参数模型，SVM是非参数模型。

(2) 从目标函数来看，区别在于逻辑回归采用的是Logistical Loss，SVM采用的是hinge loss.这两个损失函数的目的都是增加对分类影响较大的数据点的权重，减少与分类关系较小的数据点的权重。

(3) SVM的处理方法是只考虑Support Vectors，也就是和分类最相关的少数点，去学习分类器。而逻辑回归通过非线性映射，大大减小了离分类平面较远的点的权重，相对提升了与分类最相关的数据点的权重。

(4) 逻辑回归相对来说模型更简单，好理解，特别是大规模线性分类时比较方便。而SVM的理解和优化相对来说复杂一些，SVM转化为对偶问题后，分类只需要计算与少数几个支持向量的距离，这个在进行复

杂核函数计算时优势 很明显,能够大大简化模型和计算。

(5) Logic能做的SVM能做, 但可能在准确率上有问题, SVM能做的Logic有的做不了。

1.14 LR与线性回归的区别与联系?

逻辑回归的模型本质上是一个线性回归模型, 逻辑回归都是以线性回归为理论支持的。但线性回归模型无法做到sigmoid 的非线性形式, sigmoid可以轻松处理0/1分类问题。

1.15 为什么XGBoost要用泰勒展开, 优势在哪里?

XGBoost 使用泰勒展开 (尤其是二阶泰勒展开) 是其核心优化策略之一, 主要优势在于 提升模型精度、加速收敛、增强灵活性并统一损失函数形式。以下是具体分析:

核心优势解析

1.引入二阶信息, 提升优化精度

传统 GBDT 仅使用一阶梯度 (损失函数的一阶导数), 而 XGBoost 通过二阶泰勒展开引入Hessian 矩阵 (二阶导数), 同时考虑梯度和曲率信息。二阶导数能更精准地描述损失函数的局部变化, 指导参数更新方向, 使梯度下降更快、更稳定 (类似牛顿法原理)。

2.统一损失函数形式, 支持灵活自定义

(1) 问题背景: 不同损失函数 (如 MSE、Log Loss) 的原始形式差异大。例如, MSE 的梯度天然包含一阶项 (残差) 和二阶项, 而 Log Loss 没有类似结构。

(2) 泰勒展开的作用: 将任意二阶可导的损失函数在当前预测值处展开, 统一表示为:

$$\text{损失函数} \approx g_i \cdot f_t + \frac{1}{2} h_i \cdot f_t^2$$

(3) 优势: 同一套代码可处理回归 (MSE)、分类 (Log Loss) 等任务, 无需为每种损失函数单独推导优化步骤。同时, 用户可自定义损失函数 (只需二阶可导), 增强算法扩展性。

3.加速训练与收敛

(1) 二阶信息提供更精确的步长估计, 减少达到相同精度所需的迭代次数。

(2) 结合Block 预排序和并行特征分裂技术, 进一步优化计算效率。

4.增强模型鲁棒性与泛化能力

(1) 泰勒展开的近似形式天然适配正则化项 (如 L1/L2 正则), 通过控制叶子节点权重和树复杂度抑制过拟合。

(2) 对缺失值的自适应处理: 在分裂节点时动态评估缺失样本划入左右子树的损失, 无需人工填充。

1.16 集成学习介绍下有哪些，讲讲rf的随机是什么？

集成学习 (Ensemble Learning) 是一种机器学习方法，通过组合多个基本学习器 (弱学习器) 的预测结果，以达到更好的整体性能。

集成学习可以通过减小泛化误差、提高模型的稳定性和鲁棒性等方面来改进机器学习模型。

以下是几种常见的集成学习方法：

Bagging (自助聚合)：基于自助采样 (bootstrap sampling) 的集成方法，通过有放回地从训练数据中随机抽样产生多个训练集，每个训练集用于训练一个基本学习器，最后通过投票或平均等方式来进行预测。

Boosting (提升)：基于序列化的集成方法，通过迭代地训练一系列弱学习器，每次迭代都根据前一轮的结果对训练数据进行加权，使得模型能够更关注之前预测错误的样本。最终将多个弱学习器进行加权组合得到最终的预测结果。

Random Forest (随机森林)：是一种基于决策树的集成学习方法。它利用了Bagging的思想，同时引入了决策树的随机性。在每个决策树的构建过程中，随机从特征集中选取一部分特征子集，然后根据这个子集构建决策树。这种随机选择特征的过程增加了树之间的差异性，提高了集成模型的多样性。

1.17 讲讲分类评估方式都有什么？

准确率 (Accuracy)

精确率 (Precision)

召回率 (Recall)

F1值 (F1 Score)

ROC曲线

AUC值

1.18 F1公式讲一讲？

F1值是精确率 (Precision) 和召回率 (Recall) 的调和均值，用于综合评估分类模型的性能。F1值的计算公式如下：

$$F1 = 2 * (\text{精确率} * \text{召回率}) / (\text{精确率} + \text{召回率})$$

1.19 介绍梯度优化的各种算法，详细的发展历史，优化原因？

梯度下降法 (Gradient Descent)：梯度下降法是最早也是最基础的梯度优化算法之一，起源于数值分析领域。它通过迭代更新参数，沿着梯度的反方向移动，逐渐接近目标函数的最优解。随着机器学习和深度学习的发展，梯度下降法得到广泛应用。

随机梯度下降法 (Stochastic Gradient Descent, SGD)：随机梯度下降法是对梯度下降法的改进，引入了随机采样的思想。它通过每次只使用一个样本的梯度来更新参数，减少了计算开销，加快了训练速度。SGD 广泛应用于深度学习中，尤其在大规模数据集上的训练中效果显著。

动量法 (Momentum)：动量法是一种在梯度下降法基础上添加动量项的优化算法。它引入了一个动量参数，用于累积之前梯度的信息，并加速参数更新的方向。动量法在训练过程中具有惯性，有助于跳出局部最优解，加快收敛速度。

AdaGrad：AdaGrad是一种自适应学习率的优化算法，根据参数的历史梯度信息来自动调整学习率。它通过对梯度进行累积，将较大梯度的参数学习率适当减小，较小梯度的参数学习率适当增大。AdaGrad对于稀疏数据集和非平稳目标函数具有一定的优势。

RMSProp：RMSProp是对AdaGrad算法的改进，用于解决AdaGrad学习率过于降低的问题。它引入了一个衰减因子，对历史梯度进行指数加权平均。

1.20 用过xgboost吗，原理你了解吗？他是怎么做并行的？

XGBoost是一种基于梯度提升树的集成学习算法，用于解决分类和回归问题。它通过构建多个弱学习器（决策树）的集合来提高预测性能。

XGBoost的原理如下：

- 初始化：将初始预测值设置为训练数据样本的平均值。
- 迭代训练：通过迭代训练来逐步改善预测结果。
 - a. 计算残差：计算当前预测值与实际标签之间的残差。
 - b. 构建决策树：使用残差作为目标，构建一颗决策树来拟合残差。
 - c. 更新预测值：将新构建的决策树与之前的决策树进行加权相加，更新预测值。
 - d. 迭代训练：重复执行步骤a-c，直到达到指定的迭代次数或满足停止条件。
- 得到最终预测结果：将所有决策树的预测结果相加得到最终的预测结果。

XGBoost的并行化方法主要有两个方面：

- 列并行化：在构建决策树时，XGBoost可以并行计算特征列上的直方图，以加快特征的分割和选择过程。这样可以将数据集按列划分，分配给多个线程并行处理。
- 行并行化：在计算目标函数的梯度和构建决策树时，XGBoost可以并行计算不同样本的梯度和叶子节点的更新。这样可以将数据集按行划分，分配给多个线程并行处理。

五、深度学习

1.1 谈谈深度学习中的归一化问题

1.1 什么是数据归一化？

归一化（标准化）可以定义为：归一化就是要把你需要处理的数据经过处理后（通过某种算法）限制在你需要的一定范围内。首先归一化是为了后面数据处理的方便，其次是保证模型运行时收敛加快。

归一化和标准化的区别：

简单来说归一化的分母是大于等于标准化的分母的，且归一化的分子是小于分母，故归一化是把x限制在[0,1]之间，而标准化没有限制，且对噪点不敏感

1.2 为什么要进行数据归一化？

取消量纲 使得剃度始终朝着最小值的方向前进 少走弯路 加速收敛

若样本x的一个特征A是从10000-100000，另一个特征B是从0.1-1，这样训练神经网络时候会导致B的作用非常小，这是不希望看到的。

同时这样会导致求偏导时A的参数变化大，B的参数变化小，这样会导致lost最小化是不停的走弯路，浪费时间

1.3 数据归一化的方法有哪些？

$$\begin{aligned} \bullet \text{ 归一化 (normalization) : } & \frac{X_i - X_{min}}{X_{max} - X_{min}} \\ \bullet \text{ 标准化 (standardization) : } & \frac{X_i - \mu}{\sigma} \end{aligned}$$

1.4 哪些模型不需要归一化

概率模型不需要归一化，因为它们不关心变量的值，而是关心变量的分布和变量之间的条件概率，如决策树、rf。而像 adaboost、svm、lr、KNN、KMeans之类的最优化问题就需要归一化。

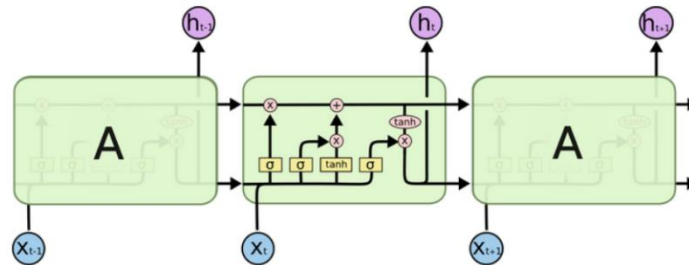
batch 归一化：不仅对输入x进行归一化，同时对各个隐藏层的z进行归一化

为什么batch 归一化会有用？

简单来说就是在mini-batch归一化时，因为只使用了部分数据，故归一化时存在噪声的，这就相当于给神经网络增加了 噪声，相当于dropout，给隐藏单元增加了噪声，使后面的隐藏单元不过分依赖于任何一个隐藏单元，有轻微正则化的作用。

1.2 LSTM结构推导，为什么比RNN好？

解析：

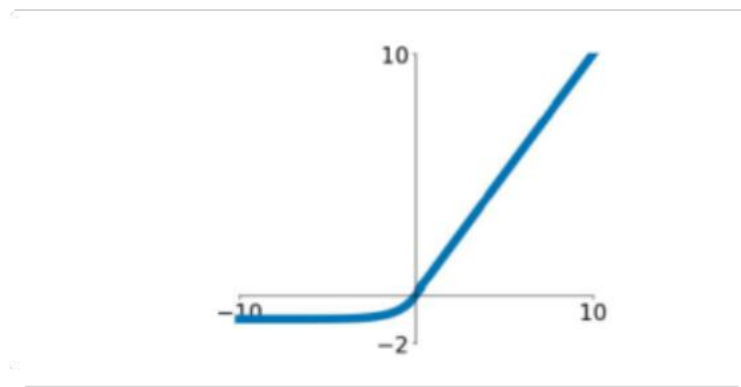


推导forget gate, input gate, cell state, hidden information等的变化；因为LSTM有进有出且当前的cell informaton是通过input gate控制之后叠加的，RNN是叠乘，因此LSTM可以防止梯度消失或者爆炸。

1.3 Sigmoid、Tanh、ReLu这三个激活函数有什么缺点或不足

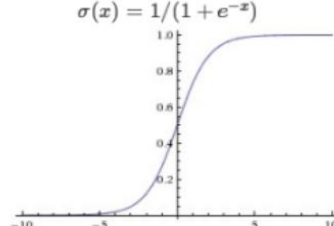
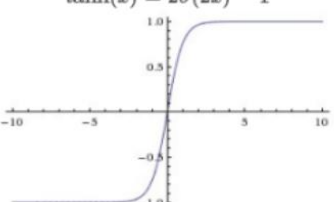
解析：

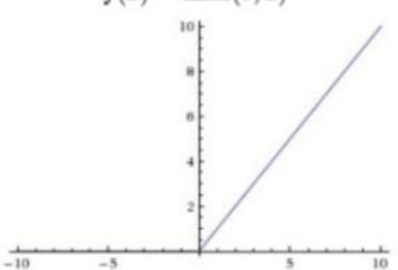
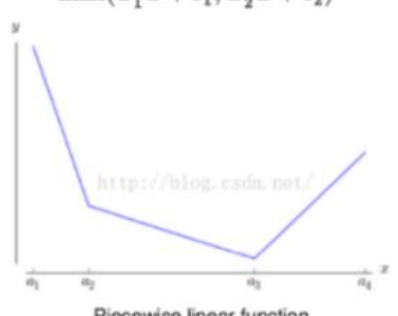
sigmoid、Tanh、ReLU的缺点在121问题中已有说明，为了解决ReLU的dead cell的情况，发明了Leaky Relu，即在输入小于0时不让输出为0，而是乘以一个较小的系数，从而保证有导数存在。同样的目的，还有一个ELU，函数示意图如下。



$$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha (\exp(x) - 1) & \text{if } x \leq 0 \end{cases}$$

还有一个激活函数是Maxout，即使用两套w,b参数，输出较大值。本质上Maxout可以看做Relu的泛化版本，因为如果一套w,b全都是0的话，那么就是普通的ReLU。Maxout可以克服Relu的缺点，但是参数数目翻倍

激活函数	公式	缺点	优点
Sigmoid	$\sigma(x) = 1/(1 + e^{-x})$ 	1、会有梯度弥散 2、不是关于原点对称 3、计算exp比较耗时	-
Tanh	$\tanh(x) = 2\sigma(2x) - 1$ 	梯度弥散没解决	1、解决了原点对称问题 2、比sigmoid更快

ReLU	$f(x) = \max(0, x)$ 	梯度弥散没完全解决，在(-)部分相当于神经元死亡而且不会复活	1、解决了部分梯度弥散问题 2、收敛速度更快
Leaky ReLU	$f(x) = 1(x < 0)(\alpha x) + 1(x \geq 0)(x)$	-	解决了神经死亡问题
Maxout	$\max(w_1^T x + b_1, w_2^T x + b_2)$  Piecewise linear function	参数比较多，本质上是在输出结果上又增加了一层	克服了ReLU的缺点，比较提倡使用

1.4 为什么引入非线性激励函数？

解析：

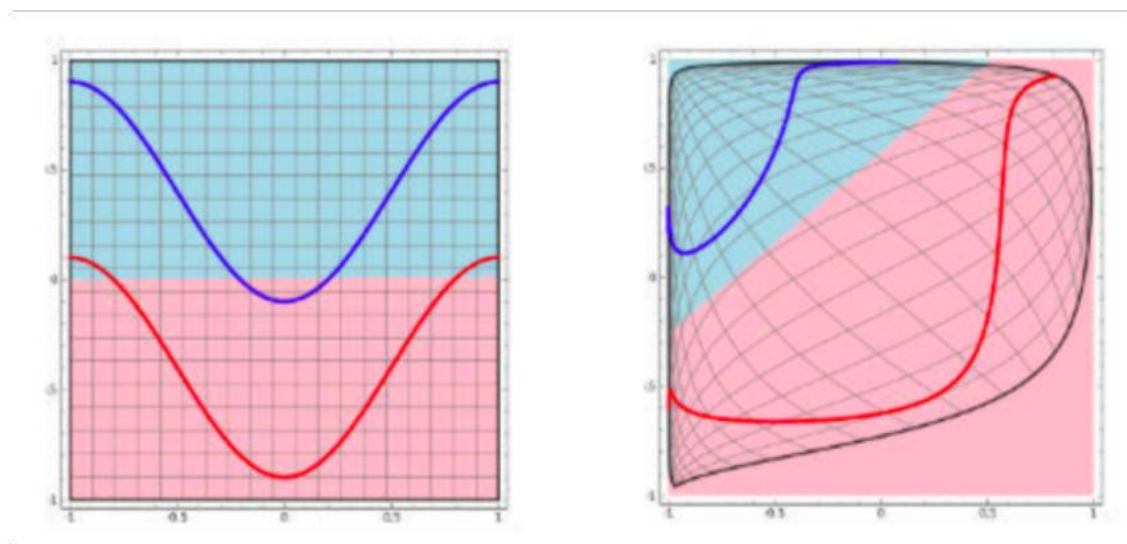
第一，对于神经网络来说，网络的每一层相当于 $f(wx+b)=f(w'x)$ ，对于线性函数，其实相当于 $f(x)=x$ ，那么在线性激活函数下，每一层相当于用一个矩阵去乘以 x ，那么多层就是反复的用矩阵去乘以输入。根据矩阵的乘法法则，多个矩阵相乘 得到一个大矩阵。所以线性激励函数下，多层网络与一层网络相当。比如，两层的网络 $f(W1*f(W2x))=W1W2x=Wx$ 。

第二，非线性变换是深度学习有效的原因之一。原因在于非线性相当于对空间进行变换，变换完成后相当于对问题空间进行简化，原来线性不可解的问题现在变得可以解了。

如果不用激励函数（其实相当于激励函数是 $f(x) = x$ ），在这种情况下你每一层输出都是上层输入的线性函数，很容易验证，无论你神经网络有多少层，输出都是输入的线性组合，与没有隐藏层效果相当，这种情况就是最原始的感知机（Perceptron）了。

正因为上面的原因，我们决定引入非线性函数作为激励函数，这样深层神经网络就有意义了（不再是输入的线性组合，可以逼近任意函数）。最早的想法是sigmoid函数或者tanh函数，输出有界，很容易充当下一层输入（以及一些人的生物解释）。

下图可以很形象的解释这个问题，左图用一根线是无法划分的。经过一系列变换后，就变成线性可解的问题了。



1.5 请问人工神经网络中为什么ReLu要好过于tanh和sigmoid ？

解析：

先回忆sigmoid、tanh和RelU的函数图

(1) 第一，采用sigmoid等函数，算激活函数时（指数运算），计算量大，反向传播求误差梯度时，求导涉及除法和指数运算，计算量相对大，而采用Relu激活函数，整个过程的计算量节省很多。

(2) 第二，对于深层网络，sigmoid函数反向传播时，很容易就会出现梯度消失的情况（在sigmoid接近饱和和区时，变换太缓慢，导数趋于0，这种情况会造成信息丢失），这种现象称为饱和，从而无法完成深层网络的训练。而ReLU就不会有饱和倾向，不会有特别小的梯度出现。

(3) 第三，Relu会使一部分神经元的输出为0，这样就造成了网络的稀疏性，并且减少了参数的相互依存关系，缓解了过拟合问题的发生（以及一些人的生物解释balabala）。当然现在也有一些对relu的改进，比如prelu，random relu等，在不同的数据集上会有一些训练速度上或者准确率上的改进，具体的大家可以找

相关的paper看。

(4) 现在主流的做法，会多做一步batch normalization，尽可能保证每一层网络的输入具有相同的分布。而最新的paper，他们在加入bypass connection之后，发现改变batch normalization的位置会有更好的效果。

1.6 为什么LSTM模型中既存在sigmoid又存在tanh两种激活函数，而不是选择统一一种sigmoid或者tanh？这样做的目的是什么？

解析：

sigmoid 用在了各种gate上，产生0~1之间的值，这个一般只有sigmoid最直接了，相当于要么是1则记住，要么是0则忘掉。tanh 用在了状态和输出上，是对数据的处理，这个用其他激活函数或许也可以。

1.7 如何解决RNN梯度爆炸和弥散的问题？

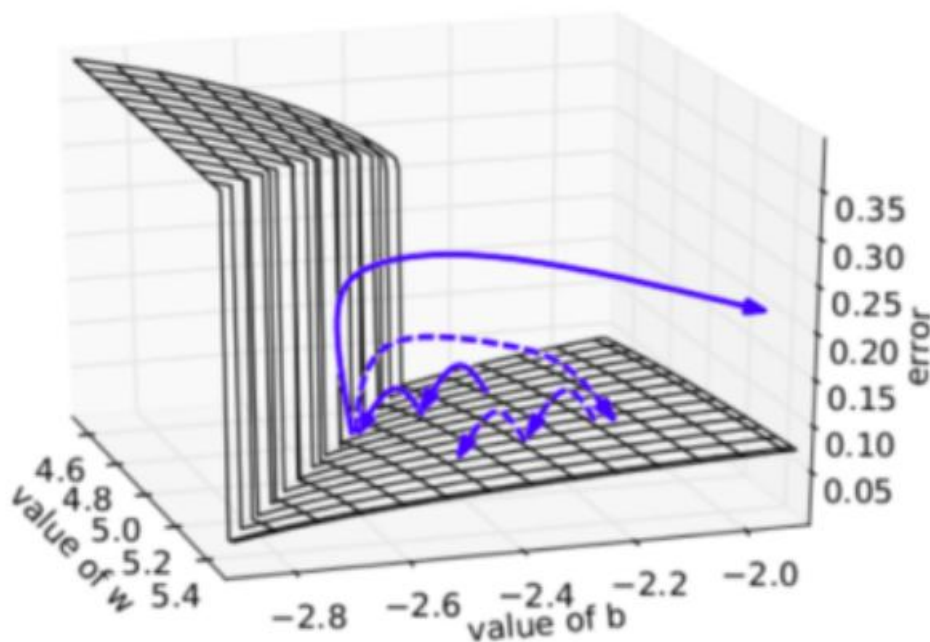
解析：

为了解决梯度爆炸问题，Thomas Mikolov首先提出了一个简单的启发性的解决方案，就是当梯度大于一定阈值的的时候，将它截断为一个较小的数。具体如算法1所述：

算法：当梯度爆炸时截断梯度（伪代码）

$$\begin{aligned}\hat{g} &\leftarrow \frac{\partial E}{\partial W} \\ \text{if } \|\hat{g}\| &\geq \text{threshold} \\ \hat{g} &\leftarrow \frac{\text{threshold}}{\|\hat{g}\|} \hat{g}\end{aligned}$$

下图可视化了梯度截断的效果。它展示了一个小的rnn（其中W为权值矩阵，b为bias项）的决策面。这个模型是一个一小段时间的rnn单元组成；实心箭头表明每步梯度下降的训练过程。当梯度下降过程中，模型的目标函数取得了较高的误差时，梯度将被送到远离决策面的位置。截断模型产生了一个虚线，它将误差梯度拉回到离原始梯度接近的位置。



梯度爆炸，梯度截断可视化

为了解决梯度弥散的问题，我们介绍了两种方法。第一种方法是将随机初始化 $W(hh)$ 改为一个有关联的矩阵初始化。第二种方法是使用ReLU (Rectified Linear Units) 代替sigmoid函数。ReLU的导数不是0就是1.因此，神经元的梯度将始终为1，而不会当梯度传播了一定时间之后变小。

1.8 什么样的资料集不适合用深度学习？

解析：

- (1) 数据集太小，数据样本不足时，深度学习相对其它机器学习算法，没有明显优势。
- (2) 数据集没有局部相关特性，目前深度学习表现比较好的领域主要是图像 / 语音 / 自然语言处理等领域，这些领域的一个共性是局部相关性。图像中像素组成物体，语音信号中音位组合成单词，文本数据中单词组合成句子，这些特征元素 的组合一旦被打乱，表示的含义同时也被改变。对于没有这样的局部相关性的数据集，不适于使用深度学习算法进行处 理。

举个例子：预测一个人的健康状况，相关的参数会有年龄、职业、收入、家庭状况等各种元素，将这些元素打乱，并不会影响相关的结果。

1.9 广义线性模型是怎被应用在深度学习中？

解析：

A Statistical View of Deep Learning (I): Recursive GLMs

深度学习从统计学角度，可以看做递归的广义线性模型。

广义线性模型相对于经典的线性模型($y=wx+b$)，核心在于引入了连接函数 $g(\cdot)$ ，形式变为： $y=g^{-1}(wx+b)$ 。

深度学习时递归的广义线性模型，神经元的激活函数，即为广义线性模型的链接函数。逻辑回归（广义线性模型的一种）的Logistic函数即为神经元激活函数中的Sigmoid函数，很多类似的方法在统计学和神经网络中的名称不一样，容易引起初学者（这里主要指我）的困惑。

下图是一个对照表

Target Type	Regression	Link	Inv link	Activation
Real	Linear	Identity	Identity	
Binary	Logistic	Logit $\log \frac{\mu}{1-\mu}$	Sigmoid σ $\frac{1}{1+\exp(-\eta)}$	Sigmoid
Binary	Probit	Inv Gauss CDF $\Phi^{-1}(\mu)$	Gauss CDF $\Phi(\eta)$	Probit
Binary	Gumbel	Compl. log-log $\log(-\log(\mu))$	Gumbel CDF $e^{-e^{-x}}$	
Binary	Logistic		Hyperbolic Tangent $\tanh(\eta)$	Tanh
Categorical	Multinomial		Multin. Logit $\frac{\eta_i}{\sum_j \eta_j}$	Softmax
Counts	Poisson	$\log(\mu)$	$\exp(\nu)$	
Counts	Poisson	$\sqrt{\mu}$	ν^2	
Non-neg.	Gamma	Reciprocal $\frac{1}{\mu}$	$\frac{1}{\nu}$	
Sparse	Tobit		max $\max(0; \nu)$	ReLU
Ordered	Ordinal		Cum. Logit $\sigma(\phi_k - \eta)$	

1.10 如何解决梯度消失和梯度膨胀

解析：

(1) 梯度消失：

根据链式法则，如果每一层神经元对上一层的输出的偏导乘上权重结果都小于1的话，那么即使这个结果是0.99，在经过足够多层传播之后，误差对输入层的偏导会趋于0

可以采用ReLU激活函数有效的解决梯度消失的情况，也可以用Batch Normalization解决这个问题。

(2) 梯度膨胀

根据链式法则，如果每一层神经元对上一层的输出的偏导乘上权重结果都大于1的话，在经过足够多层传播之后，误差对输入层的偏导会趋于无穷大。

可以通过激活函数来解决，或用Batch Normalization解决这个问题。

1.11 深度学习有哪些熟悉的优化算法？

梯度下降 (Gradient Descent)：梯度下降是最基础和常用的优化算法。它通过计算损失函数关于模型参数的梯度，并按照梯度的反方向更新参数，从而使损失函数逐步减小。

随机梯度下降 (Stochastic Gradient Descent, SGD)：SGD是梯度下降的一种变体，它在每次参数更新时只使用一个样本（或一小批样本）的梯度，从而减少计算量。SGD通常具有更快的收敛速度，但可能引入更多的噪声。

动量优化 (Momentum Optimization)：动量优化在SGD的基础上引入了动量项，用于加速参数更新。动量项考虑了之前的参数更新方向，使得在梯度方向变化较小的情况下能够继续前进，从而加快收敛速度。

AdaGrad (Adaptive Gradient)：AdaGrad算法对梯度进行逐元素平方累积，并将其作为学习率的分母，从而使得学习率能够自适应地进行调整。它适用于处理稀疏梯度和非平稳问题，但可能会导致学习率过早衰减。

RMSProp (Root Mean Square Propagation)：RMSProp算法改进了AdaGrad中学习率过早衰减的问题。它引入了一个衰减系数，使得过去梯度的贡献逐渐减小，从而平衡了历史梯度信息和当前梯度信息的重要性。

Adam (Adaptive Moment Estimation)：Adam算法结合了动量优化和RMSProp的思想，通过计算梯度的一阶矩估计和二阶矩估计来调整学习率。Adam通常在深度学习中表现出色，具有较快的收敛速度和良好的泛化性能。

1.12 Adam怎么更新的? (Adam怎么优化?)

Adam (Adaptive Moment Estimation) 是一种常用的优化算法，结合了自适应学习率和动量的特性，用于训练深度学习模型

下面是Adam算法的更新规则：

- 初始化：初始化学习率（learning rate）为 η ，动量参数为 β_1 和 β_2 ，对于每个训练步骤，进行以下更新
- 计算梯度：根据当前的参数值计算损失函数对参数的梯度。
- 更新动量：根据梯度计算动量的一阶矩估计（梯度的移动平均值）
- 更新动量方差：根据梯度的平方计算动量方差的二阶矩估计（梯度平方的移动平均值）
- 偏差校正：修正动量和动量方差的偏差
- 参数更新：根据修正后的动量和动量方差，更新参数的值。

1.13 讲讲学习率衰减？

学习率衰减（Learning Rate Decay）是指在训练过程中逐渐降低学习率的策略。通过衰减学习率，可以在训练的早期阶段使用较大的学习率以快速收敛，然后逐渐降低学习率以细化参数更新并提高模型的泛化能力。

常见的学习率衰减方法：固定衰减（Fixed Decay）、指数衰减（Exponential Decay）

1.14 pytorch里怎么从CPU迁移到GPU？

检查是否有可用的GPU设备：

使用`torch.cuda.is_available()`函数检查系统是否具有可用的GPU设备。如果返回True，表示有可用的GPU。

将模型参数迁移到GPU：`model.to(device)`

将输入数据迁移到GPU：

将输入数据从CPU迁移到GPU需要使用`.to(device)`方法

1.15 激活函数sigmoid的梯度最大点在哪儿，等于多少？

Sigmoid函数的梯度最大点发生在其输入为零的位置。在数学上，Sigmoid函数定义如下：

$$\text{sigmoid}(x) = 1 / (1 + \exp(-x))$$

1.16 batch_size对训练的影响过拟合，怎么处理？

过小的批量大小可能导致以下问题：

- 不稳定的梯度估计：小批量中的样本数量较少，梯度的估计可能不准确，导致训练过程中的参数更新不稳定。
- 较长的训练时间：由于需要处理更多的批次，训练时间可能会增加。

过大的批量大小可能导致以下问题：

- 内存消耗：较大的批量大小需要更多的内存来存储计算图和中间结果，当内存不足时，无法训练较大批量大小的模型。
- 梯度下降步长较大：较大的批量大小会导致梯度下降的步长增大，可能导致优化过程无法收敛或收敛到次优解。

要处理批量大小对过拟合的影响，可以尝试以下方法：

- 调整批量大小：可以通过尝试不同的批量大小来寻找一个合适的取值。较小的批量大小可以增加噪声，有助于正则化，但可能导致训练过程不稳定。较大的批量大小可以减少噪声，但可能导致模型过拟合。通过尝试不同的批量大小，找到一个平衡点来控制过拟合。
- 使用正则化技术：除了调整批量大小外，可以采用正则化技术来控制过拟合。例如，L1正则化、L2正则化、Dropout等技术可以帮助减少模型的过拟合现象。
- 增加数据量：过拟合通常是由于训练数据较少导致的，增加训练数据量可以有效减轻过拟合。通过数据增强技术、收集更多的标记数据或使用迁移学习等方法来增加数据量。
- 使用早停策略：早停是一种常用的防止过拟合的方法。在训练过程中，监控模型在验证集上的性能，当性能不再提升时停止训练，避免过度拟合训练数据。

1.17 gelu函数和relu函数的区别？

ReLU函数是一种分段线性函数，在输入为正时输出与输入相等，而在输入为负时输出为零。ReLU函数的定义为： $\text{ReLU}(x) = \max(0, x)$

ReLU函数简单且计算高效，容易实现，并且在一定程度上可以缓解梯度消失问题。然而，ReLU函数在输入为负时输出为零，导致一部分神经元变得不活跃，称为“神经元死亡”问题。这可能导致网络的表示能力受限，特别是在深层网络中。

GELU函数是一种连续、平滑的非线性激活函数，它在近似ReLU的同时具有更平滑的曲线。GELU函数的定义为：

$$\text{GELU}(x) = x \cdot \Phi(x)$$

其中， x 是输入； $\Phi(x)$ 是标准正态分布的累积分布函数，具体为：

$$\Phi(x) = \frac{1}{2} \left(1 + \text{erf} \left(\frac{x}{\sqrt{2}} \right) \right)$$

GELU函数通过引入高斯误差函数 (Gaussian Error Function) 的近似来实现平滑性。相比于ReLU函数，GELU函数在输入为负时不会完全抑制激活，而是通过一个平滑的曲线逐渐减小输出值。这种平滑性可以提供更

好的梯度传播和表示能力，使得神经网络更容易学习复杂的函数关系

六、自然语言处理

1. NLP基础

1.1 hmm原理、主要问题、解决方法？

HMM的原理：

HMM由两个关键组成部分构成：隐藏状态（Hidden State）和观测状态（Observation State）。隐藏状态是不可见的，代表系统内部的状态，而观测状态是可见的，代表我们可以观测到的数据。HMM假设隐藏状态与观测状态之间存在马尔可夫性质，即当前的隐藏状态只与前一个隐藏状态相关，与其他状态无关。

HMM的主要问题：

- 评估问题（Evaluation Problem）：给定一个观测序列和HMM模型，计算该观测序列出现的概率。
- 解码问题（Decoding Problem）：给定一个观测序列和HMM模型，找到最可能的隐藏状态序列。
- 学习问题（Learning Problem）：给定一组观测序列，学习HMM模型的参数，包括隐藏状态转移概率、观测状态概率以及初始隐藏状态概率。

HMM的解决方法：

- 前向-后向算法（Forward-Backward Algorithm）用于解决评估问题，通过动态规划计算观测序列的概率。
- Viterbi算法用于解决解码问题，通过动态规划找到最可能的隐藏状态序列。
- Baum-Welch算法（也称为Expectation-Maximization算法）用于解决学习问题，通过迭代计算最优的模型参数。

1.2 fasttext和word2vec差异？

Word2Vec: Word2Vec模型主要有两种训练算法，分别是Skip-gram和CBOW (Continuous Bag of Words)。这些算法通过在上下文中预测目标词汇或通过目标词汇预测上下文来学习词向量。

FastText: FastText是基于Word2Vec的扩展，它将单词视为字符级别的n-gram序列的集合。FastText通过学习字符级别的n-gram向量表示来捕捉词汇内部的信息，并将它们组合成单词向量。

1.3 介绍一下Glove?

自从2013年Google提出Word2Vec词向量后，斯坦福大学NLP研究小组于2014年提出Glove词向量。

在 Glove模型被提出之前，学习词向量的模型主要有两大类：

- 全局矩阵分解方法，例如潜在语义分析(Latent semantic analysis, LSA)
 - 虽然LSA 这样的方法有效地利用了统计信息，但在单词类比任务上表现相对较差，这表明不是最优的向量空间结构
- 局部上下文窗口方法，例如skip-gram模式的Word2Vec模型
 - 像 skip-gram 这样的方法可能在类比任务上做得更好，但是他们很少利用语料库的统计数据，因为它们在单独的局部上下文窗口上训练，而不是在全局共现计数上训练

GloVe 的全名叫 Global Vectors for Word Representation，它是一个基于全局词频统计的模型，实现分三个步骤：

- 构建共现矩阵
- 词向量和共现矩阵的近似关系
- 构造损失函数

GloVe 是一个新的全局对数双线性回归模型，用于词向量的无监督学习。它结合了全局矩阵分解和局部上下文窗口方法两者的优点。GloVe 模型通过只训练词-词共现矩阵中的非零元素，而不是整个稀疏矩阵或大型语料库中的单个上下文窗口，有效地利用了统计信息。GloVe 模型生成了一个具有有意义子结构的向量空间。

1.4 attention为什么使用多头?

- 表达能力增强：通过引入多个注意力头，每个头都可以关注不同的语义信息，从而能够捕捉更丰富的特征表示。不同的注意力头可以学习到不同的关注权重分布，帮助模型更好地理解输入序列的不同部分。
- 模型泛化性能提升：通过多头机制，模型可以同时考虑多个关注粒度的信息。每个头可以关注序列中的不同位置或不同的关系，使得模型能够处理不同尺度和不同层次的语义关系。这有助于提高模型在不同任务和不同数据集上的泛化性能。
- 抗噪能力增强：多头机制能够减少注意力机制中的随机性，因为每个头都可以专注于不同的注意力权重。这样可以降低模型对于噪声的敏感性，提高模型的鲁棒性和稳定性。
- 并行计算加速：注意力机制的计算通常是高度并行的，而多头注意力进一步增加了并行计算的程度。多个注意力头可以同时计算，加快了模型的训练和推理速度。

1.5 交叉熵和对数似然损失的区别和联系？

- 交叉熵损失：交叉熵损失是用于衡量两个概率分布之间的差异的度量。在分类任务中，它衡量了模型的预测概率分布与实际标签的概率分布之间的差异。
- 对数似然损失：对数似然损失是用于估计参数的最大似然估计的一个重要组成部分。它衡量了模型生成观测数据的概率与实际观测数据的概率之间的差异。

1.6 交叉熵和KL散度的区别和联系？

- 交叉熵：交叉熵是一种度量两个概率分布之间的差异的度量方法。对于两个概率分布P和Q，交叉熵可以通过以下公式表示： $H(P, Q) = -\sum P(x)\log(Q(x))$ 。交叉熵是基于信息论的概念，用于衡量在使用概率分布Q来编码从真实分布P中得到的信息的平均信息量。
- KL散度：KL散度也是一种度量两个概率分布之间的差异的度量方法。对于两个概率分布P和Q，KL散度可以通过以下公式表示： $D(P||Q) = \sum P(x)\log(P(x)/Q(x))$ 。KL散度衡量了在使用概率分布Q来表示真实分布P时产生的额外信息量。

1.7 rnn怎么产生梯度爆炸的，lstm怎么解决梯度消失的

梯度爆炸：

梯度爆炸指的是在反向传播过程中，梯度值变得非常大，导致权重更新过大，模型难以收敛。

在RNN中，梯度爆炸通常是由于反向传播过程中梯度值的指数级增长。当时间步数较大或激活函数的导数较大时，梯度爆炸问题尤为突出。

梯度消失：

梯度消失指的是在反向传播过程中，梯度值变得非常小，导致权重更新过小，模型难以学习长期依赖关系。

在传统的RNN中，梯度消失问题常常是由于反向传播过程中梯度值的指数级衰减。当时间步数较大或激活函数的导数较小时，梯度消失问题尤为突出。

LSTM解决梯度消失：

LSTM是一种特殊的RNN变体，通过引入门控机制来解决梯度消失问题。

LSTM中的门控单元（Gate Units）可以控制信息的流动，允许模型选择性地保留和更新信息。

LSTM中的遗忘门（Forget Gate）可以决定前一时刻的记忆是否被遗忘，输入门（Input Gate）可以决定当前输入的哪一部分信息被添加到记忆中，输出门（Output Gate）可以决定输出的哪部分记忆被传递到下一层或输出层。

通过门控机制，LSTM可以有效地捕捉和传递长期依赖关系，从而解决了传统RNN中的梯度消失问题。

总结而言，RNN中的梯度爆炸和梯度消失问题会影响模型的训练和学习能力。LSTM通过引入门控机制，特别是遗忘门、输入门和输出门，解决了梯度消失的问题，使得模型能够更好地处理长期依赖关系。

1.8 过拟合的解决方法?

数据集扩充 (Data Augmentation)

正则化 (Regularization) : 常见的正则化方法包括L1正则化和L2正则化。

提前停止 (Early Stopping)

Dropout

模型复杂度控制

可以尝试减少模型的层数、减少神经元的数量、减少参数的数量等来降低模型的复杂度

1.9 Bert为啥能解一词多意?

BERT能够解决一词多义的问题，主要是因为其预训练阶段采用了无监督的自编码器任务。

BERT模型的预训练过程包括两个任务：Masked Language Model (MLM) 和Next Sentence Prediction (NSP) 。

在MLM任务中，BERT会随机遮盖输入文本的一些单词，并要求模型预测这些被遮盖的单词是什么。这个任务促使BERT 学习到单词在上下文中的语义和语法信息，从而能够理解和预测句子中未知的单词。

1.10 Textcnn会不会? 详细介绍一下细节?

TextCNN (Text Convolutional Neural Network) 是一种用于文本分类的卷积神经网络模型。它通过卷积操作在文本输

入上提取局部特征，并通过池化操作将这些特征进行压缩和提取，最后使用全连接层进行分类。

下面是TextCNN的详细介绍及其关键细节：

- 输入表示：
- 卷积层：TextCNN采用多个不同尺寸的卷积核（也称为滤波器）对输入进行卷积操作，每个卷积核负责提取一种局部特征。
- 激活函数和池化层：
- 特征拼接和全连接层：将所有池化后的特征向量拼接成一个长向量。
- 然后，通过全连接层将特征向量映射到输出类别的概率分布。
- 最后一层使用softmax激活函数，用于多分类问题。
- 损失函数和训练：

1.11 标签平滑是怎么做的，有什么作用？

标签平滑 (Label Smoothing) 是一种在训练分类模型时用于减少过拟合和提高模型泛化能力的技术。它通过在真实标签 和其他类别标签之间引入一些平滑的噪声，来调整训练数据的标签分布。

标签平滑的具体做法是将真实标签（通常为one-hot向量）中的某个固定的位置的值得从1减少到一个较小的正数（如 0.9），同时将其他位置的值增加到一个较小的正数（如0.1）。这样可以使得真实标签不再是严格的离散概率分布，而是一个更平滑的分布。

标签平滑的作用有以下几个方面：

减少过拟合、提高泛化能力、降低模型对错误标签的敏感性、提升模型的不确定性估计

1.12 NER中加入CRF的原因是什么？

在命名实体识别 (Named Entity Recognition, NER) 任务中，加入条件随机场 (Conditional Random Field, CRF) 的目的是为了提高序列标注模型的性能和准确度。

以下是加入CRF的原因：

- 建模上下文依赖。
- 解决标签偏置问题。
- 全局推断。
- 标签转换特征。

1.13 transformer为什么用layer norm不用batch norm，解释下，什么情况可以用batch norm？

序列长度变化：在Transformer中，输入序列的长度通常是可变的，因为它们可能具有不同数量的词语或位置编码。Batch Normalization依赖于在每个批次中计算样本的均值和方差，而样本数目在不同的批次中是不同的。这导致在每个批次上计算的均值和方差存在差异，这可能会影响模型的稳定性。相比之下，Layer Normalization是对每个样本的特征维度进行归一化，不依赖于批次，因此对序列长度的变化更加鲁棒。

训练和推断的一致性：在训练和推断过程中，Batch Normalization的行为是不同的。在训练时，Batch Normalization使用每个批次的均值和方差进行归一化，但在推断时，需要使用累计的整体均值和方差。这意味着在推断过程中，需要记录每个批次的均值和方差，增加了推断的复杂性。相比之下，Layer Normalization的计算过程在训练和推断中是一致的，更加简单和可靠。

并行计算：Batch Normalization的计算涉及对每个特征维度的均值和方差进行计算，因此在处理序列时无法进行有效的并行计算。而Layer Normalization的计算是在特征维度上进行的，可以更好地支持并行计算，因此在处理长序列时更加高效。

1.14 fasttext结构是怎么样的？怎么迁移学习的，具体过程？fasttext对比lstm优点？为什么有这些优点？具体讲下层次softmax和ngram (lstm都有哪些结构)？

FastText的结构如下：

输入层：将文本表示为词袋模型，将文本中的词语转换为固定长度的向量表示。

隐含层：包含一个或多个线性映射层，将输入的词向量映射到低维表示。

输出层：使用softmax函数进行多类别分类。对于文本分类任务，输出层的每个节点对应一个类别。

迁移学习是将已经在一个任务上训练好的模型应用于另一个相关任务上的技术。在FastText中，迁移学习可以通过两种方式进行：

- 特征提取：将预训练的FastText模型用作特征提取器，提取文本的低维向量表示作为输入，然后使用这些向量表示来训练新的分类器。在这种情况下，只需冻结预训练模型的参数，只更新新的分类器的参数。
- Fine-tuning：在迁移学习中，除了使用预训练的FastText模型提取特征外，还可以继续对整个模型进行训练，以更好地适应新的任务。在这种情况下，可以通过微调预训练模型的参数，使其更好地适应新任务的特征。

FastText相对于LSTM（长短期记忆网络）具有以下优点：

- 训练速度快：FastText采用了层次Softmax和N-gram的技术，这些技术使得它在处理大规模数据时非常高效。相比之下，LSTM模型通常需要更多的计算资源和时间来训练。
- 文本表示的简单性：FastText将文本表示为词袋模型，将每个词语的向量平均作为整个文本的向量表示。这种简化的表示方法对于短文本和少量标记数据的任务效果较好，并且可以在低维空间中对文本进行分类。
- 层次Softmax：FastText中使用的一种技术，用于加速多类别分类的训练过程。它通过将类别的分布改变为树形结构，将原本计算复杂度较高的Softmax函数转化为一系列较小规模的二分类问题，从而提高了训练速度。

1.15 层次softmax为什么用霍夫曼树，其他树行不行？

层次Softmax (Hierarchical Softmax) 是一种用于加速多类别分类任务的方法，通过使用树形结构来替代传统的Softmax函数，从而减少计算复杂度。层次Softmax中使用霍夫曼树的主要原因是它具有有一些优势，使

得它在构建和训练过程中更加高效。

以下是层次Softmax使用霍夫曼树的主要原因：

- 压缩类别表示：霍夫曼树通过将高频词语放置在树的较浅层，而低频词语放置在树的较深层，从而将较大的类别空间压缩为较小的码空间。这样可以减少类别之间的距离，提高模型训练的效率。
- 减少计算复杂度：霍夫曼树的结构使得计算每个类别的概率只需要遍历树的路径，而不需要计算所有类别的概率。相比传统的Softmax函数，层次Softmax使用霍夫曼树可以显著降低计算复杂度，特别是在处理大规模多类别分类任务时。
- 子类别的共享参数：在霍夫曼树中，具有相同父节点的子类别共享参数，这样可以减少模型中需要训练的参数数量。共享参数可以提高模型的泛化能力，并且在训练数据稀缺的情况下，可以更好地处理未见过的类别。

1.16 维特比算法时间复杂度，维特比算法原理？

维特比算法（Viterbi algorithm）是一种用于解码隐马尔可夫模型（Hidden Markov Model, HMM）的动态规划算法。它可以在给定观测序列的情况下，寻找最有可能的隐藏状态序列。

维特比算法的时间复杂度取决于隐马尔可夫模型的状态数量和观测序列的长度。假设状态数量为 N ，观测序列的长度为 T ，则维特比算法的时间复杂度为 $O(N^2 * T)$ 。

维特比算法的原理如下：

- 初始化：对于初始时刻 $t=1$ ，计算每个隐藏状态 i 的初始概率，即在 $t=1$ 时刻状态为 i 的概率乘以从初始状态到状态 i 的转移概率。
- 递推：对于每个时刻 $t=2$ 到 T ，计算每个隐藏状态 i 的最大概率和对应的路径。对于每个状态 i ，在 $t-1$ 时刻的每个状态 j ，计算从状态 j 转移到状态 i 的概率乘以状态 j 的最大概率。选择最大概率作为状态 i 的最大概率，并记录该路径。
- 终止：在最后时刻 T ，选择具有最大概率的隐藏状态作为最终的状态，并记录路径。
- 回溯：从终止时刻开始，根据记录的路径逆向回溯，得到最有可能的隐藏状态序列。

1.17 CRF和HMM的优缺点，模型的结构上有什么不同吗，模型的输入输出是什么样子呢？

CRF（条件随机场）和HMM（隐马尔可夫模型）是常用于序列标注任务的统计模型，虽然它们都处理序列数据，但在模型结构、优缺点以及输入输出方面存在一些区别。

CRF的优点：

- 与HMM相比，CRF能够建模更复杂的特征关系。CRF的特征函数可以考虑更大范围的上下文信息，从而捕捉更丰富的特征相关性，提升模型性能。
- CRF直接对标签序列进行建模，因此可以进行端到端的训练和预测，无需生成观测序列。
- CRF能够有效地处理标签间的依赖关系，对标签之间的约束进行建模，从而提高标注的准确性。
- CRF可以灵活地利用各种特征，包括基于词、基于字符的特征等，适用于多种序列标注任务。

CRF的缺点：

- CRF在建模时需要考虑的特征较多，容易出现维度灾难问题，导致计算复杂度较高。
- CRF对于长距离依赖关系的建模能力相对有限，特别是当标签序列的长度较长时。

HMM的优点：

- HMM具有简单的模型结构和较低的计算复杂度，适合处理较短的序列和简单的序列标注任务。
- HMM具有较好的可解释性，可以通过转移概率和发射概率来解释模型的行为。

HMM的缺点：

- HMM假设观测状态只依赖于隐藏状态，忽略了观测状态之间的关系，限制了模型的表达能力。
- HMM对于标签之间的依赖关系建模能力较弱，很难处理复杂的标签约束。

1.18 说一下层次softmax和负采样？

层次 softmax (Hierarchical Softmax) 和负采样 (Negative Sampling) 都是用于解决大规模分类任务的方法，特别在自然语言处理 (NLP) 中广泛应用于词向量模型，如 Word2Vec。

层次 softmax 是一种基于二叉树的分类方法。在传统的 softmax 中，需要计算所有类别的概率分布，计算复杂度为 $O(V)$ ，其中 V 是类别（词汇）的数量。而层次 softmax 使用了一个二叉树结构，将类别组织成一个层次化的结构。在预测时，通过沿着二叉树的路径移动来计算目标类别的概率分布，从而减少了计算复杂度。层次 softmax 的优点是可以加速模型的训练和推断过程，特别适用于大规模分类任务。

负采样是另一种解决大规模分类任务的方法，其思想是通过训练一个二分类模型，将目标类别与一组随机采样的负类别进行区分。具体过程如下：对于每个训练样本，随机选择一些负类别（通常是根据词频进行采样），将目标类别标记为正例，负类别标记为负例，然后使用二分类模型（如 logistic 回归）进行训练。负采样的优点是大大降低了计算复杂度，因为只需要计算少量的正负例对的概率，而不需要计算所有类别的概率分布。

1.19 Add&norm层的作用？

归一化：通过对输入进行归一化操作，使得输入的均值和方差保持稳定，有助于加速模型的收敛和提高训练效果。归一化可以缓解梯度消失和梯度爆炸等问题，提高模型的数值稳定性。

残差连接：Add Norm层通常与残差连接一起使用，用于将前一层的输出与当前层的输出相加。残差连接允许信息在网络中直接传递，有助于缓解梯度消失、加速训练过程和提高模型的表达能力。

1.20 rouge指标是怎么计算的？

ROUGE是一组用于自动评估文本摘要生成质量的指标。ROUGE指标包括ROUGE-N和ROUGE-L。

ROUGE-N衡量生成摘要中的N-gram重叠程度。具体计算步骤如下：

- 预处理：对参考摘要和生成摘要进行预处理，包括分词和去除标点符号等。
- 计算N-gram：将参考摘要和生成摘要分别转换为N-gram序列。N-gram是由连续的N个单词组成的序列。
- 计算重叠：统计生成摘要中与参考摘要重叠的N-gram数量。这包括计算匹配的N-gram数和参考摘要中的总N-gram数。
- 计算精确率、召回率和F1值：使用计算得到的匹配的N-gram数、参考摘要中的N-gram数和生成摘要中的N-gram数，计算精确率（Precision）、召回率（Recall）和F1值。

ROUGE-L衡量生成摘要与参考摘要的最长公共子序列。具体计算步骤如下：

- 预处理：对参考摘要和生成摘要进行预处理，包括分词和去除标点符号等。
- 计算最长公共子序列：通过动态规划算法计算生成摘要和参考摘要之间的最长公共子序列。
- 计算精确率、召回率和F1值：使用计算得到的最长公共子序列长度、参考摘要的长度和生成摘要的长度，计算精确率、召回率和F1值。

1.21 介绍一下BERT？ BERT和BiLSTM 有什么区别？ BERT 和 Attention是什么关系？ 解释Q， K， V分别的作用？

- BERT是一款NLU模型，主要模型架构采用了Transformer的Encoder部分，基础配置有12层Encoder Block。
- 两个模型有多点区别
 - BiLSTM双向模型，但是信息流只能从一段逐步向另一端传递，步长较慢；BERT通过注意力机制将所有的步长消除，可以视作一次性计算了所有信息的融合。
 - BiLSTM的梯度消失问题随着序列长度而累积；BERT在序列方向不产生梯度消失的累积。
 - BiLSTM本身具备信息流的位置概念，x和y两个token天然的具备先后顺序；BERT必须加入位置编码

才能区分x和y，不然退化成词袋模型。

- BERT模型采用self-attention的内部机制，属于Q=K=V这种特殊情况下的一种Attention机制。
- Q即Query，代表查询张量，K即Key，代表关键信息张量。Q和K的交互计算可以通过注意力机制进行信息流的匹配；将匹配后的结果和V=Value进行取值，得到最终我们想要的结果。

1.22 词向量是什么，有哪些方式生成词向量，句子的词向量是什么

词向量是自然语言的数字化表示，以方便计算机进行处理。

词向量主要有3种模式：

- one-hot
- Word2Vec (Glove)
- Word Embedding (BERT系列)

句子的词向量：

- 可以通过分词后，每个单词词向量的加权平均。
- 也可以直接采用BERT输出层的语义向量，作为整个句子的词向量(本质上就是句向量)

1.23 说一下前缀树？

前缀树，又称单词查找树，Trie树，是一种树形结构，是一种哈希树的变种。

典型应用是用于统计，排序和保存大量的字符串，所以经常被搜索引擎系统用于文本词频统计。它的优点是利用字符串的公共前缀来减少查询时间，最大限度地减少无谓的字符串比较，查询效率比哈希树高。

Trie的核心思想是空间换时间，利用字符串的公共前缀来降低查询时间的开销以达到提高效率的目的。

前缀树(Trie树)有3个基本特性：

- 根节点不包含字符，除根节点外每一个节点都只包含一个字符。
- 从根节点到某一节点，路径上经过的字符连接起来，为该节点对应的字符串。
- 每个节点的所有子节点包含的字符都不相同。

搜索前缀树的方法：

从根结点开始一次搜索。

- 取得要查找关键词的第一个字母，并根据该字母选择对应的子树并转到该子树继续进行检索。
- 在相应的子树上，取得要查找关键词的第二个字母，并进一步选择对应的子树进行检索。
- 迭代上述过程.....
- 在某个结点处，关键词的所有字母已被取出，则读取附在该结点上的信息，完成查找

Python 复制代码

```
1 class Trie:
2     # 初始化
3     def __init__(self):
4         self.root = {}
5         self.end = '#'
6
7     # 添加单词
8     def add(self, word):
9         node = self.root
10        for char in word:
11            node = node.setdefault(char, {})
12        node[self.end] = None
13
14    # 搜索单词是否存在
15    def search(self, word):
16        node = self.root
17        for char in word:
18            if char not in node: return False
19            node = node[char]
20        return self.end in node
```

1.24 讲讲迁移学习，特征迁移和微调迁移？

迁移学习主要用于AI构建新模型时，将已有的预训练模型作为起点，直接继承过来的方法。可以达到站在巨人肩膀上的功效。

Feature-representation-transfer(特征迁移): 指找到一些好的有代表性的特征，通过特征变换把source domain和target domain的特征变换到同样的空间，使得这个空间中source domain和target domain的数据具有相同的分布。

所谓的微调迁移即Fine tuning，指将预训练模型集成为自己的训练起点，在特定的数据集上进行二次微调训练。

1.25 bert和transformer的区别？

BERT是基于Transformer架构的模型，采用了Transformer Encoder的部分，舍弃了Decoder的部分。

BERT的编码器包含12层Encoder Block，而2017版本的Transformer包含6层Encoder Block。

BERT中的位置编码是可训练的，而Transformer中是写死不变的。

Transformer中的激活函数采用Relu，BERT中的激活函数采用Gelu

1.26 bert怎么解决超级长的文本 如果不截断呢？了解XLnet么？

BERT限制文本序列长度不超过512(本质不超过510)，在面对更长的文本时可以有以下几种方案：

- 截断只取前面510个字符
- 截断只取后面510个字符

不截断的方法：

- 将超长文本进行切分，每段510个字符，分别处理。然后平均值输出。

XLNet是经典的排列语言模型，采用特殊的mask机制将token前面，后面的信息都展示给token，相当于可以同时“看见”前面的信息和后面的信息。XLNet也可以应对更长的序列长度

1.27 Word2vec是怎么计算的，点乘还是加权求和？

隐藏层张量是通过加权平均得到的。

1.28 霍夫曼树如何训练Word2vec的，介绍一下？

首先对全体语料进行词频统计。

根据单词的词频，构建霍夫曼树。每个单词都是哈夫曼树的叶子节点，词频对应相应节点的权值。

真实的训练中，采用层次softmax，负采样两种优化方法。

在霍夫曼树中，隐藏层到输出层的softmax映射不是一下子完成的，而是沿着霍夫曼树一步步完成的，因此这种softmax取名为“Hierarchical Softmax”

- 根节点对应词向量w。

- 所有中间节点对应于一个逻辑回归的判断，将路径标记为0或1.
- 所有的叶子节点对应最终的单词x.
- 从叶子节点x到根节点root的路径上的概率累乘，就是单词出现的概率.

利用哈夫曼树的两大优点:

- 计算量从N, 降低到logN
- 哈夫曼树越靠近根节点, 路径越短, 参数更新越快, 达到加速训练和收敛的目的

1.29 有论文说bert不加crf效果和加了crf效果差不多，有没有看相关论文？ 你自己预想只用bert和 bert+crf来做ner会有什么样的差别

- 有看论文，在特定数据集上的结果不能推广到生产实际中.
- BERT + CRF会让模型更快的收敛，可以用更小的数据集来得到最优结果.

2. 投满分项目面试题

2.1 了解那些激活函数？详细讲一下你用到的激活函数GeLU

常见的激活函数包括 Sigmoid、ReLU、Leaky ReLU、Tanh 等。

- Sigmoid 函数将输入值映射到 0 到 1 之间，常用于二分类问题。
- ReLU 函数在输入大于 0 时保持原样，小于 0 时为 0，它在深度神经网络中广泛使用。
- Leaky ReLU 是 ReLU 的一种变体，对于小于 0 的输入，它会有一个小的斜率，避免了神经元的死亡。
- Tanh 函数将输入值映射到-1 到 1 之间，类似于 Sigmoid，但具有更平滑的过渡。

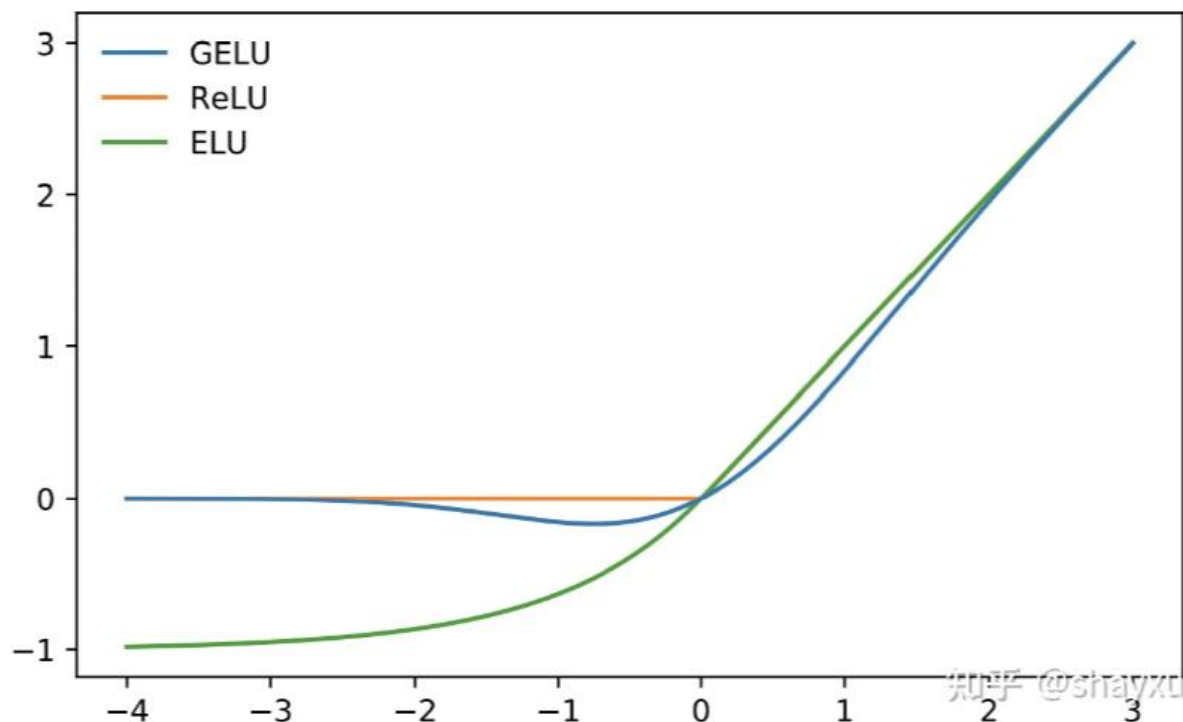
关于 GeLU (Gaussian Error Linear Unit) 激活函数，它是一种近来在深度学习中受到关注的激活函数。GeLU 函数的定义如下：

$$GELU(x) = \frac{1}{2} [1 + \tanh(\sqrt{\frac{2}{\pi}} (x + 0.047715x^3))]$$

或者

$$GELU(x) = x * \sigma(1.702x)$$

其中，Gelu 函数在输入值较大时具有类似 ReLU 的效果，而在输入值接近零的时候具有更平滑的过渡。这使得它在一些情况下能够提供更好的性能，尤其是在处理 tid/uuid 数据时。



GELU的优点

它在处理负数时不会像ReLU一样将输入裁剪到0，这可能导致梯度消失的问题。

- 具有更光滑的导数：
 - GELU函数的导数是连续的，这使得在训练深度神经网络时可以更容易地传播梯度，避免了ReLU函数在0处的导数不连续的问题，从而减少了训练过程中出现的梯度消失问题
- 可以加速收敛：
 - GELU函数在激活函数的非线性变换中引入了类似于sigmoid函数的变换，这使得GELU函数的输出可以落在一个更广的范围内，有助于加速模型的收敛速度。

2.2 项目的构建过程？

项目的构建过程通常包括以下步骤：

- 问题定义：明确项目的目标 and 需求，确定要解决的问题。
- 数据收集：收集与问题相关的数据，可能需要从多个来源获取数据，并进行数据清洗和预处理。
- 数据预处理：对数据进行清洗、转换、缩放等操作，使其适合模型的输入。
- 模型选择与训练：根据问题的特点选择合适的模型架构，并使用训练数据对模型进行训练
- 模型评估：使用验证集或测试集对训练好的模型进行评估，衡量模型的性能。
- 调整与优化：根据评估结果，对模型进行调整，如调整超参数、尝试不同的模型结构等，以提高

性能。

- 部署与应用：将训练好的模型部署到实际应用中，可能需要将模型转换为特定的格式，并与其他系统或模块进行集成。

2.3 Bert模型的构建？

Bert 模型是一种基于预训练的语言模型，通常使用大量的文本数据进行预训练。构建 Bert 模型的过程包括以下步骤：

- 数据准备：获取大量的文本数据，例如书籍、文章、网页等。
- 模型架构设计：选择合适的深度学习架构，如 Transformer 架构，用于处理文本数据。
- 预训练：使用无监督学习方法，在大量文本上进行预训练，学习语言的统计规律和语义表示。
- 微调：将预训练好的模型在特定任务上进行微调，例如分类、情感分析等，以适应具体的应用场景。

2.4 了解哪些模型压缩的方法？详细讲讲一下模型量化

模型压缩的方法有很多种，模型量化是其中一种重要的方法。模型量化的原理是将模型的参数从高精度表示转换为低精度表示，例如将 32 位浮点数转换为 8 位整数。这样可以减少模型的存储和计算开销，同时保持一定的准确性。量化的过程通常包括以下步骤：

- 量化策略选择：确定要使用的量化方法，如整数量化、浮点数量化等。
- 量化参数：对模型的参数进行量化，将高精度值映射到低精度的量化值。
- 校准：对量化后的参数进行校准，以补偿量化带来的误差。
- 推理优化：在量化后的模型上进行推理优化，如使用低精度计算、剪枝等技术。

(1) 模型量化

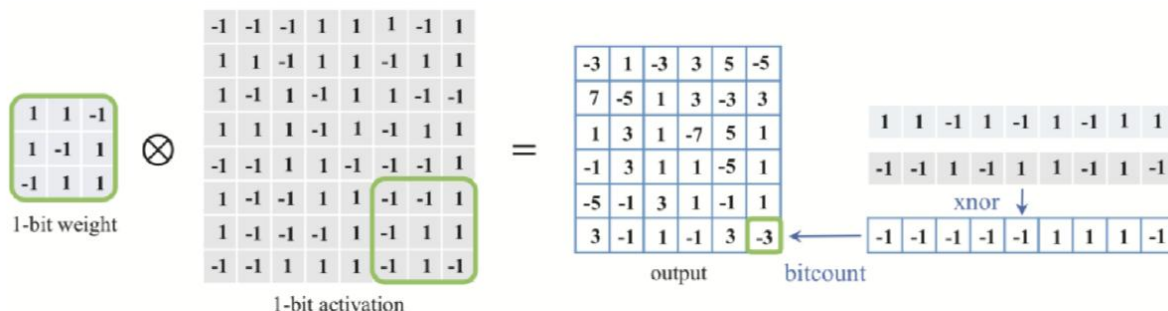
为了保证较高的精度，大部分的科学运算都是采用浮点型进行计算，常见的是32位浮点型和64位浮点型，即float32和double64。

对于深度学习模型来说，乘加计算量是非常大的，往往需要GPU等专用的计算平台才能实现实时运算，这对于端上产品来说是不可接受的，而模型量化是一个有效降低计算量的方法。

量化，即将网络的权值，激活值等从高精度转化成低精度的操作过程，例如将32位浮点数转化成8位整型数int8，同时我们期望转换后的模型准确率与转化前相近。

- 模型量化优势
 - 更小的模型尺寸。以8bit量化为例，与32bit浮点数相比，我们可以将模型的体积降低为原来的四分之一，这对于模型的存储和更新来说都更有优势。
 - 更低的功耗。移动8bit数据与移动32bit浮点型数据相比，前者比后者高4倍的效率，而在一定程度上内存的使用量与功耗是成正比的。

- 更快的计算速度。相对于浮点数，大多数处理器都支持8bit数据的更快处理，如果是二值量化，则更有优势。



(2) 模型蒸馏

一般地，大模型往往是单个复杂网络或者是若干网络的集合，拥有良好的性能和泛化能力，而小模型因为网络规模较小，表达能力有限。利用大模型学习到的知识去指导小模型训练，使得小模型具有与大模型相当的性能，但是参数数量大幅降低，从而可以实现模型压缩与加速，就是知识蒸馏与迁移学习在模型优化中的应用。

Hinton等人最早在文章“Distilling the knowledge in a neural network”[1]中提出了知识蒸馏这个概念，其核心思想是一旦复杂网络模型训练完成，便可以用另一种训练方法从复杂模型中提取出来更小的模型，因此知识蒸馏框架通常包含了一个大模型(被称为teacher模型)，和一个小模型(被称为student模型)。

以计算机视觉模型的训练为例，我们经常用在ImageNet上训练的模型作为预训练模型，之所以可以这样做，是因为深度学习模型在网络浅层学习的知识是图像的色彩和边缘等底层信息，某一个数据集学习到的信息也可以应用于其他领域。

知识蒸馏或者说迁移学习的必要性

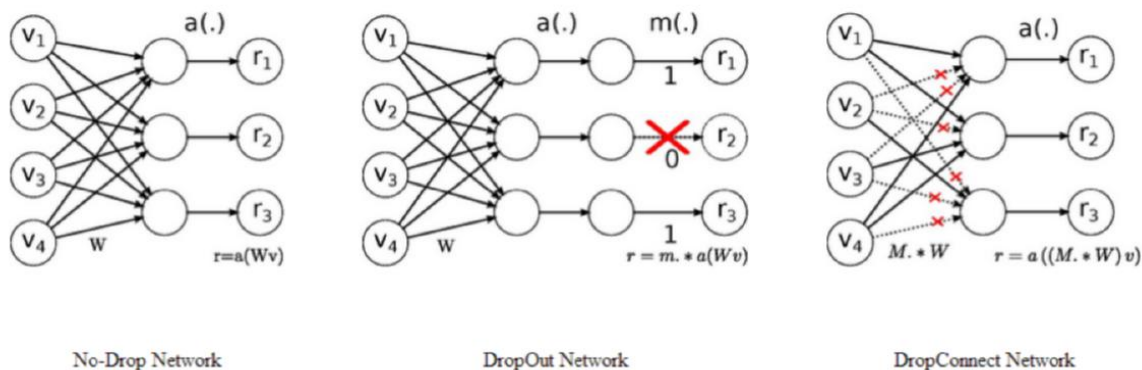
- **数据分布差异。**深度学习模型的训练场景和测试场景往往有分布差异，以自动驾驶领域为例，大部分数据集的采集都是基于白天，光照良好的天气条件下，在这样的数据集上训练的模型，当将其用于黑夜，风雪等场景时，很有可能会无法正常工作，从而使得模型的实用性能非常受限。因此，必须考虑模型从源域到目标域的迁移能力。
- **受限的数据规模。**数据的标注成本是非常高的，导致很多任务只能用少量的标注进行模型的训练。以医学领域为典型代表，数据集的规模并不大，因此在真正专用的模型训练之前往往需要先在通用任务上进行预训练。
- **通用与垂直领域。**虽然我们可以训练许多通用的模型，但是真实需求是非常垂直或者说个性化的，比如ImageNet存在1000类，但是我们可能只需要用到其中若干类。此时就可以基于1000类ImageNet模型进行知识迁移，而不需要完全从头开始训练。

因此，在工业界对知识蒸馏和迁移学习也有着非常强烈的需求。

(3) 模型剪枝

深度学习网络模型从卷积层到全连接层存在着大量冗余的参数，大量神经元激活值趋近于0，将这些神经元去除后可以表现出同样的模型表达能力，这种情况被称为过参数化，而对应的技术则被称为模型剪枝。

Dropout和DropConnect代表着非常经典的模型剪枝技术



- Dropout中随机的将一些神经元的输出置零，这就是神经元剪枝。
- DropConnect则随机的将一些神经元之间的连接置零，使得权重连接矩阵变得稀疏，这便是权重连接剪枝。它们就是最细粒度的剪枝技术，只是这个操作仅仅发生在训练中，对最终的模型不产生影响，因此没有被称为模型剪枝技术。

常用模型剪枝方式：

```
import torch.nn.utils.prune as prune

...
print(list(module.name_parameters()))
print(list(module.name_buffers())) # [weight_mask]
print(module.weight)
print(module.bias)
print(module.state_dict().keys()) # OrderedDict keys(['weight_orig', 'bias', 'weight_mask'])

prune.remove(module, name='weight')
print(list(module.name_buffers())) # []
print(module.state_dict().keys()) # OrderedDict keys(['weight', 'bias'])
```

- 对特定网络模块的剪枝 (Pruning Model)

```
# 随机非结构化剪枝
prune.random_unstructured(module, name='weight', amount=0.3)
prune.l1_unstructured(module, name='weight', amount=0.3) # L1剪枝

# 随机结构化剪枝
prune.ln_structured(module, name='weight', amount=0.3, n=2, dim=1) # Ln剪枝
```

- 多参数模块的剪枝 (Pruning multiple parameters)

```
for name, module in model.name_modules():
    if isinstance(module, torch.nn.Conv2d):
        prune.l1_unstructured(module, name='weight', amount=0.2)
    elif isinstance(module, torch.nn.Linear):
        prune.ln_structured(module, name='weight', amount=0.3, n=2, dim=0)
```

- 全局剪枝

```
parameters_to_prune = (
    (model.conv1, 'weight'),
    (model.conv2, 'weight'),
    (model.fc1, 'weight'),
    (model.fc2, 'weight'),
    (model.fc3, 'weight'))
prune.global_unstructured(parameters_to_prune, pruning_method=prune.L1Unstructured,
amount=0.2)
# 统计: float(model.fc1.weight == 0)/float(model.fc1.weight.nelement())
```

- 用户自定义剪枝 (Custom pruning)

```
class myself_pruning_method(prune.BasePruningMethod):
    PRUNING_TYPE = 'unstructured'

    def compute_mask(self, t, default_mask):
        mask = default_mask.clone()
        mask.view(-1)[:2] = 0
        return mask

def myself_unstructured_pruning(module, name):
    myself_pruning_method.apply(module, name)
    return module

myself_unstructured_pruning(model.fc3, name='bias')
```

2.5 了解那些机器学习算法？详细讲一下你用到的随机森林的算法原理

常见的机器学习算法包括决策树、随机森林、支持向量机、朴素贝叶斯、线性回归等。

- 随机森林算法的原理是通过构建多个决策树，并对这些决策树的预测结果进行集成（例如平均或投票）来进行预测。
- 随机森林通过随机选择特征和样本进行训练，从而减少过拟合的风险。

具体来说，随机森林的构建过程包括以下步骤：

- 随机抽样：从原始数据中随机抽取多个样本，用于构建决策树。
- 构建决策树：对于每个抽样得到的样本，使用剩余的特征构建决策树。
- 决策树集成：将多个决策树的预测结果进行集成，得到最终的预测结果。

2.6 讲一个你认为最能体现你各方面能力的项目？重点讲解一下做项目过程中的优化思路

我可以分享一个项目，在该项目中，我应用了多种技术和方法来优化模型的性能。例如，我采用了以下优

化思路：

- 数据增强：通过对原始数据进行随机旋转、缩放、翻转等操作，增加数据的多样性。
- 超参数调整：仔细选择合适的超参数，如学习率、正则化参数等，通过网格搜索或随机搜索进行优化。
- 模型集成：使用多个不同的模型，如多个随机森林或不同的深度学习模型，然后将它们的结果集成起来，提高预测的稳定性和准确性。
- 特征工程：对数据进行特征提取和选择，选择最相关和有信息量的特征，以提高模型的性能。
- 监控和可视化：使用监控指标和可视化工具来跟踪模型的性能，及时发现问题并进行调整。

2.7 你们模型部署是用什么框架的？具体讲讲模型部署过程

模型部署可以使用多种框架，例如 TensorFlow Lite、PyTorch Mobile、ONNX、NCNN等。模型部署的过程包括将训练好的模型转换为特定框架的格式，进行优化和压缩，然后将模型集成到应用程序中。具体步骤可能包括：

- 模型转换：将训练好的模型转换为目标框架支持的格式。
- 优化与压缩：对模型进行优化，如剪枝、量化等，以减少模型的大小和计算量。
- 部署到目标平台：将模型集成到应用程序中，并在目标平台上进行部署和测试。

2.8 你们的数据是哪来的？数据内容是什么？

数据的来源会根据具体的项目而有所不同。

- 内部数据集，例如公司内部收集的数据；
- 公开数据集，例如学术研究机构或行业组织发布的数据集；通过爬虫采集数据等。
- 数据的内容也会因项目的需求而异，例如图像数据、文本数据、音频数据、视频数据等。

2.9 项目中算法的损失函数是什么？优化器？

项目中使用的损失函数和优化器会根据算法和问题的不同而有所选择。

常见的损失函数包括均方误差、交叉熵等。优化器可以是随机梯度下降（SGD）、Adagrad、Adadelata 等。选择合适的损失函数和优化器可以帮助模型在训练过程中更好地收敛。

- 对于分类问题，交叉熵损失函数常被用于衡量预测结果与真实标签之间的差异；
- 对于回归问题，均方误差可能是一个合适的选择。

优化器的选择则取决于模型的特点和数据的规模。

(1) 交叉熵损失函数

交叉熵是信息论中的一个重要概念，主要用于度量两个概率分布间的差异性。在机器学习中，交叉熵表示真实概率分布与预测概率分布之间的差异。交叉熵的值越小，模型预测效果就越好。

以二分类问题为例，假设，正例为，反例为，则交叉熵损失函数表达式为：

$$H(p, q) = - \sum_{i=1}^n p(x_i) \log(q(x_i))$$

$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

(2) AdamW优化器

AdamW其实是在Adam的基础上加入了weight decay正则化

在Adam当中，weight decay是直接加入到梯度当中：

$$g_t = g_t + \lambda \theta_{t-1}$$

其中 g_t 是当前step的梯度， θ_{t-1} 是上一个step中的模型权重， λ 是正则化系数。

而在AdamW中，正则化变成了：

$$\theta_t = \theta_{t-1} - \gamma \lambda \theta_{t-1}$$

其中 γ 是学习率。

2.10 项目中的评估指标有哪些？都是怎么计算的？

评估指标的选择取决于项目的具体目标和问题。

常见的评估指标包括准确率、召回率、F1 值、均方误差、平均绝对误差等。这些指标的计算方式会根据具体的问题和数据而有所不同。

- 在分类问题中，准确率可以用来衡量正确分类的样本比例；
- 在回归问题中，均方误差可以衡量预测值与真实值之间的平均差异。

以下涉及到的参数解释：

- TP (True Positive)：真正例
- TN (True Negative)：真负例
- FP (False Positive)：假正例
- FN (False Negative)：假负例

(1) 准确率 Accuracy

准确率的定义是预测正确的结果占总样本的百分比，其公式如下：

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

虽然准确率可以判断总的正确率，但是在样本不平衡的情况下，并不能作为很好的指标来衡量结果。举个简单的例子，比如在一个总样本中，正样本占90%，负样本占10%，样本是严重不平衡的。对于这种情况，我们只需要将全部样本预测为正样本即可得到90%的高准确率，但实际上我们并没有很用心的分类，只是随便无脑一分而已。这就说明了：由于样本不平衡的问题，导致了得到的高准确率结果含有很大的水分。即如果样本不平衡，准确率就会失效。

(2) 精准率 Precision

精准率 (Precision) 又叫查准率，它是针对预测结果而言的，它的含义是在所有被预测为正的样本中实际为正的样本的概率，意思就是在预测为正样本的结果中，我们有多少把握可以预测正确，其公式如下：

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

精准率代表对正样本结果中的预测准确程度，而准确率则代表整体的预测准确程度，既包括正样本，也包括负样本。

(3) 召回率 Recall

召回率 (Recall) 又叫查全率，它是针对原样本而言的，它的含义是在实际为正的样本中被预测为正样本的概率，其公式如下：

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

召回率的应用场景：比如拿网贷违约率为例，相对好用户，我们更关心坏用户，不能错过任何一个坏用户。因为如果我们过多的将坏用户当成好用户，这样后续可能发生的违约金额会远超过好用户偿还的借贷利息金额，造成严重偿失。召回率越高，代表实际坏用户被预测出来的概率越高，它的含义类似：宁可错杀一千，绝不放过一个。

(4) F1值

我们希望精准率和召回率同时都非常高。但实际上这两个指标是一对矛盾体，无法做到双高。如果其中一个非常高，另一个肯定会非常低。选取合适的阈值点要根据实际需求，比如我们想要高的查全率，那么我们会牺牲一些查准率，在保证查全率最高的情况下，查准率也不那么低。在实际情况中，不会有分类器仅仅以精确度(Precision)或者召回率(Recall)作为单一的度量标准，而是使用这两者的调和平均，于是就有了F值(F-Score)，F1分数同时考虑了查准率和查全率，让二者同时达到最高，取一个平衡。

$$\text{F1 score} = 2 \text{ Precision} * \text{Recall} / (\text{Precision} + \text{Recall})$$

2.11 你们模型服务封装是用什么框架的？具体讲讲模型部署过程

Flask 是一个轻量级的 Python Web 框架，常用于 Web 服务的开发。以下是一个简单的 Flask 服务封装流程示例：

1. 部署准备：

- Flask: Python Web 开发中较流行的框架之一，其特点是轻量级。
- Gunicorn: Flask 应用是一个符合 WSGI 规范的 Python 应用，不能单独运行，需要依赖其他的组件提供服务器功能。
- Gevent: Gunicorn 默认使用同步阻塞的网络模型（-k sync），对于高并发的访问并不友好，所以需要使gevent 来提高并发量。
- Docker: 可以把它理解为一个“盒子”，有时候会倦于管理项目的部署和维护。如果使用容器封装项目，那么只需要维护一个配置文件即可完成部署需求，包括后续将整个部署的过程完全自动化，部署就会变得更便捷。

2. 构建配置文件:

- 创建 requirements.txt 文件: 作为 python 项目常用到的一个文件，可以让项目中的 Python 环境对依赖包的安装。
- 构建 Gunicorn 配置文件: Flask 应用不能独立运行，需要依赖其他组件提供服务器功能，所以需要构建Gunicorn 配置文件。
- 创建 Dockerfile 文件: 以便构建 Docker 镜像。

3. 封装服务:

- 将 python 服务做成给定参数输入和输出的格式。
- 在 python 里面导入 flask 包，以及可能用到的 request, jsonify 等。
- `app = Flask(name)`, `app` 是 Flask 的实例，它接收包或者模块的名字作为参数，但一般都是传递 `__name__`。
- 在每一个运行的接口服务前面用`@app.route()`的方式规定好接口路径和接口请求方式（一般为 post 或者get）。
- 定义函数具体内容，接收相关参数并进行具体逻辑处理。
- 返回 JSON 格式的内容。
- 用`app.run()`来确定主机和端口号。

2.12 KL散度损失和交叉熵损失有什么区别？

KL散度 (Kullback-Leibler Divergence) 和交叉熵 (Cross Entropy) 都是衡量两个概率分布之间差异的指标。它们的区别和联系如下：

- 区别：
 - 定义不同: KL散度是用来衡量两个概率分布之间的差异的一种度量方式；交叉熵衡量了模型预测的概率分布与真实概率分布之间的差异。
 - 性质不同: KL散度不满足交换律和三角不等式；交叉熵满足交换律
 - 应用不同: KL散度通常用于比较两个概率分布之间的差异，例如在无监督学习中用于评估生成模

型的性能；交叉熵通常用于衡量模型预测和真实标签之间的差异，例如，在分类任务中，交叉熵被用作损失函数，以衡量模型预测的类别分布和真实标签之间的差异。

- 联系：

- KL散度可以被用于计算代价，而在特定情况下最小化KL散度等价于最小化交叉熵，而交叉熵的运算更简单，所以用交叉熵来当做代价。

交叉熵通常用于监督学习任务中，如分类和回归等。在这些任务中，我们有一组输入样本和相应的标签。我们希望训练一个模型，使得模型能够将输入样本映射到正确的标签上。

在这种情况下，我们可以使用交叉熵作为损失函数。假设我们有一个模型预测的输出分布为 p ，真实标签的分布为 q 。那么交叉熵的公式如下：

$$H(p, q) = - \sum_i q_i \log p_i$$

i 表示可能的类别或事件， p_i 和 q_i 分别表示真实概率分布和模型预测的概率分布中类别 i 的概率。

KL散度通常用于无监督学习任务中，如聚类、降维和生成模型等。在这些任务中，我们没有相应的标签信息，因此无法使用交叉熵来评估模型的性能，所以需要一种方法来衡量模型预测的分布和真实分布之间的差异，这时就可以使用KL散度来衡量模型预测的分布和真实分布之间的差异。KL散度的公式如下：

$$D_{KL}(p||q) = \sum_i p_i \log \frac{p_i}{q_i}$$

i 表示概率分布中的一个可能的事件或状态。 p_i 和 q_i 分别表示真实概率分布和模型预测的概率分布中事件 i 的概率。KL散度衡量了模型预测的概率分布与真实概率分布之间的差异，即模型在预测上的不确定性与真实情况的不确定性之间的差距。

一般情况下：交叉熵通常用于监督学习任务中，KL散度通常用于无监督学习任务中。当我们有相应的标签信息时，应该使用交叉熵来评估模型的性能；当我们没有相应的标签信息时，使用KL散度可以衡量模型预测的分布和真实分布之间的差异。

2.13 fit(),transform(),fit_transform()有什么区别？

相同点：都是数据预处理中的方法

不同点：

- fit(): Method calculates the parameters μ and σ and saves them as internal objects. 解释：简单来说，就是求得训练集X的均值，方差，最大值，最小值,这些训练集X固有的属性。

- transform(): Method using these calculated parameters apply the transformation to a particular dataset. 解释：在fit的基础上，进行标准化，降维，归一化等操作（看具体用的是哪个工具，如PCA，StandardScaler等）。
- fit_transform(): joins the fit() and transform() method for transformation of dataset. 解释：fit_transform是fit和transform的组合，既包括了训练又包含了转换。transform()和fit_transform()二者的功能都是对数据进行某种统一处理（比如标准化 $\sim N(0,1)$ ，将数据缩放(映射)到某个固定区间，归一化，正则化等）

fit_transform(trainData)对部分数据先拟合fit，找到该part的整体指标，如均值、方差、最大值最小值等等（根据具体转换的目的），然后对该全部trainData进行转换transform（补缺失值等），从而实现数据的标准化、归一化等等。

3. 知识图谱项目面试题

3.1 医疗数据来自于哪里，原始数据什么格式？

- 医院病人的真实数据
- 病史特点、出院情况、检查项目、诊疗经过

3.2 使用了什么标注工具，如何分配的标注任务？

- doccano
- 先admin再授权

3.3 使用了什么标注规则，详细解释一下

BIO，分别是开头、中间结尾、非实体

3.4 如何实现数据预处理，中间经过了哪些步骤？

- 先把数据组织成：每个token对应一个标签
- 再根据标点符号「。！？」进行句子分段
- 一个句子序列对应一个标签序列
- 通过dataloader加载一个batch的句子标签

3.5 对比了NER哪些方法，最后选择了哪一套模型组合，为什么这么选？

最终选择了「bert+bilstm+crf」 or 「bert+crf」

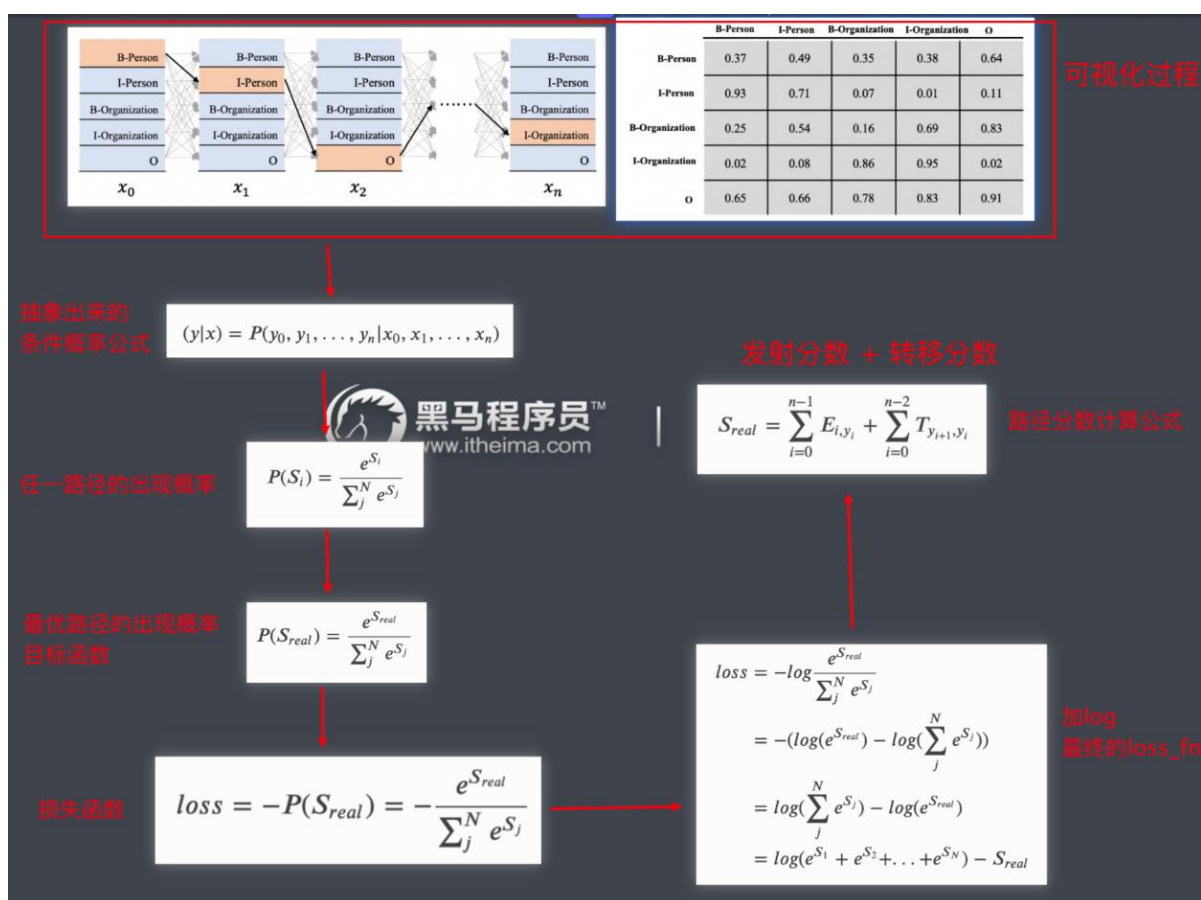
3.6 有哪些方法可以实现关系抽取，比较一下他们的原理

- 基于规则：基于文本语料构建知识库
- pipeline方法：实体抽取、关系抽取放在两个模型实现，关系抽取的好坏依赖实体抽取
- joint方法：在一个模型内部实现实体关系抽取

3.7 如何推导CRF损失函数？

解析：

两部分：归一化项 + 负真实路径的分数



3.8 CRF的路径分数是怎么计算的？

发射分数之和 + 转移分数之和

3.9 实现关系抽取过程中，都遇到了哪些棘手问题？

- SEO (Single Entity Overlap)：单一实体重叠问题，指同一个实体参与到多个关系中。比如在文本中，“北京是中国的首都”以及“北京举办过奥运会”，实体“北京”参与了两种不同关系。
- EPO (Entity Pair Overlap)：实体对重叠问题，指同一个实体对参与到了多个关系中。例如“小明是小红的朋友”以及“小明是小红的同事”，“小明”和“小红”这一实体对存在两种关系。

3.10 比较pipeline和joint方法的不同

关键点：pipeline是多个模型；joint是一个模型

3.11 在哪些场景用过基于规则方法，核心是什么，用了哪些工具？

- 简单场景
- 核心是构建知识库、词性标注、正则匹配
- jieba.posseg、正则

3.12 基于pipeline实现关系抽取，如何组织数据的，中间经过了哪些过程？

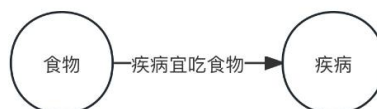
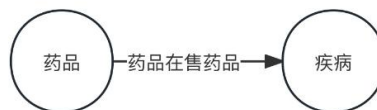
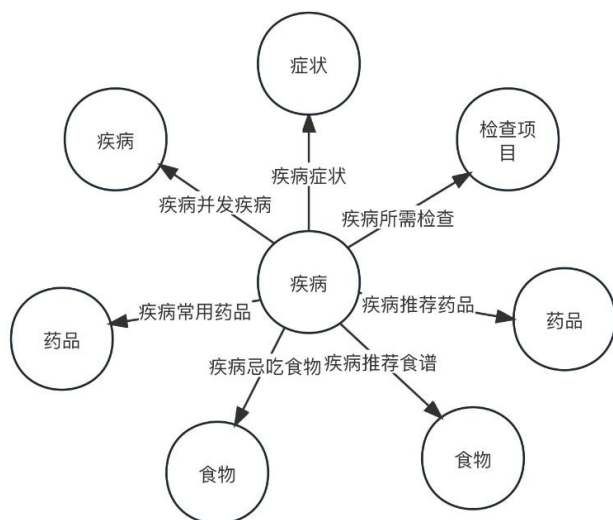
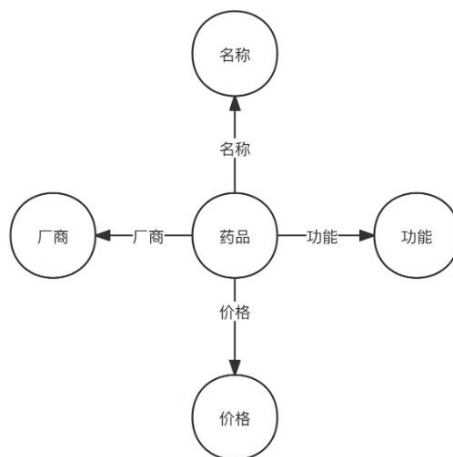
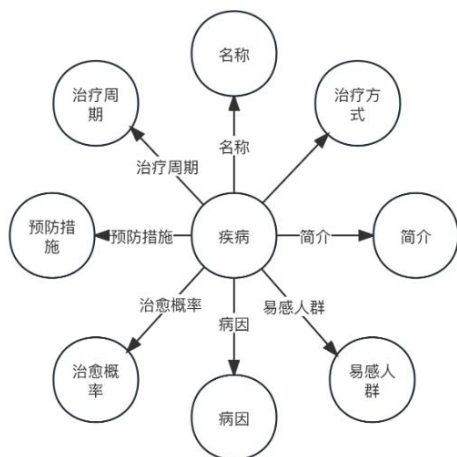
- 数据格式：实体1 + 实体2 + 关系类型 + 原句
- 直接在实体识别预测中拿到实体组合，再用doccano句子文本分类标注

3.13 如何部署上线的，用了哪些工具？

- Flask
- neo4j

3.14 如何定义的知识图谱的本体和实例？

- 本体就是模式层 or 概念层



- 实例就是具体的例子：eg (姚明，出生地，上海)

七、大语言模型

1. 大语言模型基础

1.1 目前主流的开源模型体系有哪些？

目前主流的开源LLM（语言模型）模型体系包括以下几个：

- GPT (Generative Pre-trained Transformer) 系列：由OpenAI发布的一系列基于Transformer架构的语言模型，包括GPT、GPT-2、GPT-3等。GPT模型通过在大规模无标签文本上进行预训练，然后在特定任务上进行微调，具有很强的生成能力和语言理解能力。
- BERT (Bidirectional Encoder Representations from Transformers)：由Google发布的一种基于Transformer架构的双向预训练语言模型。BERT模型通过在大规模无标签文本上进行预训练，然后在下游任务上进行微调，具有强大的语言理解能力和表征能力。
- LLaMA模型：
是一系列具有不同参数量的大规模语言模型，参数范围从70亿到650亿不等。由Meta公司（前身为Facebook）开发的一系列大规模语言模型
- 阿里云Qwen2：
这是由阿里云发布的高性能开源模型，性能超越了许多中国的闭源模型，成为全球开发者的一个主流选择。
- AliceMind：
达摩院（阿里巴巴的研究机构）开源的深度语言模型体系，覆盖了多语言、多模态和结构化数据等多个预训练语言模型。
- XLNet：由CMU和Google Brain发布的一种基于Transformer架构的自回归预训练语言模型。XLNet模型通过自回归方式预训练，可以建模全局依赖关系，具有更好的语言建模能力和生成能力。
- RoBERTa：由Facebook发布的一种基于Transformer架构的预训练语言模型。RoBERTa模型在BERT的基础上进行了改进，通过更大规模的数据和更长的训练时间，取得了更好的性能。
- T5 (Text-to-Text Transfer Transformer)：由Google发布的一种基于Transformer架构的多任务预训练语言模型。T5模型通过在大规模数据集上进行预训练，可以用于多种自然语言处理任务，如文本分类、机器翻译、问答等。

这些模型在自然语言处理领域取得了显著的成果，并被广泛应用于各种任务和应用中。

1.2 prefix LM 和 causal LM 区别是什么？

Prefix LM（前缀语言模型）和Causal LM（因果语言模型）是两种不同类型的语言模型，它们的区别在于生成文本的方式和训练目标。

- Prefix LM

Prefix LM其实是Encoder-Decoder模型的变体，为什么这样说？解释如下：

- 1) 在标准的Encoder-Decoder模型中，Encoder和Decoder各自使用一个独立的Transformer
- 2) 而在Prefix LM，Encoder和Decoder则共享了同一个Transformer结构，在Transformer内部通过Attention Mask机制来实现。

与标准Encoder-Decoder类似，Prefix LM在Encoder部分采用Auto Encoding (AE-自编码)模式，即前缀序列中任意两个token都相互可见，而Decoder部分采用Auto Regressive (AR-自回归)模式，即待生

成的token可以看到Encoder侧所有token(包括上下文)和Decoder侧已经生成的token,但不能看未来尚未产生的token。

下面的图很形象地解释了Prefix LM的Attention Mask机制(左)及流转过程(右)。

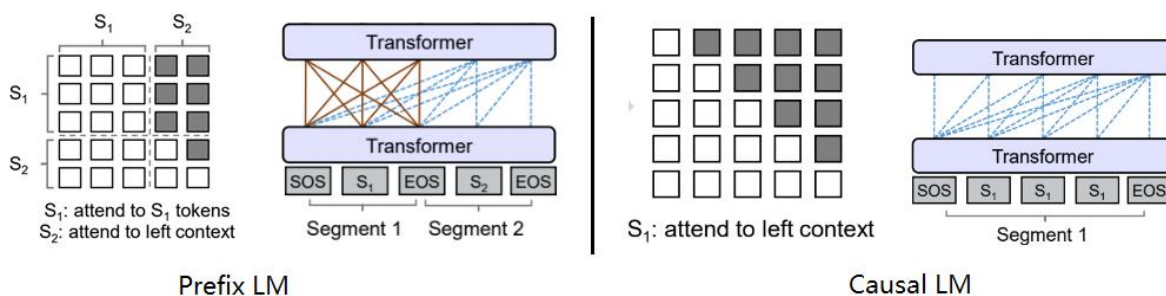
Prefix LM的代表模型有UniLM、T5、GLM(清华滴~)

● Causal LM

Causal LM是因果语言模型，目前流行地大多数模型都是这种结构，别无他因，因为GPT系列模型内部结构就是它，还有开源界的LLaMa也是。

Causal LM只涉及到Encoder-Decoder中的Decoder部分，采用Auto Regressive模式，直白地说，就是根据历史的token来预测下一个token，也是在Attention Mask这里做的手脚。

参照着Prefix LM，可以看下Causal LM的Attention Mask机制(左)及流转过程(右)。



● 总结

Prefix LM: 前缀语言模型是一种生成模型，它在生成每个词时都可以考虑之前的上下文信息。在生成时，前缀语言模型会根据给定的前缀（即部分文本序列）预测下一个可能的词。这种模型可以用于文本生成、机器翻译等任务。

Causal LM: 因果语言模型是一种自回归模型，它只能根据之前的文本生成后续的文本，而不能根据后续的文本生成之前的文本。在训练时，因果语言模型的目标是预测下一个词的概率，给定之前的所有词作为上下文。这种模型可以用于文本生成、语言建模等任务。

总结来说，前缀语言模型可以根据给定的前缀生成后续的文本，而因果语言模型只能根据之前的文本生成后续的文本。它们的训练目标和生成方式略有不同，适用于不同的任务和应用场景。

1.3 为何现在的大模型大部分是Decoder only结构

- Encoder的低秩问题：Encoder的双向注意力会存在低秩问题，这可能会削弱模型表达能力，就生成任务而言，引入双向注意力并无实质好处。
- 更好的Zero-Shot性能、更适合于大语料自监督学习：decoder-only 模型在没有任何 tuning 数据的情况下、zero-shot 表现最好，而 encoder-decoder 则需要一定量的标注数据上做 multitask

finetuning 才能激发最佳性能。

- 效率问题：decoder-only支持一直复用KV-Cache，对多轮对话更友好，因为每个Token的表示之和它之前的输入有关，而encoder-decoder和PrefixLM就难以做到。

1.4 LLMs复读机问题

1.4.1 什么是 LLMs 复读机问题？

LLMs复读机问题（LLMs Parroting Problem）是指大型语言模型在生成文本时过度依赖输入文本的复制，而缺乏创造性和独特性。当面对一个问题或指令时，模型可能会简单地复制输入文本的一部分或全部内容，并将其作为生成的输出，而不是提供有意义或新颖的回应。

1.4.2 为什么会出现 LLMs 复读机问题？

- 1) **数据偏差**：大型语言模型通常是通过预训练阶段使用大规模无标签数据进行训练的。如果训练数据中存在大量的重复文本或者某些特定的句子或短语出现频率较高，模型在生成文本时可能会倾向于复制这些常见的模式。
- 2) **训练目标的限制**：大型语言模型的训练通常是基于自监督学习的方法，通过预测下一个词或掩盖词来学习语言模型。这样的训练目标可能使得模型更倾向于生成与输入相似的文本，导致复读机问题的出现。
- 3) **缺乏多样性的训练数据**：虽然大型语言模型可以处理大规模的数据，但如果训练数据中缺乏多样性的语言表达和语境，模型可能无法学习到足够的多样性和创造性，导致复读机问题的出现。
- 4) **模型结构和参数设置**：大型语言模型的结构和参数设置也可能对复读机问题产生影响。例如，模型的注意力机制和生成策略可能导致模型更倾向于复制输入的文本。

1.4.3 如何缓解 LLMs 复读机问题？

为了缓解LLMs复读机问题，可以尝试以下方法：

- 1) **多样性训练数据**：在训练阶段，使用多样性的语料库来训练模型，避免数据偏差和重复文本的问题。这可以包括从不同领域、不同来源和不同风格的文本中获取数据。
- 2) **引入噪声**：在生成文本时，引入一些随机性或噪声，例如通过采样不同的词或短语，或者引入随机的变换操作，以增加生成文本的多样性。这可以通过在生成过程中对模型的输出进行采样或添加随机性来实现。
- 3) **温度参数调整**：温度参数是用来控制生成文本的多样性的一个参数。通过调整温度参数的值，可以控制生成文本的独创性和多样性。较高的温度值会增加随机性，从而减少复读机问题的出现。
- 4) **Beam搜索调整**：在生成文本时，可以调整Beam搜索算法的参数。Beam搜索是一种常用的生成策略，它在生成过程中维护了一个候选序列的集合。通过调整Beam大小和搜索宽度，可以控制生成文本的多

样性和创造性。

- 5) **后处理和过滤**：对生成的文本进行后处理和过滤，去除重复的句子或短语，以提高生成文本的质量和多样性。可以使用文本相似度计算方法或规则来检测和去除重复的文本。
- 6) **人工干预和控制**：对于关键任务或敏感场景，可以引入人工干预和控制机制，对生成的文本进行审查和筛选，确保生成结果的准确性和多样性。

需要注意的是，缓解LLMs复读机问题是一个复杂的任务，没有一种通用的解决方案。不同的方法可能适用于不同的场景和任务，需要根据具体情况进行选择和调整。此外，解决复读机问题还需要综合考虑数据、训练目标、模型架构和生成策略等多个因素，需要进一步的研究和实践来提高大型语言模型的生成文本多样性和创造性。

1.5 如何让大模型处理更长的文本？

要让大模型处理更长的文本，可以考虑以下几个方法：

- 1) **分块处理**：将长文本分割成较短的片段，然后逐个片段输入模型进行处理。这样可以避免长文本对模型内存和计算资源的压力。在处理分块文本时，可以使用重叠的方式，即将相邻片段的一部分重叠，以保持上下文的连贯性。
- 2) **层次建模**：通过引入层次结构，将长文本划分为更小的单元。例如，可以将文本分为段落、句子或子句等层次，然后逐层输入模型进行处理。这样可以减少每个单元的长度，提高模型处理长文本的能力。
- 3) **部分生成**：如果只需要模型生成文本的一部分，而不是整个文本，可以只输入部分文本作为上下文，然后让模型生成所需的部分。例如，输入前一部分文本，让模型生成后续的内容。
- 4) **注意力机制**：注意力机制可以帮助模型关注输入中的重要部分，可以用于处理长文本时的上下文建模。通过引入注意力机制，模型可以更好地捕捉长文本中的关键信息。
- 5) **模型结构优化**：通过优化模型结构和参数设置，可以提高模型处理长文本的能力。例如，可以增加模型的层数或参数量，以增加模型的表达能力。还可以使用更高效的模型架构，如Transformer等，以提高长文本的处理效率。

需要注意的是，处理长文本时还需考虑计算资源和时间的限制。较长的文本可能需要更多的内存和计算时间，因此在实际应用中需要根据具体情况进行权衡和调整。

2. 大语言模型架构

2.1 讲讲对Attention的理解？

Attention机制是一种在处理时序相关问题的时候常用的技术，主要用于处理序列数据。

核心思想是在处理序列数据时，网络应该更关注输入中的重要部分，而忽略不重要的部分，它通过学习不同部分的权重，将输入的序列中的重要部分显式地加权，从而使得模型可以更好地关注与输出有关的信息。

在序列建模任务中，比如机器翻译、文本摘要、语言理解等，输入序列的不同部分可能具有不同的重要性。传统的循环神经网络（RNN）或卷积神经网络（CNN）在处理整个序列时，难以捕捉到序列中不同位置的重要程度，可能导致信息传递不够高效，特别是在处理长序列时表现更明显。

Attention机制的关键是引入一种机制来动态地计算输入序列中各个位置的权重，从而在每个时间步上，对输入序列的不同部分进行加权求和，得到当前时间步的输出。这样就实现了模型对输入中不同部分的关注度的自适应调整。

2.2 Attention的计算步骤是什么？

具体的计算步骤如下：

- 1) **计算查询（Query）**：查询是当前时间步的输入，用于和序列中其他位置的信息进行比较。
- 2) **计算键（Key）和值（Value）**：键表示序列中其他位置的信息，值是对应位置的表示。键和值用来和查询进行比较。
- 3) **计算注意力权重**：通过将查询和键进行内积运算，然后应用softmax函数，得到注意力权重。这些权重表示了在当前时间步，模型应该关注序列中其他位置的重要程度。
- 4) **加权求和**：根据注意力权重将值进行加权求和，得到当前时间步的输出。

在Transformer中，Self-Attention 被称为"Scaled Dot-Product Attention"，其计算过程如下：

- 1) 对于输入序列中的每个位置，通过计算其与所有其他位置之间的相似度得分（通常通过点积计算）。
- 2) 对得分进行缩放处理，以防止梯度爆炸。
- 3) 将得分用softmax函数转换为注意力权重，以便计算每个位置的加权和。
- 4) 使用注意力权重对输入序列中的所有位置进行加权求和，得到每个位置的自注意输出。

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V$$

2.3 Attention机制和传统的Seq2Seq模型有什么区别？

Seq2Seq模型是一种基于编码器-解码器结构的模型，主要用于处理序列到序列的任务，例如机器翻译、语音识别等。

传统的Seq2Seq模型只使用编码器来捕捉输入序列的信息，而解码器只从编码器的最后状态中获取信息，并将其用于生成输出序列。

而Attention机制则允许解码器在生成每个输出时，根据输入序列的不同部分给予不同的注意力，从而使得

模型更好地关注到输入序列中的重要信息。

2.4 transformer中multi-head attention中每个head为什么要进行降维？

在Transformer的Multi-Head Attention中，对每个head进行降维是为了增加模型的表达能力和效率。

每个head是独立的注意力机制，它们可以学习不同类型的特征和关系。通过使用多个注意力头，Transformer可以并行地学习多种不同的特征表示，从而增强了模型的表示能力。

然而，在使用多个注意力头的同时，注意力机制的计算复杂度也会增加。原始的Scaled Dot-Product Attention的计算复杂度为 $O(d^2)$ ，其中 d 是输入向量的维度。如果使用 h 个注意力头，计算复杂度将增加到 $O(hd^2)$ 。这可能会导致Transformer在处理大规模输入时变得非常耗时。

为了缓解计算复杂度的问题，Transformer中在每个head上进行降维。在每个注意力头中，输入向量通过线性变换被映射到一个较低维度的空间。这个降维过程使用两个矩阵：一个是查询（Q）和键（K）的降维矩阵 W_q 和 W_k ，另一个是值（V）的降维矩阵 W_v 。

通过降低每个head的维度，Transformer可以在保持较高的表达能力的同时，大大减少计算复杂度。降维后的计算复杂度为 $O(h\hat{d}^2)$ ，其中 $\hat{d}$ 是降维后的维度。通常情况下， $\hat{d}$ 会远小于原始维度 d ，这样就可以显著提高模型的计算效率。

2.5 Encoder编码器与Decoder掩码有什么区别？

Encoder主要使用自注意力掩码和填充掩码，而Decoder除了自注意力掩码外，还需要使用编码器-解码器注意力掩码来避免未来位置信息的泄露。这些掩码操作保证了Transformer在处理自然语言序列时能够准确、有效地进行计算，从而获得更好的表现。

2.6 为什么BERT选择mask掉15%这个比例的词，可以是其他的比例吗？

BERT选择mask掉15%的词是一种经验性的选择，是原论文中的一种选择，并没有一个固定的理论依据，实际中当然可以尝试不同的比例，15%的比例是由BERT的作者们在原始论文中提出，并在实验中发现对于BERT的训练效果是有效的。

2.7 BERT非线性的来源在哪里？

主要来自两个地方：前馈层的gelu激活函数和self-attention。

- **前馈神经网络层**：在BERT的Encoder中，每个自注意力层之后都跟着一个前馈神经网络层。前馈神经网络层是全连接的神经网络，通常包括一个线性变换和一个非线性的激活函数，如gelu。这样的非线性激活函数引入了非线性变换，使得模型能够学习更加复杂的特征表示。
- **self-attention layer**：在自注意力层中，查询（Query）、键（Key）、值（Value）之间的点积得分会经过softmax操作，形成注意力权重，然后将这些权重与值向量相乘得到每个位置的自注意输出。这个过程中涉及了softmax操作，使得模型的计算是非线性的。

2.8 为什么要进行LN呢？

使用Layer Normalization（LN）的主要原因是为了改善训练过程的稳定性并加速收敛。下面是一些具体的原因和解释：

1. **消除梯度消失和爆炸**：在深度神经网络中，梯度消失和爆炸问题是常见的，尤其是在使用递归神经网络（RNNs）和长短期记忆网络（LSTMs）时。通过在每一层对激活值进行归一化，LN可以帮助控制梯度的大小，从而减轻这些问题。
2. **加速收敛**：LN通过减少内部协变量移位（internal covariate shift）来帮助模型更快地收敛。内部协变量移位指的是网络中层的输入分布变化，这会增加优化的难度。通过保持每一层输入的稳定分布，LN有助于加速训练过程。
3. **适应不同批量大小**：相比于Batch Normalization（BN），LN的一个显著优点是它不依赖于批量大小。BN通常需要较大的批量来获得准确的均值和方差估计，这对于NLP任务可能不是最优的，因为NLP模型往往需要处理变长的输入序列，并且较大的批量可能会占用过多的内存。LN直接对每一层的特征进行归一化，因此适用于小批量或甚至单个样本。
4. **序列内部一致性**：在NLP中，我们通常希望一个句子内部的表示（即隐藏状态）具有一定的内部一致性。LN通过在特征维度上进行操作，确保了同一层中每个样本的特征向量具有相似的尺度和分布，这对于维持序列内的一致性很重要。
5. **Transformer架构的需求**：Transformer模型使用自注意力机制，这种机制对输入的尺度敏感。LN在Transformer中尤其重要，因为它有助于稳定自注意力层的输入，从而提高整个模型的性能。
6. **模型泛化能力**：通过规范化每一层的输出，LN有助于模型在不同的输入上获得更加一致的表现，从而可能提升模型的泛化能力。

3. 训练数据集

3.1 SFT（有监督微调）的数据集格式？

对于大语言模型的训练中，SFT（Supervised Fine-Tuning）的数据集格式可以采用以下方式：

- 1) **输入数据**：输入数据是一个文本序列，通常是一个句子或者一个段落。每个样本可以是一个字符串或者是一个tokenized的文本序列。
- 2) **标签数据**：标签数据是与输入数据对应的标签或类别。标签可以是单个类别，也可以是多个类别的集合。对于多分类任务，通常使用one-hot编码或整数编码来表示标签。
- 3) **数据集划分**：数据集通常需要划分为训练集、验证集和测试集。训练集用于模型的训练，验证集用于调整模型的超参数和监控模型的性能，测试集用于评估模型的最终性能。
- 4) **数据集格式**：数据集可以以文本文件（如CSV、JSON等）或数据库的形式存储。每个样本包含输入数据和对应的标签。可以使用表格形式存储数据，每一列代表一个特征或标签。

下面是一个示例数据集的格式：

```
Input,Label
"This is a sentence.",1
"Another sentence.",0
...
```

在这个示例中，输入数据是一个句子，标签是一个二分类的标签（1代表正例，0代表负例）。每一行代表一个样本，第一列是输入数据，第二列是对应的标签。

需要注意的是，具体的数据集格式可能会因任务类型、数据来源和使用的深度学习框架而有所不同。因此，在进行SFT训练时，建议根据具体任务和框架的要求来定义和处理数据集格式。

3.2 RM（奖励模型）的数据格式？

在大语言模型训练中，RM（Reward Model，奖励模型）的数据格式可以采用以下方式：

- 1) **输入数据**：输入数据是一个文本序列，通常是一个句子或者一个段落。每个样本可以是一个字符串或者是一个tokenized的文本序列。
- 2) **奖励数据**：奖励数据是与输入数据对应的奖励或评分。奖励可以是一个实数值，表示对输入数据的评价。也可以是一个离散的标签，表示对输入数据的分类。奖励数据可以是人工标注的，也可以是通过其他方式（如人工评估、强化学习等）得到的。
- 3) **数据集格式**：数据集可以以文本文件（如CSV、JSON等）或数据库的形式存储。每个样本包含输入数据和对应的奖励数据。可以使用表格形式存储数据，每一列代表一个特征或标签。

下面是一个示例数据集的格式：

Input,Reward

"This is a sentence.",0.8

"Another sentence.",0.2

...

在这个示例中，输入数据是一个句子，**奖励数据是一个实数值**，表示对输入数据的评价。每一行代表一个样本，第一列是输入数据，第二列是对应的奖励数据。

需要注意的是，具体的数据集格式可能会因任务类型、数据来源和使用的深度学习框架而有所不同。因此，在使用RM进行大语言模型训练时，建议根据具体任务和框架的要求来定义和处理数据集格式。

3.3 PPO（强化学习）的数据格式？

在大语言模型训练中，PPO（Proximal Policy Optimization，近端策略优化）是一种常用的强化学习算法。PPO的数据格式可以采用以下方式：

- 1) **输入数据**：输入数据是一个文本序列，通常是一个句子或者一个段落。每个样本可以是一个字符串或者是一个tokenized的文本序列。
- 2) **奖励数据**：奖励数据是与输入数据对应的奖励或评分。奖励可以是一个实数值，表示对输入数据的评价。也可以是一个离散的标签，表示对输入数据的分类。奖励数据可以是人工标注的，也可以是通过其他方式（如人工评估、模型评估等）得到的。
- 3) **动作数据**：动作数据是模型在给定输入数据下的输出动作。对于语言模型，动作通常是生成的文本序列。动作数据可以是一个字符串或者是一个tokenized的文本序列。
- 4) **状态数据**：状态数据是模型在给定输入数据和动作数据下的状态信息。对于语言模型，状态数据可以是模型的隐藏状态或其他中间表示。状态数据的具体形式可以根据具体任务和模型结构进行定义。
- 5) **数据集格式**：数据集可以以文本文件（如CSV、JSON等）或数据库的形式存储。每个样本包含输入数据、奖励数据、动作数据和状态数据。可以使用表格形式存储数据，每一列代表一个特征或标签。

下面是一个示例数据集的格式：

Input,Reward,Action,State

"This is a sentence.",0.8,"This is a generated sentence.",[0.1, 0.2, 0.3, ...]

"Another sentence.",0.2,"Another generated sentence.",[0.4, 0.5, 0.6, ...]

...

在这个示例中，输入数据是一个句子，奖励数据是一个实数值，动作数据是生成的句子，状态数据是模型的隐藏状态。每一行代表一个样本，第一列是输入数据，第二列是对应的奖励数据，第三列是生成的动作数据，第四列是状态数据。

需要注意的是，具体的数据集格式可能会因任务类型、数据来源和使用的深度学习框架而有所不同。因此，在使用PPO进行大语言模型训练时，建议根据具体任务和框架的要求来定义和处理数据集格式。

4. 有监督微调

4.1 微调方法是啥？如何微调？

微调 (Fine-tuning) 是一种迁移学习的方法，用于在一个预训练模型的基础上，通过在特定任务的数据上进行有监督训练，来适应该任务的要求并提高模型性能。微调利用了预训练模型在大规模通用数据上学习到的语言知识和表示能力，将其迁移到特定任务上。

下面是一般的微调步骤：

- 1) **预训练模型选择**：选择一个在大规模数据上进行预训练的模型作为基础模型。例如，可以选择一种预训练的语言模型，如BERT、GPT等。
- 2) **数据准备**：准备用于微调的特定任务数据集。这些数据集应包含任务相关的样本和相应的标签或目标。确保数据集与任务的特定领域或问题相关。
- 3) **构建任务特定的模型头**：根据任务的要求，构建一个特定的模型头 (task-specific head)。模型头是添加到预训练模型之上的额外层或结构，用于根据任务要求进行输出预测或分类。例如，对于文本分类任务，可以添加一个全连接层和softmax激活函数。
- 4) **参数初始化**：将预训练模型的参数作为初始参数加载到微调模型中。这些参数可以被视为模型已经学习到的通用语言表示。
- 5) **微调训练**：使用特定任务的数据集对模型进行有监督训练。这包括将任务数据输入到模型中，计算损失函数，并通过反向传播和优化算法（如梯度下降）更新模型参数。在微调过程中，只有模型头的参数会被更新，而预训练模型的参数会保持不变。
- 6) **调整超参数**：微调过程中，可以根据需要调整学习率、批量大小、训练迭代次数等超参数，以达到更好的性能。
- 7) **评估和验证**：在微调完成后，使用验证集或测试集对微调模型进行评估，以评估其在特定任务上的性能。可以使用各种指标，如准确率、精确率、召回率等。
- 8) **可选的后续微调**：根据实际情况，可以选择在特定任务的数据上进行进一步的微调迭代，以进一步提高模型性能。

微调的关键是在预训练模型的基础上进行训练，从而将模型的知识迁移到特定任务上。通过这种方式，可以在较少的数据和计算资源下，快速构建和训练高性能的模型。

4.2 微调和参数高效微调之间的区别是什么？

微调和参数高效微调是机器学习中用于提高预训练模型在特定任务上的性能的两种方法。

微调就是把一个预先训练好的模型用新的数据在一个新的任务上进一步训练它。整个预训练模型通常在微调中进行训练，包括它的所有层和参数。这个过程在计算上非常昂贵且耗时，特别是对于大型模型。

另一方面，**参数高效微调**是一种专注于只训练预训练模型参数的子集的微调方法。这种方法包括为新任务识别最重要的参数，并且只在训练期间更新这些参数。这样，PEFT可以显著减少微调所需的计算量。

4.3 PEFT 有什么优点？

在这里，只讨论PEFT相对于传统微调的好处。因此，理解为什么参数有效的微调比微调更有益。

- 1) **减少计算和存储成本**：PEFT只涉及微调少量额外的模型参数，而冻结预训练llm的大部分参数，从而显著降低计算和存储成本
- 2) **克服灾难性遗忘**：在LLM的全面微调期间，灾难性遗忘可能发生在模型忘记它在预训练期间学到的知识的地方。PEFT通过只更新几个参数来克服这个问题。
- 3) **低数据环境下更好的性能**：PEFT方法在低数据环境下的表现优于完全微调，并且可以更好地推广到域外场景。
- 4) **可移植性**：与全面微调的大检查点相比，PEFT方法使用户能够获得价值几mb的小检查点。这使得来自PEFT方法的训练权重易于部署和用于多个任务，而无需替换整个模型。
- 5) **与完全微调相当的性能**：PEFT仅使用少量可训练参数即可实现与完全微调相当的性能。

4.4 多种不同的高效微调方法对比

参数有效策略可能涉及多种技术：

- 1) **选择性层调整 (Selective Layer Tuning)**：可以只微调层的一个子集，而不是微调模型的所有层。这减少了需要更新的参数数量。
- 2) **适配器 (Adapters)**：适配器层是插入预训练模型层之间的小型神经网络。在微调过程中，只训练这些适配器层，保持预先训练的参数冻结。通过这种方式，适配器学习将预先训练的模型提取的特征适应新任务。
- 3) **稀疏微调 (Sparse Fine-Tuning)**：传统的微调会略微调整所有参数，但稀疏微调只涉及更改模型参数的一个子集。这通常是基于一些标准来完成的，这些标准标识了与新任务最相关的参数。
- 4) **低秩近似 (Low-Rank Approximations)**：另一种策略是用一个参数较少但在任务中表现相似的模型来近似微调后的模型。
- 5) **正则化技术 (Regularization Techniques)**：可以将正则化项添加到损失函数中，以阻止参数发生较大变化，从而以更“参数高效”的方式有效地微调模型。
- 6) **任务特定的头 (Task-specific Heads)**：有时，在预先训练的模型架构中添加一个任务特定的层或“头”，只对这个头进行微调，从而减少需要学习的参数数量。

4.5 当前高效微调技术存在的一些问题

当前的高效微调技术很难在类似方法之间进行直接比较并评估它们的真实性能，主要的原因如下所示：

- 1) **参数计算口径不一致**：参数计算可以分为三类：可训练参数的数量、微调模型与原始模型相比改变的参数的数量、微调模型和原始模型之间差异的等级。例如，DiffPruning更新0.5%的参数，但是实际参与训练的参数量是200%。这为比较带来了困难。尽管可训练的参数量是最可靠的存储高效指标，但是也不完美。Ladder-side Tuning使用一个单独的小网络，参数量高于LoRA或BitFit，但是因为反向传播不经过主网络，其消耗的内存反而更小。
- 2) **缺乏模型大小的考虑**：已有工作表明，大模型在微调中需要更新的参数量更小（无论是以百分比相对而言还是以绝对数量而论），因此（基）模型大小在比较不同PEFT方法时也要考虑到。
- 3) **缺乏测量基准和评价标准**：不同方法所使用的模型/数据集组合都不一样，评价指标也不一样，难以得到有意义的结论。
- 4) **代码实现可读性差**：很多开源代码都是简单拷贝Transformer代码库，然后进行小修小补。这些拷贝也不使用git fork，难以找出改了哪里。即便是能找到，可复用性也比较差（通常指定某个Transformer版本，没有说明如何脱离已有代码库复用这些方法）。

5. 推理

5.1 为什么大模型推理时显存涨的那么多还一直占着？

大语言模型进行推理时，显存涨得很多且一直占着显存不释放的原因主要有以下几点：

模型参数占用显存：大语言模型通常具有巨大的参数量，这些参数需要存储在显存中以供推理使用。因此，在推理过程中，模型参数会占用相当大的显存空间。

输入数据占用显存：进行推理时，需要将输入数据加载到显存中。对于大语言模型而言，输入数据通常也会占用较大的显存空间，尤其是对于较长的文本输入。

中间计算结果占用显存：在推理过程中，模型会进行一系列的计算操作，生成中间结果。这些中间结果也需要存储在显存中，以便后续计算使用。对于大语言模型而言，中间计算结果可能会占用较多的显存空间。

内存管理策略：某些深度学习框架在推理时采用了一种延迟释放显存的策略，即显存不会立即释放，而是保留一段时间以备后续使用。这种策略可以减少显存的分配和释放频率，提高推理效率，但也会导致显存一直占用的现象。

需要注意的是，显存的占用情况可能会受到硬件设备、深度学习框架和模型实现的影响。不同的环境和设置可能会导致显存占用的差异。如果显存占用过多导致资源不足或性能下降，可以考虑调整模型的批量大小、优化显存分配策略或使用更高性能的硬件设备来解决问题。

5.2 大模型在GPU和CPU上推理速度如何

大语言模型在GPU和CPU上进行推理的速度存在显著差异。一般情况下，GPU在进行深度学习推理任务时具有更高的计算性能，因此大语言模型在GPU上的推理速度通常会比在CPU上更快。

以下是GPU和CPU在大语言模型推理速度方面的一些特点：

- 1) **GPU推理速度快**：GPU具有大量的并行计算单元，可以同时处理多个计算任务。对于大语言模型而言，GPU可以更高效地执行矩阵运算和神经网络计算，从而加速推理过程。
- 2) **CPU推理速度相对较慢**：相较于GPU，CPU的计算能力较弱，主要用于通用计算任务。虽然CPU也可以执行大语言模型的推理任务，但由于计算能力有限，推理速度通常会较慢。
- 3) **使用GPU加速推理**：为了充分利用GPU的计算能力，通常会使用深度学习框架提供的GPU加速功能，如CUDA或OpenCL。这些加速库可以将计算任务分配给GPU并利用其并行计算能力，从而加快大语言模型的推理速度。

需要注意的是，推理速度还受到模型大小、输入数据大小、计算操作的复杂度以及硬件设备的性能等因素的影响。因此，具体的推理速度会因具体情况而异。一般来说，使用GPU进行大语言模型的推理可以获得更快的速度。

5.3 推理速度上，INT8和FP16比起来怎么样？

在大语言模型的推理速度上，使用INT8（8位整数量化）和FP16（半精度浮点数）相对于FP32（单精度浮点数）可以带来一定的加速效果。这是因为INT8和FP16的数据类型在表示数据时所需的内存和计算资源较少，从而可以加快推理速度。

具体来说，INT8在相同的内存空间下可以存储更多的数据，从而可以在相同的计算资源下进行更多的并行计算。这可以提高每秒推理操作数（Operations Per Second, OPS）的数量，加速推理速度。

FP16在相对较小的数据范围内进行计算，因此在相同的计算资源下可以执行更多的计算操作。虽然FP16的精度相对较低，但对于某些应用场景，如图像处理和语音识别等，FP16的精度已经足够满足需求。

需要注意的是，INT8和FP16的加速效果可能会受到硬件设备的支持程度和具体实现的影响。某些硬件设备可能对INT8和FP16有更好的优化支持，从而进一步提高推理速度。

综上所述，使用INT8和FP16数据类型可以在大语言模型的推理过程中提高推理速度，但需要根据具体场景和硬件设备的支持情况进行评估和选择。

5.4 大模型有推理能力吗？

逻辑推理是大语言模型“智能涌现”出的核心能力之一，好像AI有了人的意识一样。而推理能力的关键，在于一个技术——思维链（Chain of Thought, CoT）。当模型规模足够大的时候，LLM本身是具备推理能力的。

在简单推理问题上，LLM已经达到了很好的能力；复杂推理问题上，还需要更多深入的研究。

5.5 大模型生成时的参数怎么设置？

- **Temperature**：用于调整随机从生成模型中抽样的程度，使得相同的提示可能会产生不同的输出。温度为 0 将始终产生相同的输出，该参数设置越高随机性越大。
- **波束搜索宽度**：波束搜索是许多 NLP 和语音识别模型中常用的一种算法，作为在给定可能选项的情况下选择最佳输出的最终决策步骤。波束搜索宽度是一个参数，用于确定算法在搜索的每个步骤中应该考虑的候选数量。
- **Top p**：动态设置tokens候选列表的大小^{**}。^{**} 将可能性之和不超过特定值的top tokens列入候选名单。Top p 通常设置为较高的值（如 0.75），目的是限制可能被采样的低概率 token 的长度。
- **Top k**：允许其他高分tokens有机会被选中^{**}。^{**} 这种采样引入的随机性有助于在很多情况下生成的质量。Top k 参数设置为 3 则意味着选择前三个tokens。

若 Top k 和 Top p 都启用，则 Top p 在 Top k 之后起作用

5.6 有哪些省内存的大语言模型训练/微调/推理方法？

有一些方法可以帮助省内存的大语言模型训练、微调和推理，以下是一些常见的方法：

- 1) **参数共享 (Parameter Sharing)**：通过共享模型中的参数，可以减少内存占用。例如，可以在不同的位置共享相同的嵌入层或注意力机制。
- 2) **梯度累积 (Gradient Accumulation)**：在训练过程中，将多个小批次的梯度累积起来，然后进行一次参数更新。这样可以减少每个小批次的内存需求，特别适用于GPU内存较小的情况。
- 3) **梯度裁剪 (Gradient Clipping)**：通过限制梯度的大小，可以避免梯度爆炸的问题，从而减少内存使用。
- 4) **分布式训练 (Distributed Training)**：将训练过程分布到多台机器或多个设备上，可以减少单个设备的内存占用。分布式训练还可以加速训练过程。
- 5) **量化 (Quantization)**：将模型参数从高精度表示（如FP32）转换为低精度表示（如INT8或FP16），可以减少内存占用。量化方法可以通过减少参数位数或使用整数表示来实现。
- 6) **剪枝 (Pruning)**：通过去除冗余或不重要的模型参数，可以减少模型的内存占用。剪枝方法可以根据参数的重要性进行选择，从而保持模型性能的同时减少内存需求。
- 7) **蒸馏 (Knowledge Distillation)**：使用较小的模型（教师模型）来指导训练较大的模型（学生模型），可以从教师模型中提取知识，减少内存占用。
- 8) **分块处理 (Chunking)**：将输入数据或模型分成较小的块进行处理，可以减少内存需求。例如，在推

理过程中，可以将较长的输入序列分成多个较短的子序列进行处理。

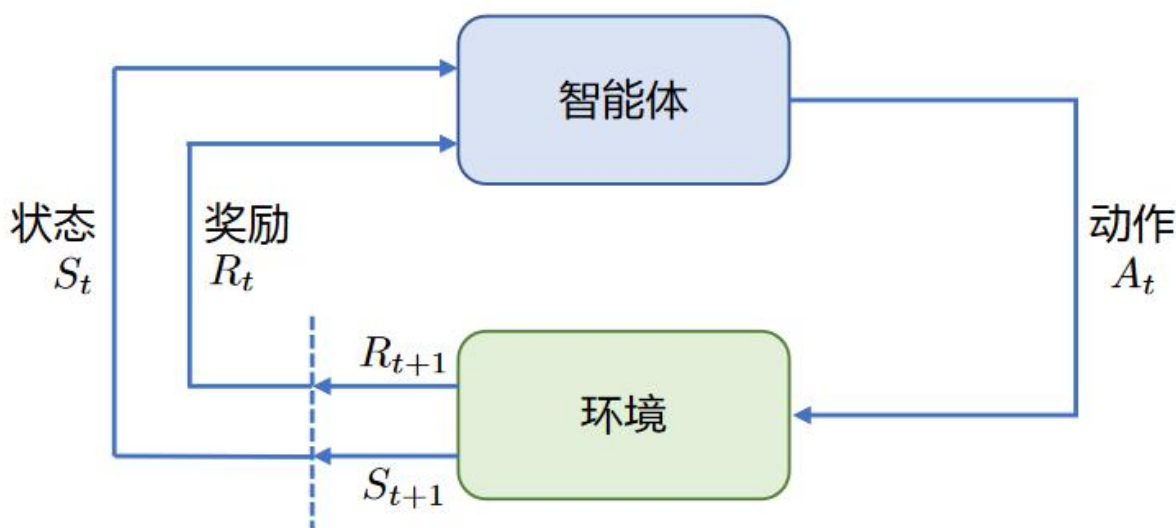
这些方法可以结合使用，根据具体场景和需求进行选择 and 调整。同时，不同的方法可能对不同的模型和任务有不同的效果，因此需要进行实验和评估。

6. 强化学习

6.1 简单介绍强化学习？

强化学习 (Reinforcement Learning, RL) 研究的问题是智能体 (Agent) 与环境 (Environment) 交互的问题，其目标是使智能体在复杂且不确定的环境中最大化奖励 (Reward)。

强化学习基本框架如图所示，主要由两部分组成：**智能体**和**环境**。在强化学习过程中，智能体与环境不断交互。智能体在环境中获取某个状态后，会根据该状态输出一个动作 (Action)，也称为决策 (Decision)。动作会在环境中执行，环境会根据智能体采取的动作，给出下一个状态以及当前动作所带来的奖励。智能体的目标就是尽可能多地从环境中获取奖励。本节中将介绍强化学习的基本概念、强化学习与有监督学习的区别，以及在大语言模型中基于人类反馈的强化学习流程。



强化学习在大语言模型上的重要作用可以概括为以下几个方面：

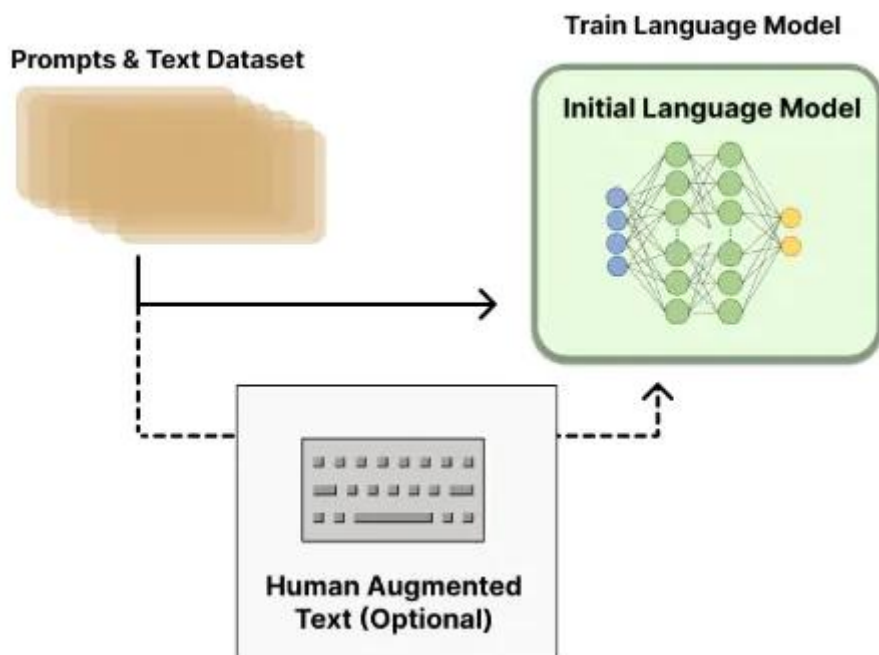
- 1) **强化学习比有监督学习更可以考虑整体影响**：有监督学习针对单个词元进行反馈，其目标是要求模型针对给定的输入给出的确切答案。而强化学习是针对整个输出文本进行反馈，并不针对特定的词元。
- 2) **强化学习更容易解决幻觉问题**：有监督学习算法非常容易使得求知型查询产生幻觉。在模型并不包含或者知道答案的情况下，有监督训练仍然会促使模型给出答案。而使用强化学习方法，则可以通过定制奖励函数，将正确答案赋予非常高的分数，放弃回答的答案赋予中低分数，不正确的答案赋予非常高的负分，使得模型学会依赖内部知识选择放弃回答，从而在一定程度上缓解模型幻觉问题。
- 3) **强化学习可以更好的解决多轮对话奖励累积问题**：使用强化学习方法，可以通过构建奖励函数，将当

前输出考虑整个对话的 背景和连贯性

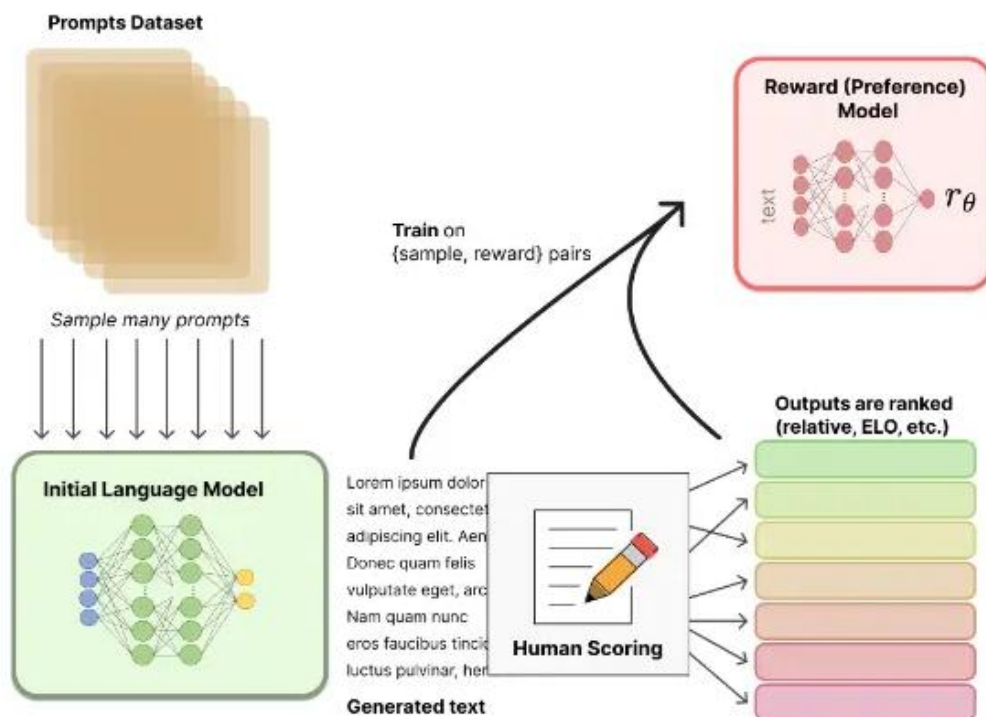
6.2 简单介绍一下 RLHF?

RLHF就是基于人类反馈（Human Feedback）对语言模型进行强化学习（Reinforcement Learning），一般分为以下三个步骤：

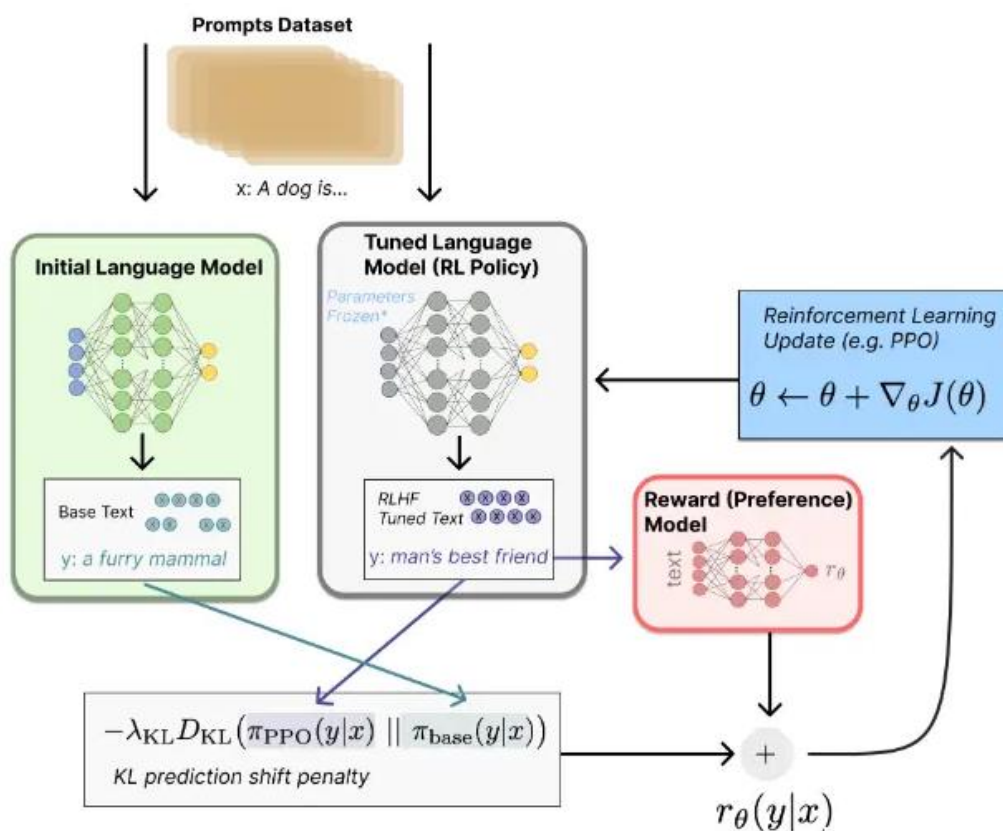
- 1) 预训练语言模型（收集样本数据，有监督微调）：在人类标注的数据上微调出来的模型叫做 有监督的微调（supervised fine-tuning），这是训练出来的第一个模型



- 2) 训练奖励模型（收集排序数据，训练奖励模型）：
 - a) 给定一个问题，让上一步训练好的预训练模型 SFT 生成答案
 - b) GPT 每一次预测一个词的概率，可以根据这个概率采样出很多答案，通常来说可以用 beam search
 - c) 这里生成了四个答案，然后把这四个答案的好坏进行人工标注，进行排序标注
 - d) 有了这些排序之后，再训练一个奖励模型（Reward Model, RM），这个模型是说给定 prompt 得到输出，然后对这个输出生成一个分数，可以认为这个分数是一个奖励或者是打分，使得对答案的分数能够满足人工排序的关系（大小关系保持一致），一旦这个模型生成好之后，就能够对生成的答案进行打分



- 3) 用强化学习微调（使用RM模型优化SFT模型）：继续微调之前训练好的 SFT模型，使得它生成的答案能够尽量得到一个比较高的分数，即每一次将它生成的答案放进 RM 中打分，然后优化 SFT 的参数使得它生成的答案在 RM 中获得更高的分数。



备注：两次对模型的微调：GPT3模型 → SFT模型 → RL模型，其实这里始终都是同一个模型，只是不同过程中名称不同。

- **需要SFT模型的原因：** GPT3模型不一定能够保证根据人的指示、有帮助的、安全的生成答案需要人工标注数据进行微调。
- **需要RM模型的原因：** 标注排序的判别式标注成本远远低于生成答案的生成式标注。
- **需要RL模型的原因：** 在对SFT模型进行微调时生成的答案分布也会发生变化，会导致RM模型的评分会有偏差，需要用到强化学习。

6.3 奖励模型需要和基础模型一致吗？

奖励模型和基础模型在训练过程中可以是一致的，也可以是不同的。这取决于你的任务需求和优化目标。

如果你希望优化一个包含多个子任务的复杂任务，那么你可能需要为每个子任务定义一个奖励模型，然后将这些奖励模型整合到一个统一的奖励函数中。这样，你可以根据任务的具体情况调整每个子任务的权重，以实现更好的性能。

另一方面，如果你的任务是单任务的，那么你可能只需要一个基础模型和一个对应的奖励模型，这两个模型可以共享相同的参数。在这种情况下，你可以通过调整奖励模型的权重来控制任务的优化方向。

总之，**奖励模型和基础模型的一致性取决于你的任务需求和优化目标**。在实践中，你可能需要尝试不同的模型结构和奖励函数，以找到最适合你任务的解决方案。

6.4 RLHF 在实践中存在哪些不足？

RLHF (Reinforcement Learning from Human Feedback) 是一种通过人类反馈进行增强学习的方法，尽管具有一定的优势，但在实践过程中仍然存在以下几个不足之处：

- 1) **人类反馈的代价高昂：** 获取高质量的人类反馈通常需要大量的人力和时间成本。人类专家需要花费时间来评估模型的行为并提供准确的反馈，这可能限制了RLHF方法的可扩展性和应用范围。
- 2) **人类反馈的主观性：** 人类反馈往往是主观的，不同的专家可能会有不同的意见和判断。这可能导致模型在不同专家之间的反馈上存在差异，从而影响模型的训练和性能。
- 3) **反馈延迟和稀疏性：** 获取人类反馈可能存在延迟和稀疏性的问题。人类专家不可能实时监控和评估模型的每一个动作，因此模型可能需要等待一段时间才能收到反馈，这可能会导致训练的效率 and 效果下降。
- 4) **错误反馈的影响：** 人类反馈可能存在错误或误导性的情况，这可能会对模型的训练产生负面影响。如果模型在错误的反馈指导下进行训练，可能会导致模型产生错误的行为策略。
- 5) **缺乏探索与利用的平衡：** 在RLHF中，人类反馈通常用于指导模型的行为，但可能会导致模型过于依赖人类反馈而缺乏探索的能力。这可能限制了模型发现新策略和优化性能的能力。

针对这些不足，研究人员正在探索改进RLHF方法，如设计更高效的人类反馈收集机制、开发更准确的反馈

评估方法、结合自适应探索策略等，以提高RLHF方法的实用性和性能。

6.5 什么是 LLM Agent?

LLM Agent 是一种人工智能系统，它利用大型语言模型（LLM）作为其核心计算引擎，展示文本生成之外的功能，包括进行对话、完成任务、推理，并可以展示一定程度的自主行为。

LLM Agent 根据设计阶段授予的功能，Agent 从纯粹的被动到高度主动的自主行为。同时利用大模型的推理能力，让 Agent 可以在人工监督下管理相对独立的工作流程：分析目标，项目规划，执行，回顾过去的工作，迭代细化。

6.6 LLM Agent 有什么关键能力?

- 1) Agent利用LLM的语言能力理解指令、上下文和目标。可以根据人类提示自主和半自主操作。
- 2) 可以利用工具套件（计算器、API、搜索引擎）来收集信息并采取行动来完成分配的任务。它们不仅仅局限于语言处理。
- 3) 可以做逻辑推理类型的任务。例如，chain-of-thought , tree-of-thought。
- 4) 可以量身定制文本，例如邮件，报告，市场材料。
- 5) 可以自动或半自动的响应用户的需求。
- 6) Agent可以和不同类型的AI系统对接，例如LLM+image generators。

6.7 怎样构建基于 LLM 的 Agents?

Agent = LLM + Prompt Recipe + Tools + Interface + Knowledge + Memory

- 1) Prompt Recipe: 特定的内容要求、目标受众、所需的语气、输出长度、创造力水平等。
- 2) Tools: 工具集成允许通过API和外部服务完成任务。Agents 能够理解自然语言、推理提示、积累记忆并采取明智的行动。但是，Agents 的表现和一致性取决于他们收到的提示的质量。
- 3) Knowledge: 知识适用于所有用户的一般专业知识。知识扩展了LLM的内容。一般分为专业知识、常识知识和程序知识。
- 4) Memory: 单个用户或单个任务的上下文和记录细节。分为短期记忆和长期记忆。记忆服务与特定用户，在时间维度的体验。使特定用户的上下文对话个性化同时保持多步骤任务的一致性。记忆侧重暂时的用户和任务细节。

7. 大语言模型评估

7.1 大模型怎么评测？

自动评测和人工评测。这两种方法在评测语言模型和机器翻译等任务时起着重要的作用。

自动评测方法基于计算机算法和自动生成的指标，能够快速且高效地评测模型的性能。

而人工评测则侧重于人类专家的主观判断和质量评测，能够提供更深入、细致的分析和意见。了解和掌握这两种评测方法对准确评测和改进语言模型的能力十分重要。

7.2 大模型的 honest 原则是如何实现的？模型如何判断回答的知识是训练过的已知的知识，怎么训练这种能力？

大模型需要遵循的helpful, honest, harmless的原则。

可以有意构造如下的训练样本，以提升模型遵守 honest 原则，可以算 trick 了：微调时构造知识问答类训练集，给出不知道的不回答，加强 honest 原则；阅读理解题，读过的要回答，没读过的不回答，不要胡说八道。

7.3 如何衡量大模型水平？

在评测 LLMs 的性能时，选择合适的任务和领域对于展示大型语言模型的表现、优势和劣势至关重要。为了更清晰地展示 LLMs 的能力水平，文章将现有的任务划分为以下7个不同的类别：

- 1) 自然语言处理：包括自然语言理解、推理、自然语言生成和多语言任务
- 2) 鲁棒性、伦理、偏见和真实性
- 3) 医学应用：包括医学问答、医学考试、医学教育和医学助手
- 4) 社会科学
- 5) 自然科学与工程：包括数学、通用科学和工程
- 6) 代理应用：将 LLMs 作为代理使用
- 7) 其他应用

7.4 大模型评估方法有哪些？

- 1) 首先是“直接评估指标”这一类别。这些是在人工智能领域长期以来广泛使用的传统指标。像准确率（accuracy）和F1得分（F1 score）等指标属于这个类别。通常情况下，这种方法涉及从模型中获取单一的输出，并将其与参考值进行比较，可以通过约束条件或提取所需信息的方式来实现评估
- 2) 接下来是第二类方法，称为“间接或分解的启发式方法（indirect or decomposed heuristics）”。在这种方法中，我们利用较小的模型（smaller models）来评估主模型（the main model）生成的答案，这些较小的模型可以是微调过的模型或原始的分解模型（raw decompositions）。
- 3) 第三类评估方法被称为“基于模型的评估”。在这种方法中，模型本身提供最终的评估分数或评估结果。然而，这也引入了额外的可变因素。即使模型可以获取到ground truth信息，评估指标本身也可能在评分过程中产生随机因素或不确定因素

7.5 什么是大模型幻觉？

在语言模型的背景下，幻觉指的是一本正经的胡说八道：看似流畅自然的表述，实则不符合事实或者是错误的。

幻觉现象的存在严重影响LLM应用的可靠性，本文将探讨大型语言模型(LLMs)的幻觉问题，以及解决幻觉现象的一些常见方法。

7.6 为什么需要解决LLM的幻觉问题？

LLMs的幻觉可能会产生如传播错误信息或侵犯隐私等严重后果。比如在医疗应用中，对患者生成的报告如果存在幻觉可能导致错误诊断甚至影响生命安全。

幻觉影响了模型的可靠性和可信度，因此需要解决LLM的幻觉问题。

7.7 幻觉一定是有害的吗？

幻觉不一定是有害的，特别是在一些需要创造力或灵感的场合，比如写电影剧情，幻觉的存在可能带来一些奇思妙想，使得生成的文本充满想象力。

因此，对幻觉的容忍度取决于具体的应用场景。

7.8 幻觉有哪些不同类型？

幻觉主要可以分为两类：即**内在幻觉**和**外在幻觉**。

- **内在幻觉**：生成的内容与源内容相矛盾。
- **外部幻觉**：生成的内容不能从源内容中得到验证，既不受源内容支持也不受其反驳。

7.9 为什么LLM会产生幻觉？

有一些研究也在致力于分析幻觉出现的不同原因，已知的一些原因包括：

- 1) **源与目标的差异**：当我们在存在源与目标差异的数据上训练模型时，模型产生的文本可能与原始源内容产生偏差。这种差异，有时可能是在数据收集过程中不经意间产生的，有时则是故意为之。
- 2) **无意识的源-目标差异**：这种差异的产生有多种原因。例如，数据可能是基于某种经验法则编制的，使得目标信息并不总是完全依赖源信息。举例来说，如果从两家不同的新闻网站获得相同事件的报道作为源与目标，目标报道中可能包含源报道没有的信息，从而导致二者不同。
- 3) **有意识的源-目标差异**：某些任务在本质上并不追求源与目标的严格一致，尤其是在需要多样性输出的情境下。
- 4) **训练数据的重复性**：训练过程中使用的数据，如果存在大量重复，可能导致模型在生成时过于偏好某些高频短语，这也可能引发“幻觉”。
- 5) **数据噪声的影响**：使用充斥噪声的数据进行训练，往往是导致“幻觉”出现的关键因素之一。
- 6) **解码过程中的随机性**：某些旨在增加输出多样性的解码策略，如top-k采样、top-p方法以及温度调节，有时会增加“幻觉”的产生。这往往是因为模型在选择输出词汇时引入了随机性，而没有始终选择最可能的词汇。
- 7) **模型的知识偏向**：有研究表明，模型在处理信息时，可能更依赖其在预训练阶段所积累的知识，而忽略了实时提供的上下文信息，从而偏离了正确的输出路径。
- 8) **训练与实际应用中的解码差异**：在常见的训练方法中，我们鼓励模型基于真实数据预测下一个词汇。但在实际应用中，模型则是根据自己先前生成的内容进行预测。这种方法上的差异，尤其在处理长文本时，可能会导致模型的输出出现“幻觉”。

最后，如GPT之类的生成模型，其实只是学会了文本中词汇间的统计规律，所以它们生成内容的准确性仍然是有限的。

7.10 如何度量幻觉？

最有效可靠的方式当然是靠人来评估，但是人工评估的成本太高了。因此有了一些自动化评估的指标：

- **命名实体误差**：命名实体（NEs）是“事实”描述的关键组成部分，我们可以利用NE匹配来计算生成文本与参考资料之间的一致性。直观上，如果一个模型生成了不在原始知识源中的NE，那么它可以被视为产生了幻觉（或者说，有事实上的错误）。
- **蕴含率**：该指标定义为被参考文本所蕴含的句子数量与生成输出中的总句子数量的比例。为了实现这一点，可以采用成熟的蕴含/NLI模型。
- **基于模型的评估**：应对复杂的句法和语义变化。
- **利用问答系统**：此方法的思路是，如果生成的文本在事实上与参考材料一致，那么对同一个问题，其答案应该与参考材料相似。具体而言，对于给定的生成文本，问题生成模型会创建一组问题-答案对。接下来，问答模型将使用原始的参考文本来回答这些问题，并计算所得答案的相似性。
- **利用信息提取系统**：此方法使用信息提取模型将知识简化为关系元组，例如<主体，关系，对象>。这些模型从生成的文本中提取此类元组，并与从原始材料中提取的元组进行比较。

8. 大语言模型应用

8.1 什么是 LangChain?

LangChain 是一个基于语言模型的框架，用于构建聊天机器人、生成式问答（GQA）、摘要等功能。它的核心思想是将不同的组件“链”在一起，以创建更高级的语言模型应用。

LangChain 框架核心目标是**为了连接多种大语言模型**（如 OpenAI、LLaMA 等）和外部资源（如 Google、Wikipedia、Notion 以及 Wolfram 等），**提供抽象和工具以在文本输入和输出之间进行接口处理**。大语言模型和组件通过“链（Chain）”连接，使得开发人员可以快速开发原型系统和 应用程序。

LangChain 的主要价值在于以下几个方面：

- 1) **组件化**：LangChain 框架提供了用于处理语言模型的抽象组件，以及每个抽象组件的一系列 实现。这些组件具有模块化设计，易于使用，无论是否使用 LangChain 框架的其他部分，都 可以方便地使用这些组件。
- 2) **现成的链式组装**：LangChain 框架提供了一些现成的链式组装，用于完成特定的高级任务。这 些现成的链式组装使得入门变得更加容易。对于更复杂的应用程序，LangChain 框架也支持 自定义现有链式组装或构建新的链式组装。
- 3) **简化开发难度**：通过提供组件化和现成的链式组装，LangChain 框架可以大大简化大语言模 型应用的开发难度。开发人员可以更专注于业务逻辑，而无需花费大量时间和精力处理底层 技术细节。

8.2 LangChain 包含哪些核心模块

LangChain 的提供了以下 6 种标准化、可扩展的接口并且可以外部集成的核心模块：

- 模型输入/输出 (Model I/O) : 与语言模型交互的接口;
- 数据连接 (Data connection) : 与特定应用程序的数据进行交互的接口;
- 链 (Chains) : 用于复杂的应用的调用序列;
- 智能体 (Agents) : 语言模型作为推理器决定要执行的动作序列;
- 记忆 (Memory) : 用于链的多次运行之间持久化应用程序状态;
- 回调 (Callbacks) : 记录和流式传输任何链式组装的中间步骤。

8.3 LangChain 如何调用 LLMs 生成回复?

要调用LLMs生成回复,可以使用LangChain框架提供的LLMChain类。LLMChain类是LangChain的一个组件,用于与语言模型进行交互并生成回复。以下是一个示例代码片段,展示了如何使用LLMChain类调用LLMs生成回复:

```
from langchain.llms import OpenAI
from langchain.chains import LLMChain

llm = OpenAI(temperature=0.9) # 创建 LLM 实例
prompt = "用户的问题" # 设置用户的问题

# 创建 LLMChain 实例
chain = LLMChain(llm=llm, prompt=prompt)

# 调用 LLMs 生成回复
response = chain.generate()

print(response) # 打印生成的回复
```

在上面的代码中,首先创建了一个LLM实例,然后设置了用户的问题作为LLMChain的prompt。接下来,调用LLMChain的generate方法来生成回复。最后,打印生成的回复。

请注意,可以根据需要自定义LLM的参数,例如温度 (temperature)、最大令牌数 (max_tokens) 等。LangChain文档中有关于LLMChain类和LLM参数的更多详细信息。

8.4 LangChain 如何修改 提示模板?

要修改LangChain的提示模板,可以使用LangChain框架提供的ChatPromptTemplate类。

ChatPromptTemplate类允许您创建自定义的聊天消息提示,并根据需要进行修改。以下是一个示例代码片

段，展示了如何使用ChatPromptTemplate类修改提示模板：

```
from langchain.llms import OpenAI
from langchain.chains import LLMChain

llm = OpenAI(temperature=0.9) # 创建 LLM 实例
prompt = "用户的问题" # 设置用户的问题

# 创建 LLMChain 实例
chain = LLMChain(llm=llm, prompt=prompt)

# 调用 LLMs 生成回复
response = chain.generate()

print(response) # 打印生成的回复
```

在上面的代码中，首先创建了一个空的ChatPromptTemplate实例。然后，使用add_message方法添加了聊天消息提示。接下来，我们使用set_message_content方法修改了第一个和最后一个聊天消息的内容。最后，我们使用format_messages方法格式化聊天消息，并打印出来。

请注意，可以根据需要添加、删除和修改聊天消息提示。ChatPromptTemplate类提供了多种方法来操作提示模板。更多详细信息和示例代码可以在LangChain文档中找到。

八、数据结构与算法

1. 背景知识

1.1 python数据结构

解答算法题目可以用到的python数据类型：列表、元组、字典 python列表可以作为数据，也可以作为堆栈，还可以作为队列使用

```
# 列表
# 数组
l = [1,2,3,4]
print(l[2])
l.append(5)
print(l[-1])
```

堆栈 FILO first in last out

```
stack = []
stack.append(1)
stack.append(2)
# 删除栈顶元素
e1 = stack.pop()
# 查看栈顶元素
print(stack[-1])
```

队列 FIFO first in first out

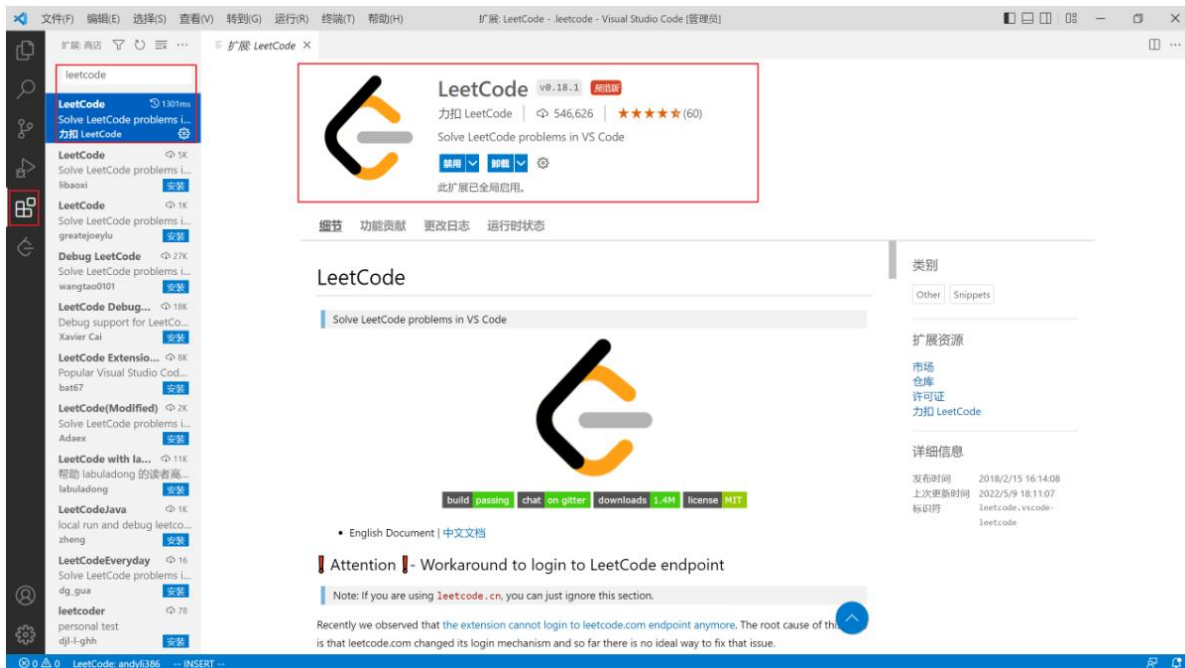
```
q = []
q.append(1)
q.append(2)
q.pop(0)
```

字典

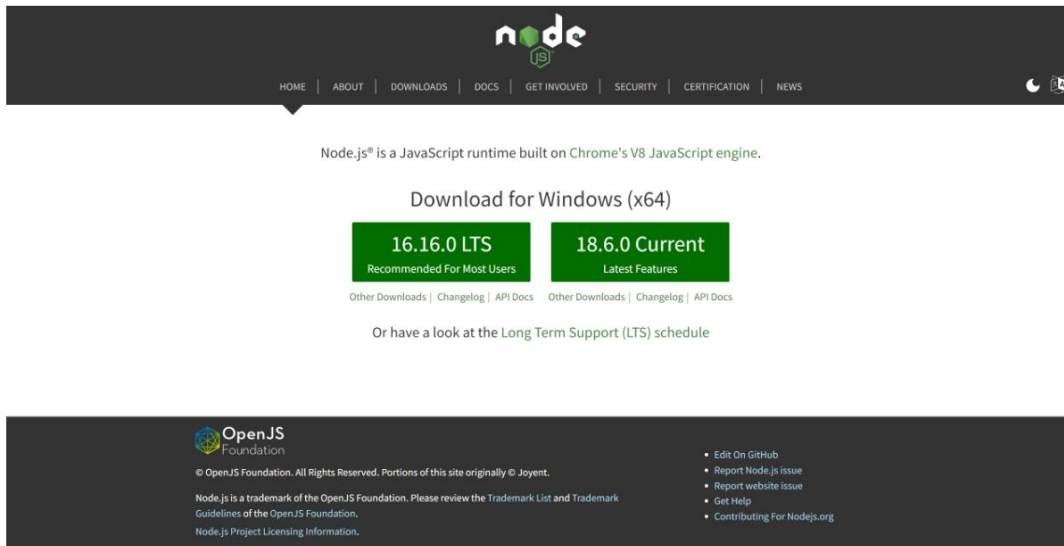
字典可以作为hash表使用

1.2 插件

vscode这个软件，装leetcode这个插件



这个插件需要安装[Node.js \(nodejs.org\)](https://nodejs.org) 安装时会提示是否需要把nodejs添加到环境变量，要添加



2. 二 热身

2.1 两数之和

talk is cheap show me the code

3. 排序

3.1 经典排序

3.1.1 冒泡排序

算法步骤

- 1 比较相邻的元素。如果第一个比第二个大，就交换他们两个。
- 2 对每一对相邻元素作同样的工作，从开始第一对到结尾的最后一对。这步做完后，最后的元素会是最大的数。
- 3 针对所有的元素重复以上的步骤，除了最后一个。
- 4 持续每次对越来越少的元素重复上面的步骤，直到没有任何一对数字需要比较

什么时候最快

当输入的数据已经是正序时（都已经是正序了，我还要你冒泡排序有何用啊）。

什么时候最慢

当输入的数据是反序时（写一个 `for` 循环反序输出数据不就行了，干嘛要用你冒泡排序呢，我是闲的吗）。

```
def bubbleSort(arr):
    for i in range(1, len(arr)):
        for j in range(0, len(arr)-i):
            if arr[j] > arr[j+1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr
```

```
seq = [5,3,0,6,1,4]
print ('排序前: ', seq)
result = bubbleSort(seq)
print ('排序后: ',result)
```

3.1.2 选择排序

算法步骤

- 1 首先在未排序序列中找到最小（大）元素，存放到排序序列的起始位置
- 2 再从剩余未排序元素中继续寻找最小（大）元素，然后放到已排序序列的末尾。
- 3 重复第二步，直到所有元素均排序完毕。

```
def selectionSort(arr):
    for i in range(len(arr) - 1):
        # 记录最小数的索引
        minIndex = i
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[minIndex]:
                minIndex = j
        # i 不是最小数时，将 i 和最小数进行交换
        if i != minIndex:
            arr[i], arr[minIndex] = arr[minIndex], arr[i]
```

```
return arr
```

```
seq = [5,3,0,6,1,4]
print ('排序前: ', seq)
result = selectionSort(seq)
print ('排序后: ',result)
```

3.1.3 插入排序

算法步骤

- 1 将待排序序列第一个元素看做一个有序序列，把第二个元素到最后一个元素当成是未排序序列。
- 2 从头到尾依次扫描未排序序列，将扫描到的每个元素插入1。（如果待插入的元素与有序序列中的某个元素相等，则将待插入元素插入到相等元素的后面。

```
def insertionSort(arr):
    for i in range(1, len(arr)):
        j = i-1
        current = arr[i]
        while j >= 0 and arr[j] > current:
            arr[j+1] = arr[j]
            j -= 1
        arr[j+1] = current

    return arr
```

```
seq = [5,3,0,6,1,4]
print ('排序前: ', seq)
result = insertionSort(seq)
print ('排序后: ',result)
```

3.1.4 归并排序

算法步骤

- 1 申请空间，使其大小为两个已经排序序列之和，该空间用来存放合并后的序列；
- 2 设定两个指针4，最初位置分别为两个已经排序序列的起始位置；

- 3 比较两个指针所指向的元素，选择相对小的元素放入到合并空间，并移动指针到下一位置；
- 4 重复步骤 3 直到某一指针达到序列尾；
- 5 将另一序列剩下的所有元素直接复制到合并序列尾。

```
def mergeSort(arr):
    if(len(arr)<2):
        return arr
    middle = len(arr)//2
    left, right = arr[0:middle], arr[middle:]
    return merge(mergeSort(left), mergeSort(right))
```

```
def merge(left,right):
    result = []
    while left and right:
        if left[0] <= right[0]:
            result.append(left.pop(0));
        else:
            result.append(right.pop(0));
    while left:
        result.append(left.pop(0));
    while right:
        result.append(right.pop(0));
    return result
```

```
seq = [5,3,0,6,1,4]
print ('排序前: ', seq)
result = mergeSort(seq)
print ('排序后: ',result)
```

3.1.5 快速排序

算法步骤

- 1 从数列中挑出一个元素，称为“基准” (pivot) ；

2 重新排序数列，所有元素比基准值小的摆放在基准前面，所有元素比基准值大的摆在基准的后面（相同的数可以到任一边）。在这个分区退出之后，该基准就处于数列的中间位置。这个称为分区（partition）操作；

3 递归地（recursive）把小于基准值元素的子数列和大于基准值元素的子数列排序；

4 递归的最底部情形，是数列的大小是零或一，也就是永远都已经被排序好了。虽然一直递归下去，但是这个算法总会退出，因为在每次的迭代（iteration）中，它至少会把一个元素摆到它最后的位置去。

```
def partition(arr,low,high):
    i = low-1
    pivot = arr[high]

    for j in range(low , high):
        if arr[j] <= pivot:
            i = i+1
            arr[i], arr[j] = arr[j], arr[i]

    arr[i+1], arr[high] = arr[high], arr[i+1]

    return i+1

def quickSort(arr, low=None, high=None):
    low = 0 if not isinstance(low,(int, float)) else low
    high = len(arr)-1 if not isinstance(high,(int, float)) else high

    if low < high:
        pi = partition(arr,low,high)
        quickSort(arr, low, pi-1)
        quickSort(arr, pi+1, high)

    return arr

seq = [54,26,93,17,77,54,31,44,55,20]
print ('排序前: ', seq)
result = quickSort(seq)
print ('排序后: ',result)
```

3.2 148. 排序链表

这是一个单链表

```
class ListNode:
```

```
    def __init__(self, val=0, next=None):
```

```
        self.val = val
```

```
        self.next = next
```

这个一般情况不需要自己构建，一般是默认有这个东西

但是也有可能让我们自己构建

需求：根据列表 `a = [4,2,3,1]` 构造一个链表

```
head = ListNode(val=a[0])
```

```
tmp = head
```

```
for v in a[1:]:
```

```
    t = ListNode(v)
```

```
    tmp.next = t
```

```
    tmp = tmp.next
```

构造完成后，head指向的就是链表的头部

使用归并排序

4. 滑动窗口算法

思路很简单，就是维护一个滑动窗口，不断更新答案。

4.1 3. 无重复字符的最长子串

4.2 76. 最小覆盖子串

思路

1. 在字符串S中使用左右双指针，初始化left=right=0，把左闭右开区间[left,right)称为一个窗口。
2. 不断增加right指针扩大窗口，直到包含T中所有字符
3. 此时停止增加right，转而不断增加left直到窗口中字符串不再符合要求，每次增加left就要更新一轮结果
4. 重复2、3，直到right到达S结尾

4.3 [438. 找到字符串中所有字母异位词](#)

4.4 [567. 字符串的排列](#)

5. 动态规划

求解动态规划的核心问题是穷举。 特点：

1. 存在重叠子问题。如果采用暴力穷举效率低下，所有需要备忘录或者DP table避免不必要的计算。
2. 具备最优子结构。
3. 列出正确状态转移方程。

5.1 [509. 斐波那契数](#)

暴力求解 备忘录求解 DP表求解 状态压缩

5.2 [322. 零钱兑换](#)

这是动态规划问题，因为具有最优子结构，子问题之间互相独立。例如求amount=11时最小硬币数（原问题），如果知道凑出amount=10的最小硬币（子问题），只需要把子问题的答案加1就是原问题答案。硬币数量时没有限制的，所以子问题没有互相制约，是互相独立的。

self.memo 下标就是需要去凑的面额，对应下标存的值，面额数值为下标数值的，凑出这个面额的最小次数

5.3 [416. 分割等和子集](#)

这是0-1背包问题，一个可装载重量为W的背包和N各物品，每个物品有重量和价格两个属性，其中第i个物品的重量为wt[i]，价值为val[i]，现在用这个背包装物品，最多能装的价值是多少？01背包的递推公式为：

$dp[j] = \max(dp[j], dp[j - \text{weight}[i]] + \text{value}[i])$; 假设 $j = 10, \text{weight}[i] = 5$ $dp[5] + \text{value}[i]$

本题，可以先对集合求和，得到总和S，可把问题转化成，给一个可装载重量为S/2的背包和N个物品，每个物品重量为nums[i]，现在装物品，是否存在一种装法，恰好把背包装满。本题，相当于背包里放入数值，那么物品i的重量是nums[i]，其价值也是nums[i]。所以递推公式： $dp[j] = \max(dp[j], dp[j - \text{nums}[i]] + \text{nums}[i])$ ；j表示 背包总容量是j，dp[j]最大可以凑成j的子集总和为dp[j]。 $dp[j] \leq j$

[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

val = 1

j = 11

while j >= val

```
d[11] = 1
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
j = 10
d[10] = 1
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1]
...
j = 1
d[1] = 1
[0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

```
val = 5
j = 11
while j >= val
    d[11] = 6
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 6]
    j = 10
    d[10] = 6
    [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 6, 6]
    ...
    j = 5
    d[5] = 6
    [0, 1, 1, 1, 1, 6, 6, 6, 6, 6, 6, 6]
```

```
val = 11
j = 11
while j >= val
    d[11] = 11
    return True
```

假设第三个数是5

```
[1,5,5,11]
val = 5
j = 11
while j >= val
    d[11] = 11
```

6. 买卖股票最佳时机

[股票问题](#)

6.1 [188. 买卖股票的最佳时机 IV](#)

6.2 [121. 买卖股票的最佳时机](#)

6.3 [122. 买卖股票的最佳时机 II](#)

6.4 [123. 买卖股票的最佳时机 III](#)

6.5 [309. 最佳买卖股票时机含冷冻期](#)

6.6 [714. 买卖股票的最佳时机含手续费](#)

7. 回溯算法

回溯算法就是把所有可能路径都遍历并保存的过程，需要考虑，已经做出的选择、当前可以做的选择、结束条件

```
result = []
```

```
def backtrack(路径, 选择列表):
```

```
    如果满足结束条件
```

```
    result.add(路径)
```

```
    返回
```

```
for 选择 in 选择列表:
```

```
    做选择
```

```
    backtrack(路径, 选择列表)
```

```
    撤销选择
```

7.1 [46. 全排列](#)

1 2 3 1 3 2 2 1 3 2 3 1 3 1 2 3 2 1

7.2 51. N 皇后

按照国际象棋的规则，皇后可以攻击与之处在同一行或同一列或同一斜线上的棋子。比全排列多了一个判断的过程。

```
.Q..  
...Q  
Q...  
..Q.
```

8. BFS算法

广度优先搜索（BFS）和深度优先搜索（DFS）是特别常用的两种算法，DFS可以认为就是回溯算法。核心思想很简单，就是把问题抽象成图，从一个点开始向四周扩散。一般BFS算法都是用队列这种数据结构，每次把一个节点周围所有的节点都加入队列。BFS找到的路径一定是最短路径，但是代价比DFS大很多。一般来说，找最短路径时使用BFS，其他使用DFS。

8.1 111. 二叉树的最小深度

少数情况需要自定义二叉树这种数据结构，大多数情况都不需要自己定义。面试官要求你定义一个二叉树，需要先定义二叉树节点

```
# class TreeNode:  
  
#     def __init__(self, val=0, left=None, right=None):  
  
#         self.val = val  
  
#         self.left = left  
  
#         self.right = right  
  
root = [3,9,20,null,null,15,7]  
->从一个列表构造出二叉树
```

8.2 [752. 打开转盘锁](#)

9. 其它算法题目

[104. 二叉树的最大深度](#)

[5. 最长回文子串](#)

[45. 跳跃游戏 II](#)

[283. 移动零](#)

[19. 删除链表的倒数第 N 个结点](#)

[329. 矩阵中的最长递增路径](#)

[62. 不同路径](#)

[1855. 下标对中的最大距离](#)

[22. 括号生成](#)

[70. 爬楼梯](#)

[300. 最长递增子序列](#)

[20. 有效的括号](#)

[49. 字母异位词分组](#)

九、计算机视觉

1. 图像分类

1.1 GoogLeNet采用了模块化的结构（Inception结构），Inception历经了V1、V2、V3、V4等多个版本的发展，不断趋于完善，请说出3条以上V1-V4演进中的变化

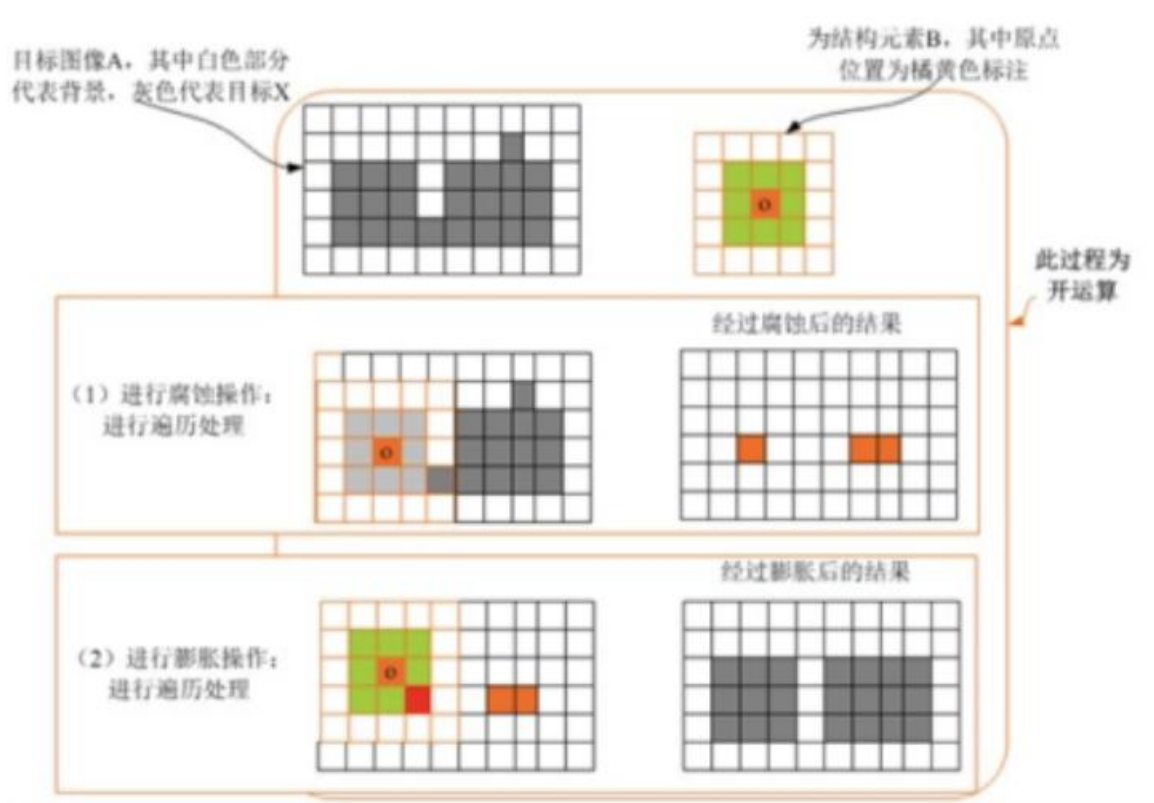
1) Inception V4 相比 V3 主要是结合了ResNet**，加入残差思想**将错误率进一步减少

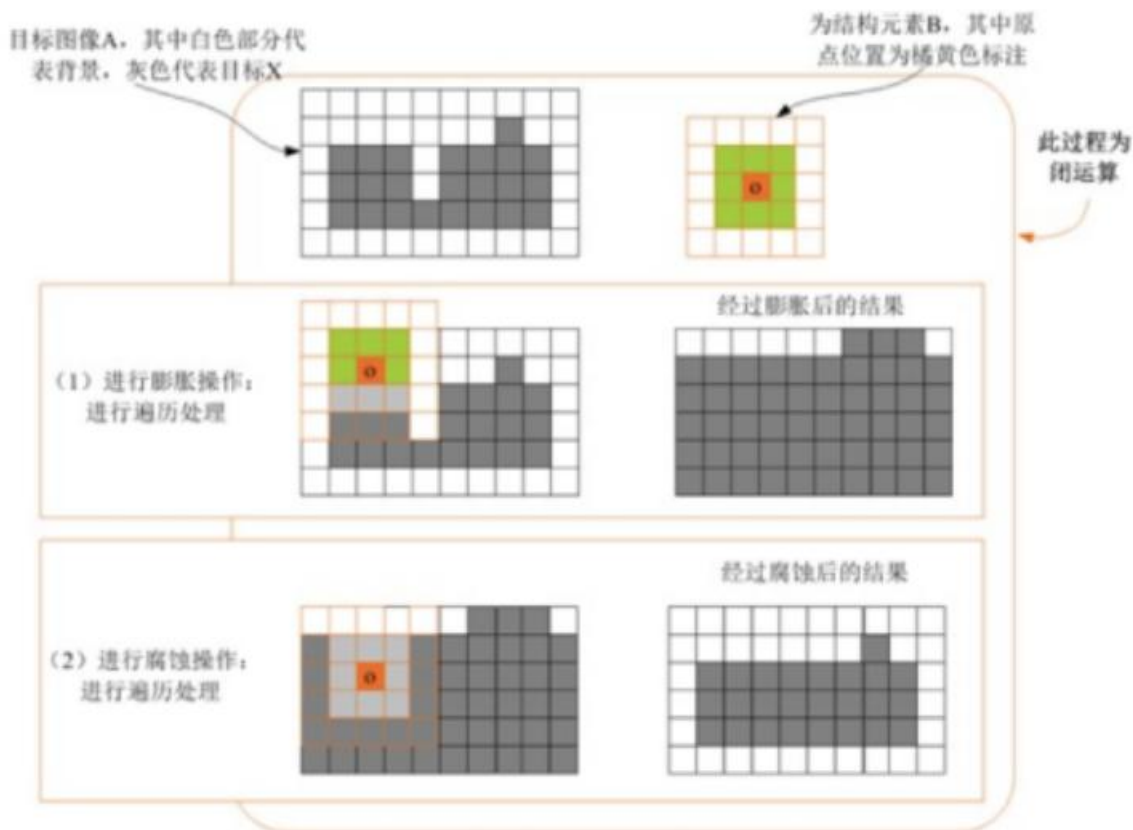
- 2) Inception V3继续分解卷积, $1 \times n + n \times 1$ 代替 $n \times n$ 进一步减少参数**, 将 7×7 分解成两个一维的卷积 ($1 \times 7, 7 \times 1$), 3×3 也是一样 ($1 \times 3, 3 \times 1$), 这样的好处, 既可以加速计算, 又可以将1个卷积拆成2个卷积, 使得网络深度进一步增加, 增加了网络的非线性。
- 3) Inception V2卷积分解, 两个 3×3 代替 5×5 , 即在保持感受野范围的同时又减少了参数量
- 4) Inception V1多尺度融合将CNN中常用的卷积 ($1 \times 1, 3 \times 3, 5 \times 5$)、池化操作 (3×3) 堆叠

1.2 请分别描述开运算，闭运算操作的特点及功能

开运算：先腐蚀运算，再膨胀运算，把细微连在一起的两块目标分开了，开运算能够除去孤立的小点，毛刺和小桥，而总的位置和形状不变。

闭运算：先膨胀运算，再腐蚀运算，将两个细微连接的图块封闭在一起，闭运算能够填平小湖（即小孔），弥合小裂缝，而总的位置和形状不变。





开运算先腐蚀后膨胀，去除了小的黑点，方块边缘变得圆润了，也就是开运算对图像轮廓进行平滑，而闭运算先膨胀后腐蚀，把中心比较大的块连接为一个整体，填补了空白的缝隙。

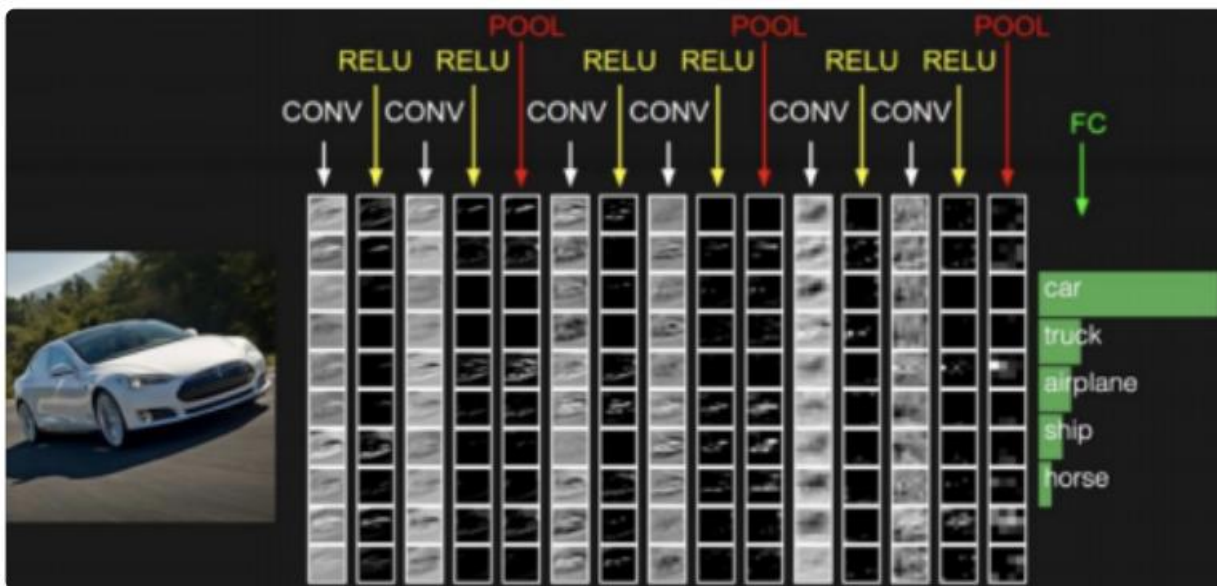
开运算能够除去孤立的小点，毛刺和小桥，而物体的形状轮廓基本不变。

闭运算与开运算相反，能够填平前景物体内的小裂缝、间断和小孔，而总的位置和形状不变。

结构元素大小的不同将导致滤波效果的不同，不同的结构元素的选择导致了不同的分割，即提取出不同的特征。

1.3 卷积神经网络的组成

以 CIFAR-10 数据集举例，一个典型的该数据集上的卷积神经网络分类器应该有[INPUT - CONV - RELU - POOL - FC]的结构，具体说来是这样的：



INPUT[32323]包含原始图片数据中的全部像素，长宽都是 32，有 RGB 3 个颜色通道。

CONV 卷积层中，每个神经元会和上一层的若干小区域连接，计算权重和小区域像素的内积，举个例子可能得出的结果数据是[323212]的。

RELU层，就是神经元激励层，主要的计算就是 $\max(0, x)$ ，结果数据依旧是[323212]。

POOLing 层做的事情，可以理解成一个下采样，可能得到的结果维度就变为[161612]了。

全连接层一般用于最后计算类别得分，得到的结果为[1110]的，其中的10 对应10个不同的类别。和名字一样，这一层的所有神经元会和上一层的所有神经元有连接。

这样，卷积神经网络作为一个中间的通道，就一步步把原始的图像数据转成最后的类别得分了。有一个点要提一下，刚才说到了有几种不同的神经网络层，其中有一些层是有待训练参数的，另外一些没有。详细一点说，卷积层和全连接层包含权重和偏移的；而 RELU 和 POOLing 层只是一个固定的函数运算，是不包含权重和偏移参数的。不过 POOLing层包含了我们手动指定的超参数。

在卷积神经网络中，有 3 种最主要的层：

- 卷积运算层
- pooling 层
- 全连接层及激活函数层

一个完整的神经网络就是由这三种层叠加组成的。

问题拓展

- 一个卷积神经网络由多种不同类型的层(卷积层/全连接层/RELU 层/POOLing 层等) 叠加而成。
- 每一层的输入结构是 3 维的数据，计算完输出依旧是 3 维的数据。
- 卷积层和全连接层包含训练参数，RELU 和 POOLing 层不包含。

- 卷积层，全连接层和 POOLing 层包含超参数，RELU 层没有。

1.4 为什么 CNN 对像素级别的分类很难？

传统的基于 CNN 的分割方法：为了对一个像素分类，使用该像素周围的一个图像块作为 CNN 的输入，用于训练与预测，这种方法主要有几个缺点：

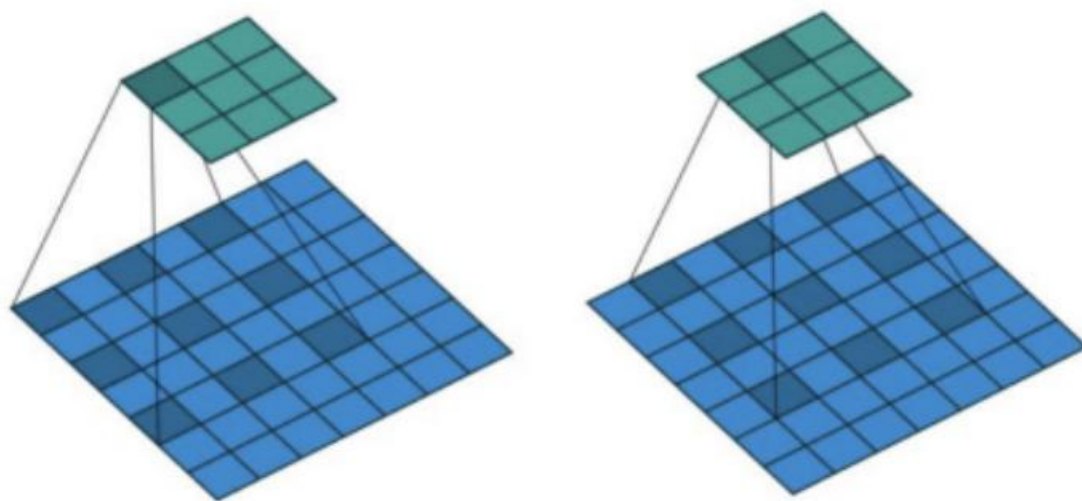
- 存储开销大，例如，对每个像素使用 15×15 的图像块，然后不断滑动窗口，将图像块输入到 CNN 中进行类别判断，因此，需要的存储空间随滑动窗口的次数和大小急剧上升；
- 效率低下，相邻像素块基本上是重复的，针对每个像素块逐个计算卷积，这种计算有很大程度上的重复；
- 像素块的大小限制了感受区域的大小，通常像素块的大小比整幅图像的大小小很多，只能提取一些局部特征，从而导致分类性能受到限制。

而全卷积网络(FCN)则是从抽象的特征中恢复出每个像素所属的类别。即从图像级别的分类进一步延伸到像素级别的分类。

- 存储开销很大。
- 计算效率低下。
- 像素块大小的限制了感知区域的大小。

1.5 什么是扩张卷积？

带洞的 (atrous) 卷积，扩张的卷积引入了另一个被称为扩张率 (dilation rate) 的卷积层。这定义了卷积核中值之间的间隔。一个 3×3 卷积核的扩张率为 2，它的视图与 5×5 卷积核相同，而只使用 9 个参数。想象一下，取一个 5×5 卷积核，每两行或两列删除一行或一列。

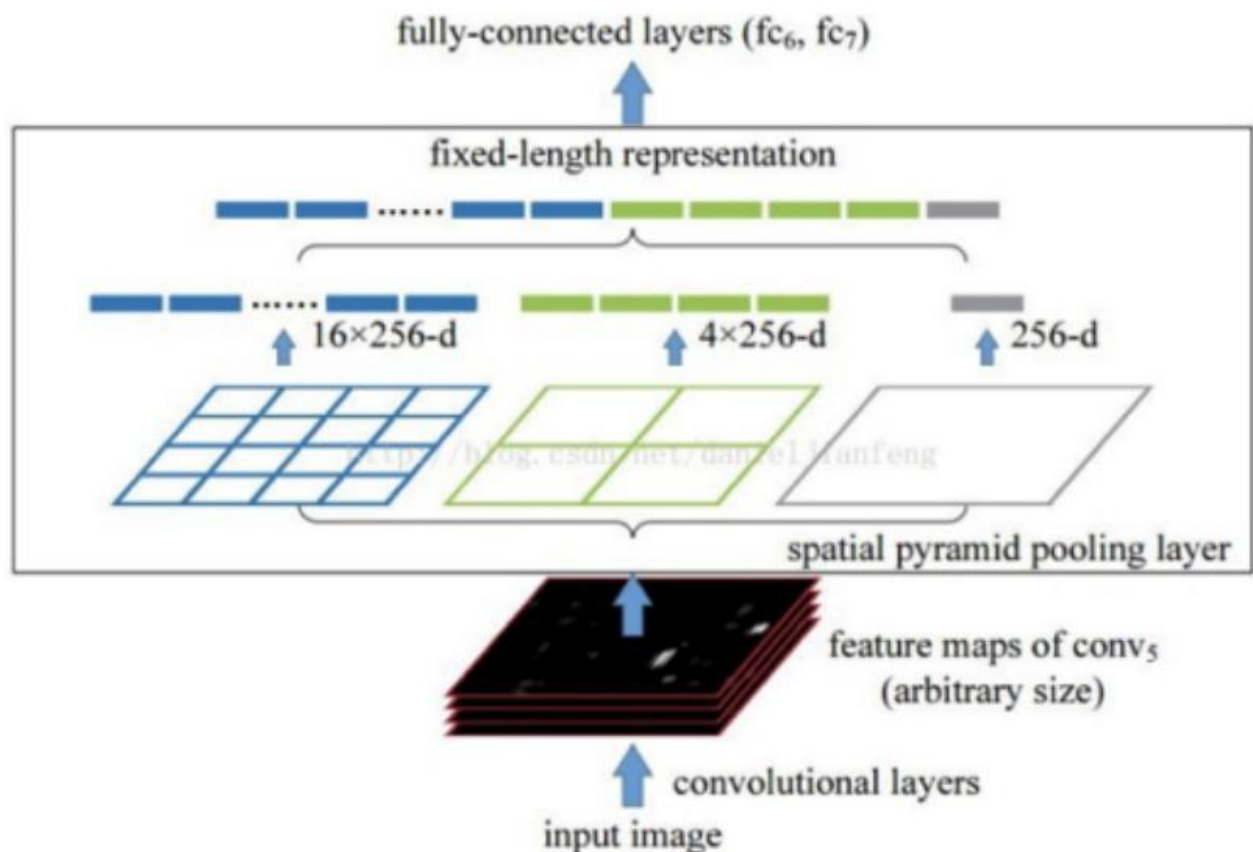


卷积核大小为 3、扩张率为 2 并且无边界扩充的二维卷积

空洞卷积将以同样的计算代价提供更广阔的视角,扩张的卷积在实时分割领域特别受欢迎。如果需要广泛的视图，并且不能负担多个卷积或更大的卷积核，那么可以使用它们。

1.6 什么是空间金字塔池化 (Spatial Pyramid Pooling)

空间金字塔池化的思想来自于 Spatial Pyramid Model，它一个 pooling 变成了多个 scale 的 pooling。用不同大小池化窗口作用于卷积特征，得到1X1,2X2,4X4 的池化结果，由于 conv5 中共有 256 个过滤器，所以得到 1 个 256 维的特征，4 个 256 个特征，以及 16 个 256维的特征，然后把这 21 个 256 维特征链接起来输入全连接层，通过这种方式把不同大小的图像转化成相同维度的特征。



对于不同的图像要得到相同大小的 pooling 结果，就需要根据图像的大小动态的计算池化窗口的大小和步长。假设conv5 输出的大小为 aa，需要得到 nn 大小的池化结果，可以让窗口大小 sizeX 为 $\lceil a/n \rceil$,步长 $\lceil a/n \rceil$ 。下图以 conv5 输出的大小为 13*13 为例。

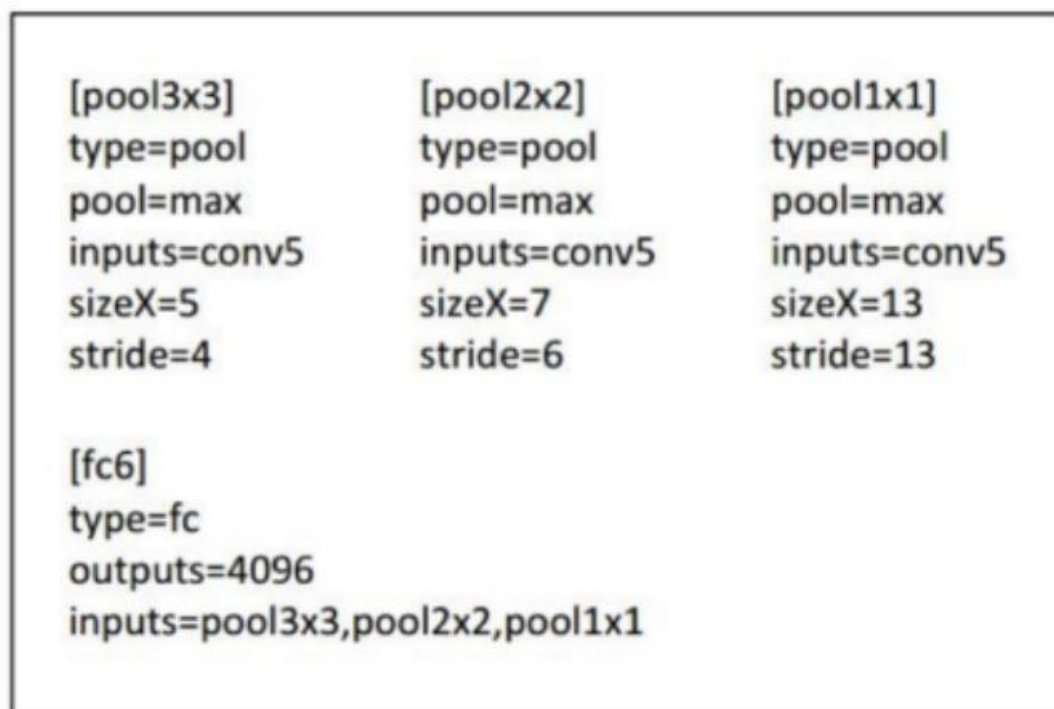
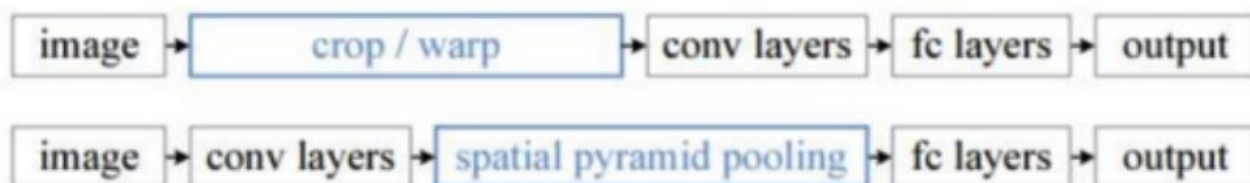


Figure 4: An example 3-level pyramid pooling in the convnet style [3]. Here sizeX is the size of the pooling window. This configuration is for a network whose feature map size of conv₅ is 13×13, so the pool_{3×3}, pool_{2×2}, and pool_{1×1} layers will have 3×3, 2×2, and 1×1 bins respectively.

如果 conv5 输出 的大小 为 1414 ， [pool11]的 sizeX=stride=14 ， [pool22]的sizeX=stride=7， 注意， [pool44]的sizeX=5， stride=4， 最后一列和最后一行特征没有被池化操作计算在内。

一般的 CNN 都需要输入图像的大小是固定的，这是因为全连接层的输入需要固定输入维度，但在卷积操作是没有对图像尺度有限制，所有提出了空间金字塔池化，先让图像进行卷积操作，然后转化成维度相同的特征输入到全连接层，这个可以把 CNN 扩展到任意大小的图像。SPP 其实就是一种多个 scale 的 pooling，可以获取图像中的多尺度信息；在CNN 中加入SPP 后，可以让 CNN 处理任意大小的输入，这让模型变得更加的 flexible。

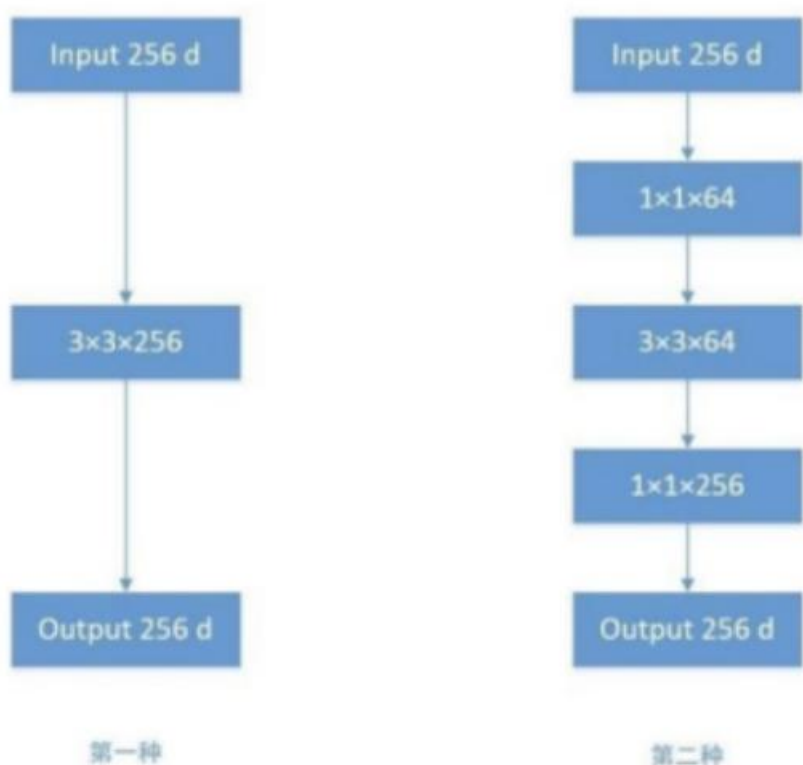
空间金字塔池化可以把任何尺度的图像的卷积特征转化成相同维度，这不仅可以让 CNN处理任意尺度的图像，还能避免cropping 和 warping 操作，导致一些信息的丢失，具有非常重要的意义。



1.7 1×1 卷积核的作用是什么？

根据上图，我们来做对比计算，假设输入 feature map 的维度为 256 维，要求输出维度也是 256 维。有以下两种操作：

1) 256 维的输入直接经过一个 $3 \times 3 \times 256$ 的卷积层，输出一个 256 维的 feature map，那么参数量为： $256 \times 3 \times 3 \times 256 = 589,824$



2) 256 维的输入先经过一个 $1 \times 1 \times 64$ 的卷积层，再经过一个 $3 \times 3 \times 64$ 的卷积层，最后经过一个 $1 \times 1 \times 256$ 的卷积层，输出 256 维，参数量为： $256 \times 1 \times 1 \times 64 + 64 \times 3 \times 3 \times 64 + 64 \times 1 \times 1 \times 256 = 69,632$ 。足足把第一种操作的参数量降低到九分之一！

1×1 卷积核也被认为是影响深远的操作，往后大型的网络为了降低参数量都会应用上 1×1 卷积核。

1×1 的卷积有两个方面的作用：

- 实现跨通道的交互和信息整合。
- 进行卷积核通道数的降维和升维。

1.8 窄卷积和宽卷积分别是什么？

补零法 (zero-padding)：所有落在矩阵范围之外的元素值都默认为0。这样就可以对输入矩阵的每一个元素做滤波了，输出一个同样大小或是更大的矩阵。

补零法又被称为是宽卷积，不使用补零的方法则被称为窄卷积。

在pytorch的实现中,conv2d的padding='SAME',默认的就是宽卷积的方式,会进行补零。padding = 'VALID'就是窄卷积的方式。

具体如下所示：

- "VALID" = without padding:

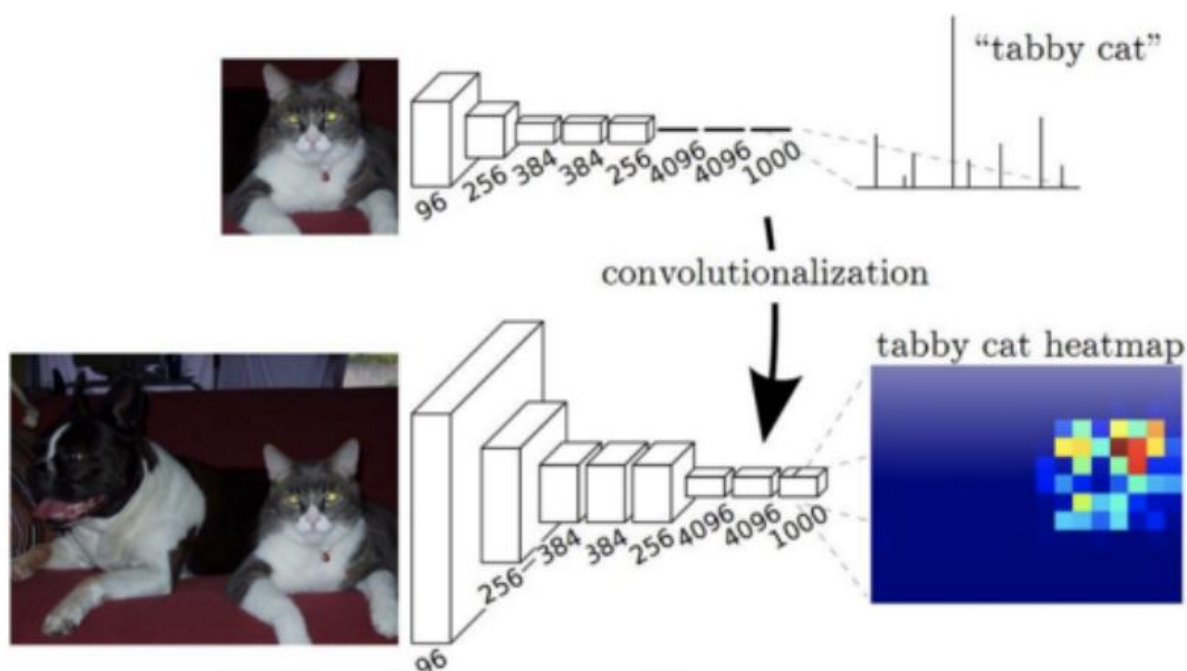


- "SAME" = with zero padding:



1.9 FCN 网络结构是什么？

通常 CNN 网络在卷积层之后会接上若干个全连接层，将卷积层产生的特征图(feature map) 映射成一个固定长度的特征向量。以 AlexNet 为代表的经典 CNN 结构适合于图像级的分类和回归任务，因为它们最后都得到整个输入图像的一个概率向量，比如 AlexNet 的 ImageNet 模型输出一个 1000 维的向量表示输入图像属于每一类的概率(softmax 归一化)。



如图所示：

- 1) 在 CNN 中, 猫的图片输入到 AlexNet, 得到一个长为 1000 的输出向量, 表示输入图像属于每一类的概率, 其中在“tabby cat”这一类统计概率最高, 用来做分类任务。
- 2) FCN 与 CNN 的区别在于把于 CNN 最后的全连接层转换成卷积层, 输出的是一张已经 Label 好的图片, 而这个图片就可以做语义分割
- 3) CNN 的强大之处在于它的多层结构能自动学习特征, 并且可以学习到多个层次的特征: 较浅的卷积层感知域较小, 学习到一些局部区域的特征 较深的卷积层具有较大的感知域, 能够学习到更加抽象一些的特征。高层的抽象特征对物体的大小、位置和方向等敏感性更低, 从而有助于识别性能的提高, 所以我们常常可以将卷积层看作是特征提取器。

FCN 对图像进行像素级的分类, 从而解决了语义级别的图像分割 (semantic segmentation) 问题。与经典的 CNN 在卷积层之后使用全连接层得到固定长度的特征向量进行分类 (全联接层 + softmax 输出) 不同, FCN 可以接受任意尺寸的输入图像, 采用反卷积层对最后一个卷积层的 feature map 进行上采样, 使它恢复到输入图像相同的尺寸, 从而可以对每个像素都产生了一个预测, 同时保留了原始输入图像中的空间信息, 最后在上采样的特征图上进行逐像素分类。

1.10 如何进行卷积神经网络进行参数设置?

- 1) learning-rate 学习速率: 学习速率越小, 模型收敛花费的时间就越长, 但是可以提高模型精确度。一般初始设置为0.1, 然后每次除以 2 或者 5 来改进, 得到最终值;
- 2) batch-size 样本批次容量: 影响模型的优化程度和收敛速度, 需要参考你的数据集大小来设置, 具体问题具体分析;
- 3) weight-decay 权重衰减: 用来在反向传播中更新权重和偏置, 一般设置为 0.005;
- 4) epoch-number 训练次数: 包括所有训练样本的一个正向传递和一个反向传递, 训练至 模型收敛即可; (注: 和迭代周期 iteration 不一样) 而且, 也不是训练的次数越多, 测试精度就会越高, 可能会因为预训练的模型太复杂, 而数据集样本数量太少, 种类太单一, 而出现过拟合

具体如下所示:

- 1) learning-rate 学习速率
- 2) batch-size 样本批次容量
- 3) weight-decay 权重衰减
- 4) epoch-number 训练次数

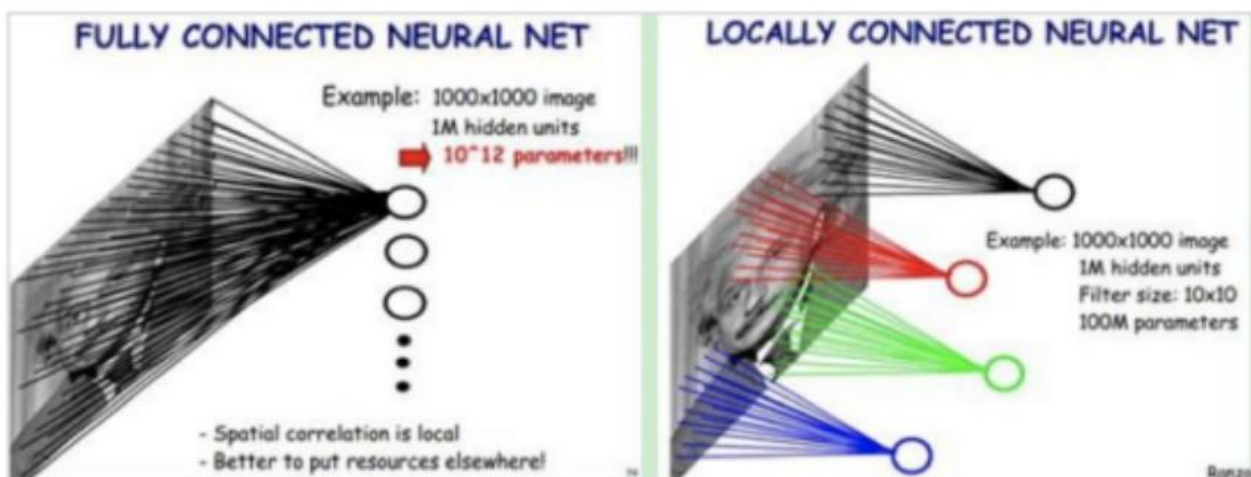
一般来说, 调参经验及提高泛化能力的方法主要有以下几个:

- 正则化
- 增加神经网络层数
- 使用正确的代价函数
- 使用好的权重初始化技术

- 人为拓展训练集
- 弃权技术

1.11 CNN为什么要进行局部连接？

下图是一个很经典的图示，左边是全连接，右边是局部连接



对于一个 1000×1000 的输入图像而言，如果下一个隐藏层的神经元数目为 10^6 个，采用全连接则有 $1000 \times 1000 \times 10^6 = 10^{12}$ 个权值参数，如此数目巨大的参数几乎难以训练；

而采用局部连接，隐藏层的每个神经元仅与图像中 10×10 的局部图像相连接，那么此时的权值参数数量为 $10 \times 10 \times 10^6 = 10^8$ ，将直接减少 4 个数量级。

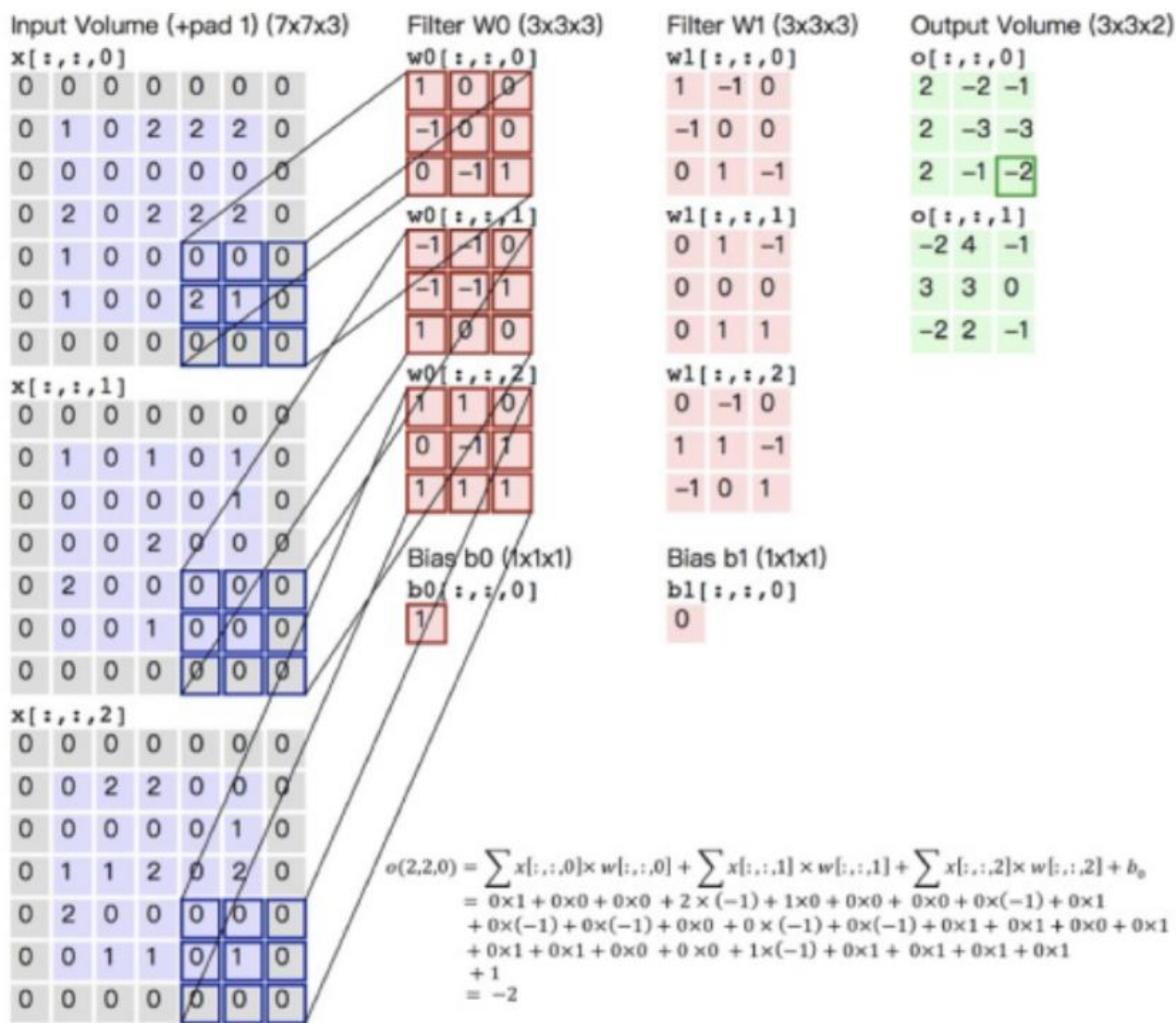
局部连接使网络可以提取数据的局部特征且参数量更少。

在这里首先明白一个概念——感受野，即每个神经元仅与输入神经元的一块区域连接，这块局部区域称作感受野（receptive field）。

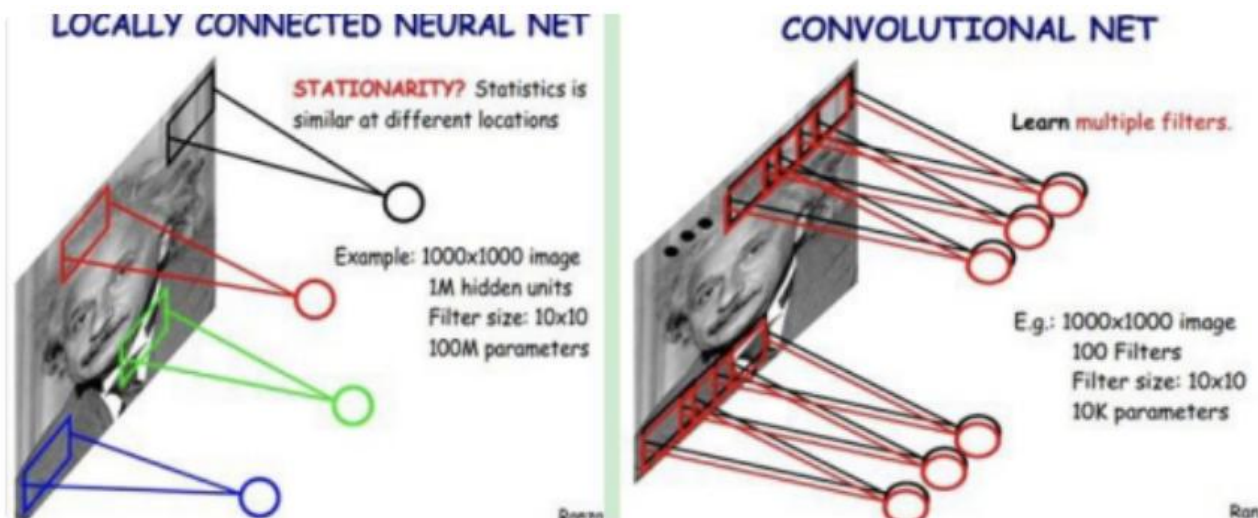
在图像卷积操作中，即神经元在空间维度是局部连接。对于二维图像本身而言，也是局部像素关联较强。这种局部连接保证了学习后的过滤器能够对于局部的输入特征有最强的响应。局部连接的思想，也是受启发于生物学里面的视觉系统结构，视觉皮层的神经元就是局部接受信息的。

1.12 什么是权值共享，有什么用？

权重共享：计算同一个深度切片的神经元时采用的滤波器是共享的。如下图中计算 $o[:, :, 0]$ 的每个神经元的滤波器均相同都为 W_0 ，这样可以很大程度上减少参数。



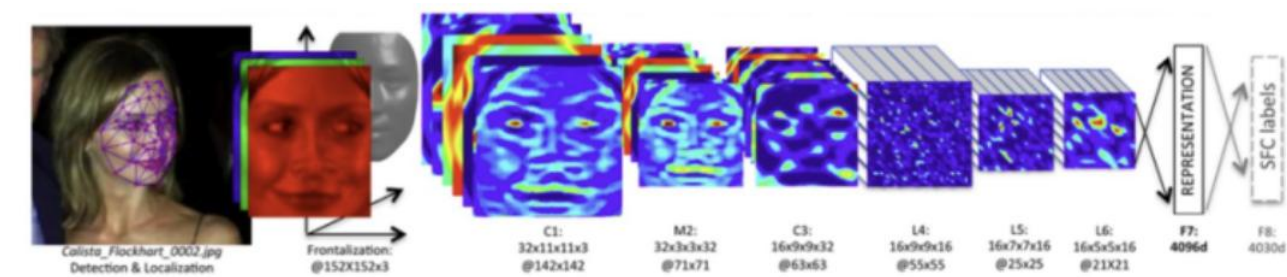
具体做法是，在局部连接中隐藏层的每个特征图连接的是一个 10×10 的卷积核，因此有 10×10 个权值参数，将这 10×10 个权值参数共享给剩下的特征图，也就是说此时不管隐藏层神经元的数目是多少，需要训练的参数就是这 10×10 个权值参数（也就是卷积核(也称滤波器)的大小），如下图。



共享权重在一定程度上是有意义的，例如图片的底层边缘特征与特征在图中的具体位置无关。但是在一些场景中是无意的，比如输入的图片是人脸，眼睛和头发位于不同的位置，希望在不同的位置学到不同的特征。注意权重只是对于同一深度切片的神经元是共享的，在卷积层，通常采用多组卷积核提取不同特征，即对应不同深度切片的特征，不同深度切片的神经元权重是不共享。另外，偏重对同一深度切片的所有神经元都是共享的并且权值共享大大降低了网络的训练难度，减少了参数量。

上述仅提取了图像的一种特征，如果要多提取出一些特征，可以增加多个卷积核，不同的卷积核能够得到图像的不同映射下的特征，称之为 Feature Map。如果有 100 个卷积核，最终的权值参数也仅为 $100 \times 100 = 10^4$ 个而已。另外，偏置参数也是共享的，同一种滤波器共享一个。

1.13 为什么很多做人脸的项目会最后加入一个 Local Connected Conv (即参数不共享) ?



以上图为例，图像都是 152×152 的图像，输入到上述网络结构中，该结构的参数如下：

- Conv: 32 个 $11 \times 11 \times 3$ 的卷积核 max-pooling: 3×3 , stride=2 Conv: 16 个 9×9 的卷积核。
- Local-Conv: 16 个 9×9 的卷积核，Local 的意思是卷积核的参数不共享。
- Local-Conv: 16 个 7×7 的卷积核，参数不共享 Local-Conv: 16 个 5×5 的卷积核，参数不共享 Fully-connected: 4096 维 Softmax: 4030 维。

前三层的目的在于提取低层次的特征，比如简单的边和纹理。其中 Max-pooling 层使得卷积的输出对微小的偏移情况更加鲁棒。但没有用太多的 Max-pooling 层，因为太多的 Max-pooling 层会使得网络损失图像信息。

全连接层将上一层的每个单元和本层的所有单元相连，用来捕捉人脸图像不同位置的特征之间的相关性。其中，第 7 层 (4096-d) 被用来表示人脸。全连接层的输出可以用于 Softmax 的输入，Softmax 层用于分类。

- 1) 不共享的卷积核并不增加抽取特征时的计算量，而会增加训练时的计算量
- 2) 使用不共享的卷积核，需要训练的参数量大大增加，因而需要很大的数据量，然而这个条件在某些人脸项目中刚好满足。

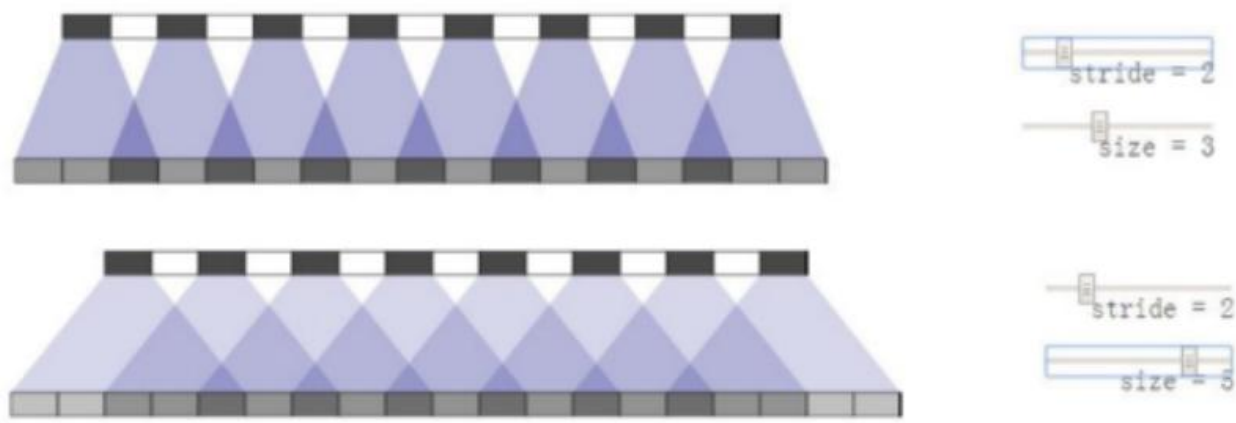
1.14 为什么出现棋盘现象？

现在，生成图像时，神经网络通常使用多层反卷积，从一系列较低分辨率的描述中迭代建立更大的图像。直观地看，假设生成的图像中包含 1 只黑猫。黑猫身体部分的像素颜色应平滑过渡，或极端地说，该部分应全为黑色。实际生成的图像中该部分却有深深浅浅的近黑方块组成，很像棋盘的网格，即棋盘效应。

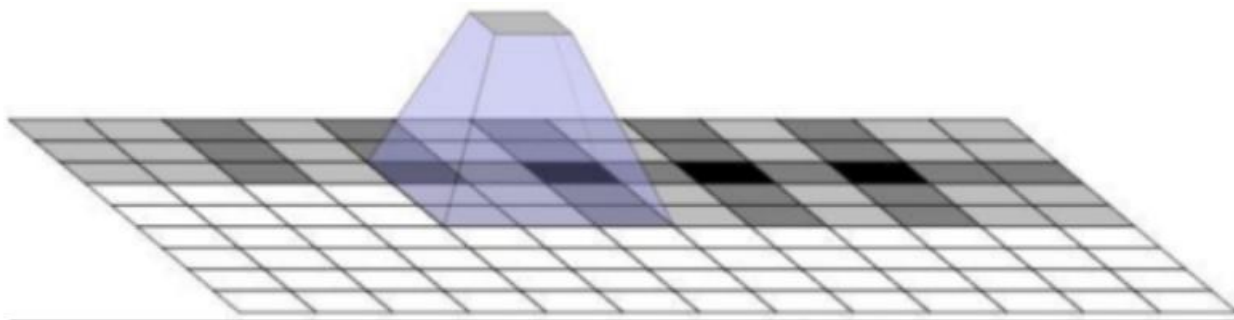


图像生成网络的上采样部分通常用反卷积网络，不合理的卷积核大小和步长会使反卷积操作产生棋盘效应 (checkerboard artifacts)。

问题产生：kernel size无法被stride整除，stride为1，kernel size为3，这导致了重复上采样绘图的部分不均匀（下图深色部分）



重叠图案也在二维中形成，两个轴上的不均匀重叠相乘，产生不同亮度的棋盘状图案。事实上，不均匀重叠往往在二维上更极端！因为两个模式相乘，所以它的不均匀性是原来的平方。



1.15 有哪些方法可以避免棋盘效应?

缩放卷积为线性操作：假设原图像为 A ，经过插值后的图像为 $A+B$ ；用卷积核 C 对插值缩放后的图像卷积，得到最终的图像 $C^{**T}(A^{*}+B) = C^{**T} * A + C^{**T} * B$ ，其中 $*$ 为卷积操作。则可将缩放卷积分解为原图像卷积和插值增量图像卷积。

C 为卷积操作的卷积核。此时为上采样，理解为反卷积操作中的卷积核。

第一种方法是用到的反卷积核的大小可被步长整除，从而避免重叠效应。

2. 目标检测

2.1 从R-CNN到Faster R-CNN谈谈物体检测

物体检测模型从R-CNN到Faster R-CNN代表着深度学习模型在物体检测领域取得了关键性的突破，与传统的物体检测方法产生了分离，把物体检测带进了深度学习时代。

模型进化流程：

1) R-CNN

- (1) 候选区域选择
- (2) CNN特征提取
- (3) 分类与边界框回归

2) Fast R-CNN

- 引入ROI Pooling层，输入图片无需缩放

3) Faster R-CNN

- 引入RPN模块生成候选框，CNN取代Selective Search传统提取方法

2.2 目标检测two-stage模型有哪些

Two-stage模型先由算法生成一系列样本的候选框(anchor)，再通过卷积神经网络进行样本分类，r-cnn 采

用传统的ss方法生成anchor，后续的模型采用cnn方法生成anchor。

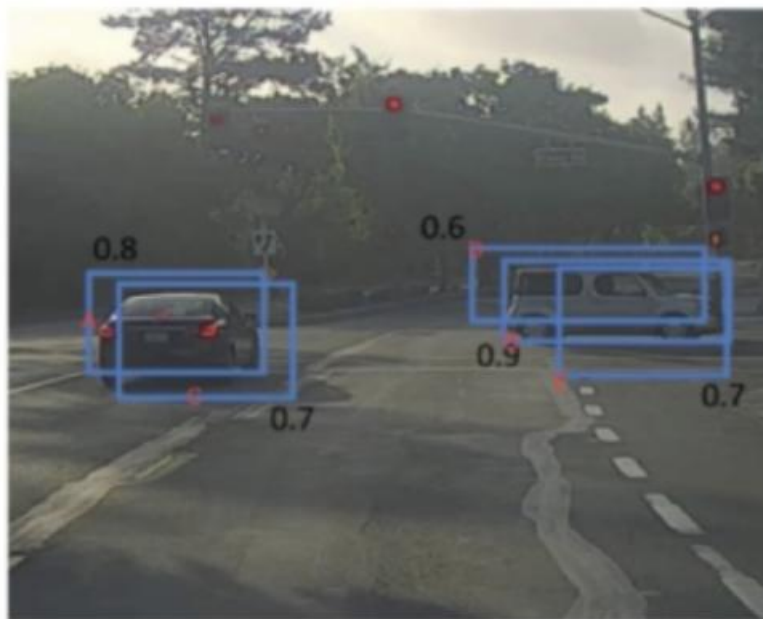
RCNN-->SPPNet-->Fast RCNN-->Faster RCNN-->RFCN-->DCN-->DCNv2

2.3 说一下NMS 的处理思想

NMS是一种Post-Processon（后处理）方式，跟算法无关的方式，应用在大多数物体检测的方法里。

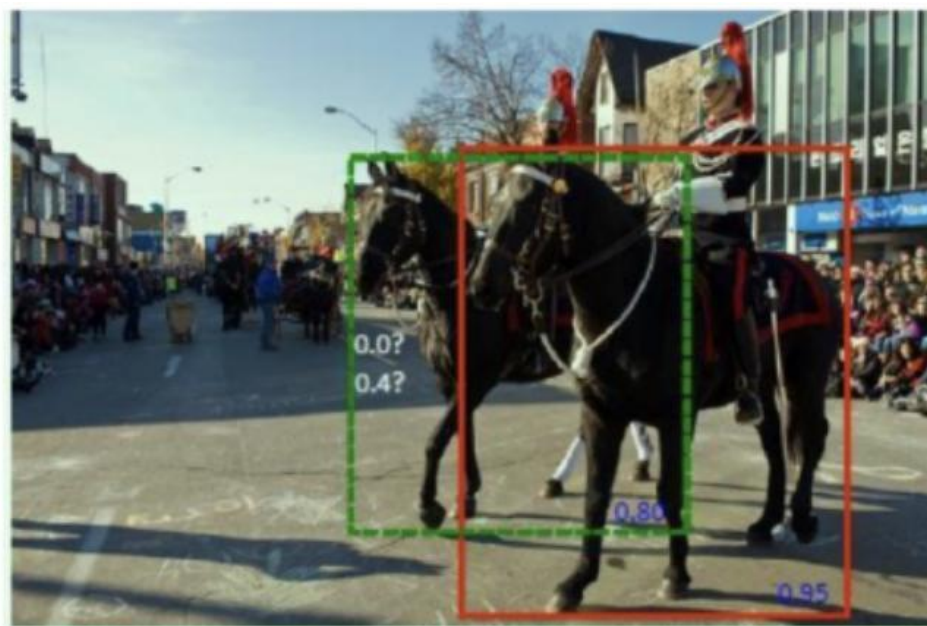
NMS把所有候选框的检测结果按照分值从高到底排序,保留最高分数的box, 删除其余值。

NMS用于筛选候选区域。



2.4 Soft nms为什么可以提升模型效果

如图所示，检测算法本来应该输出两个框，但是传统的NMS算法可能会把score较低的绿框过滤掉（如果绿框和红框的IOU大于设定的阈值就会被过滤掉），导致只检测出一个object（一个马）



NMS公式

$$s_i = \begin{cases} s_i, & \text{iou}(\mathcal{M}, b_i) < N_t \\ 0, & \text{iou}(\mathcal{M}, b_i) \geq N_t \end{cases}$$

Soft NMS公式

$$s_i = \begin{cases} s_i, & \text{iou}(\mathcal{M}, b_i) < N_t \\ s_i(1 - \text{iou}(\mathcal{M}, b_i)), & \text{iou}(\mathcal{M}, b_i) \geq N_t \end{cases}$$

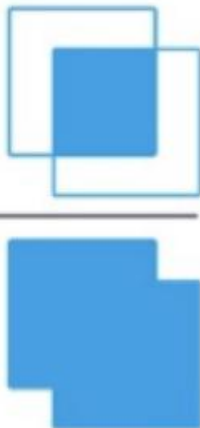
弥补NMS方法的不足，提高密集候选区域筛选质量

2.5 描述一下IOU的作用

IOU主要用在模型训练阶段时计算正负样本，以及模型测试阶段帮助NMS筛选候选框

IOU计算：

两个区域的重叠程度overlap：候选区域和标定区域的IoU值

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$


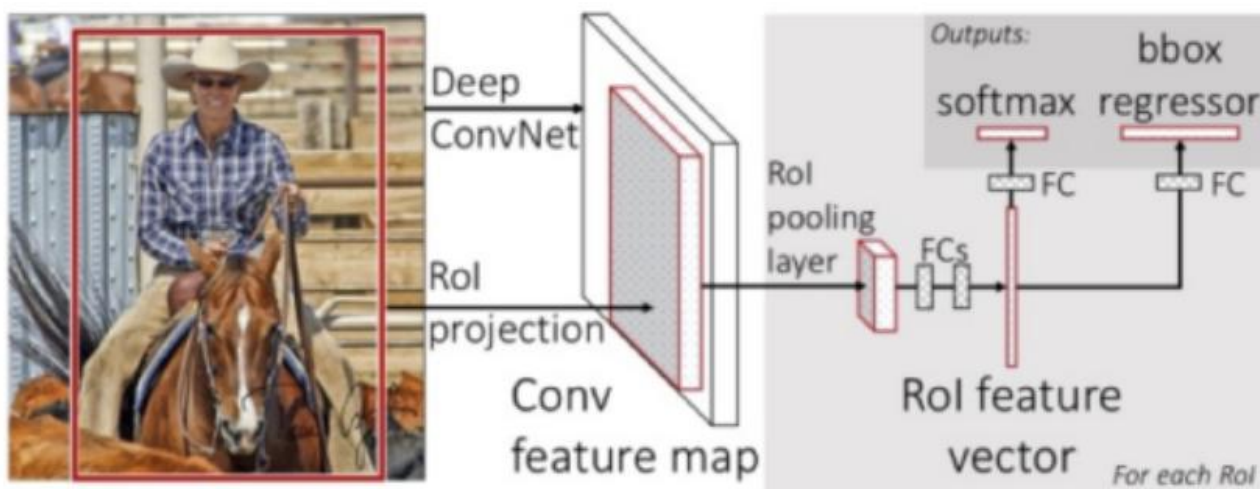
通常Correct: 类别正确 且 $IoU > 0.5$

IoU 用于计算“预测的边框”和“真实的边框”的交集和并集的比值

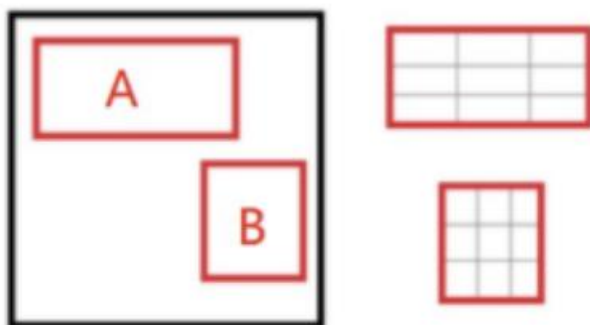
2.6 roi pooling 实现怎样的映射

在RCNN中，在进行卷积操作之前一般都是先将图片分割与形变到固定尺寸，这会让图像产生形变，或者图像变得过小，使一些特征产生了损失，继而对之后的特征选择产生影响。

Fast RCNN在数据的输入上并不对其有什么限制，而实现这一没有限制的关键所在正是ROI Pooling层。



RoI Pooling层将每个候选区域分为 $m * n$ 个块。针对每个块执行最大池化操作，使得特征映射上不同大小的候选区域被变换为均匀大小的特征向量



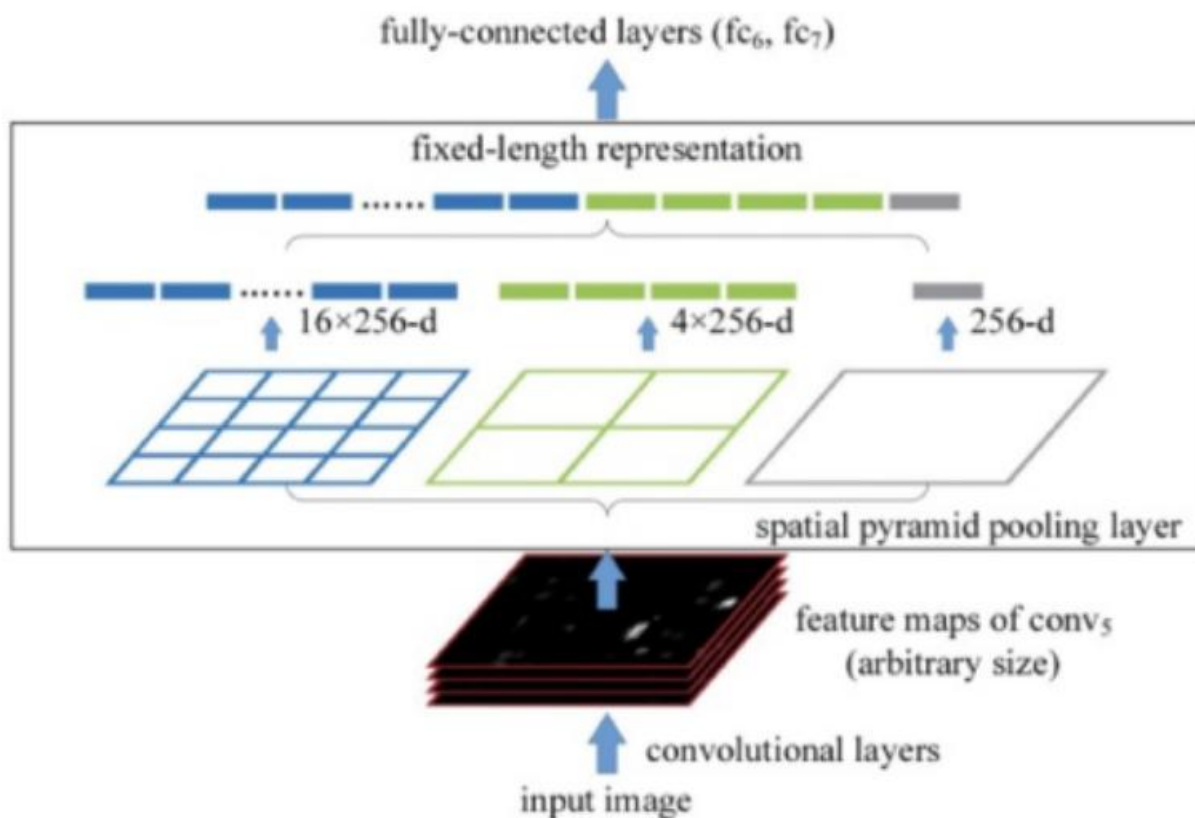
ROI Pooling层可以把不同大小的候选区域特征映射为固定的维度特征

2.7 SPP具体是指什么

在SPPNet中引入了一种空间金字塔池化(spatial pyramid pooling, SPP)层，用于移除模型对图片固定尺寸的限制。

spp layer会将每一个候选区域分成1x1, 2x2, 4x4三张子图，对每个子图的每个区域作max pooling。

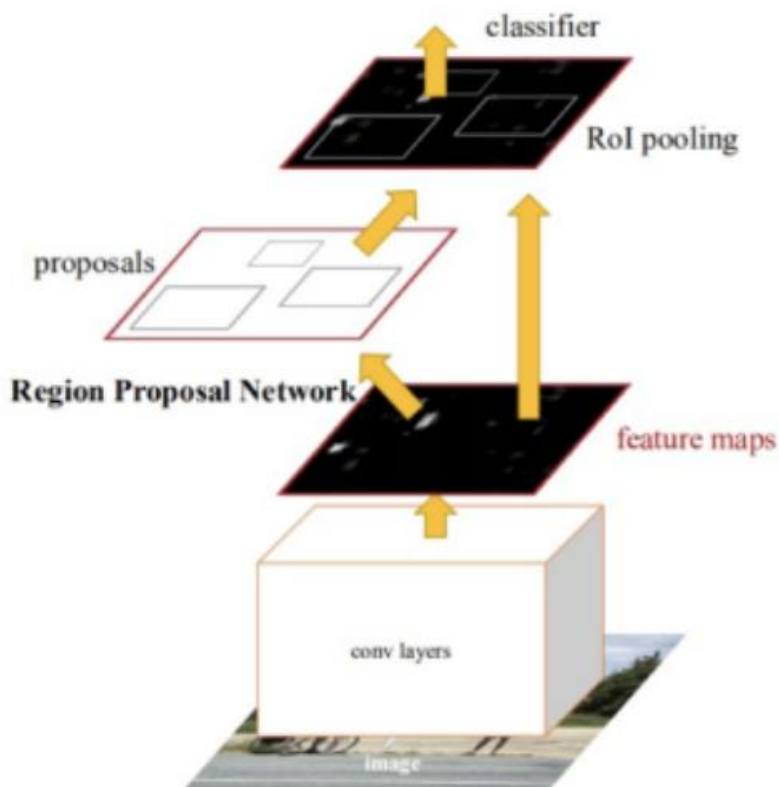
SPP层放在最后一个卷积层之后。SPP层对特征进行池化，并产生固定长度的输出，这个输出送给全连接层



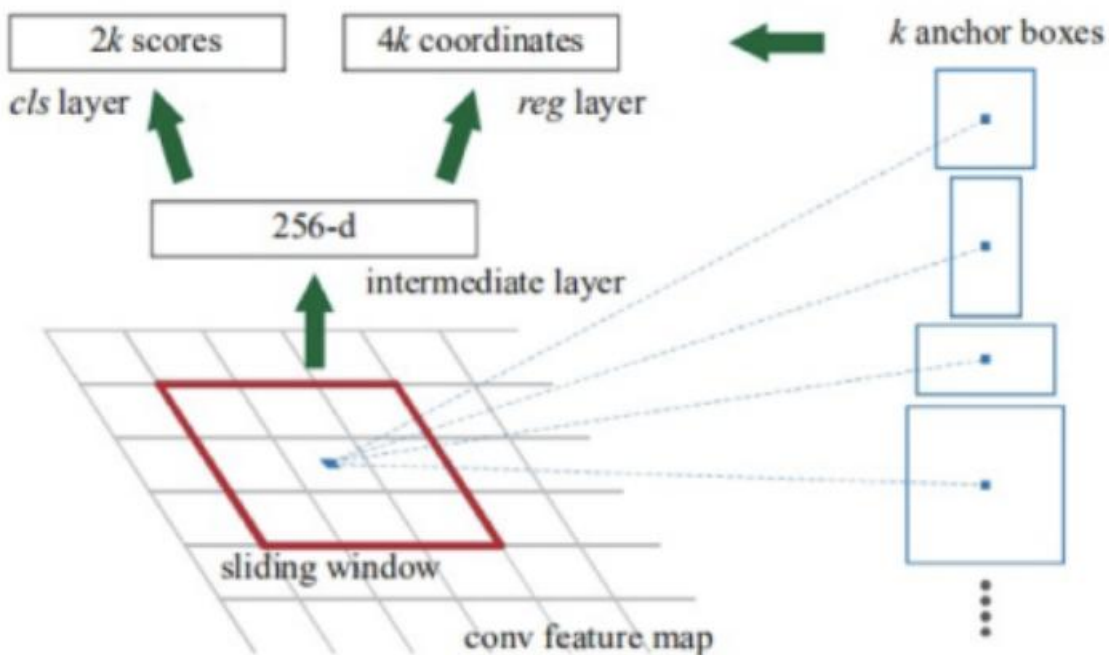
SPP具体是指空间金字塔池化层，为全连接层生成固定长度的特征向量

2.8 说一下RPN的作用

RPN网络出现在Faster r-cnn 模型中，用来代替Fast R-CNN中的选择性搜索方法



！ 在每一个滑动窗口的位置，同时预测 k 个候选框，即 $4k$ 个坐标值， $2k$ 个二分类得分

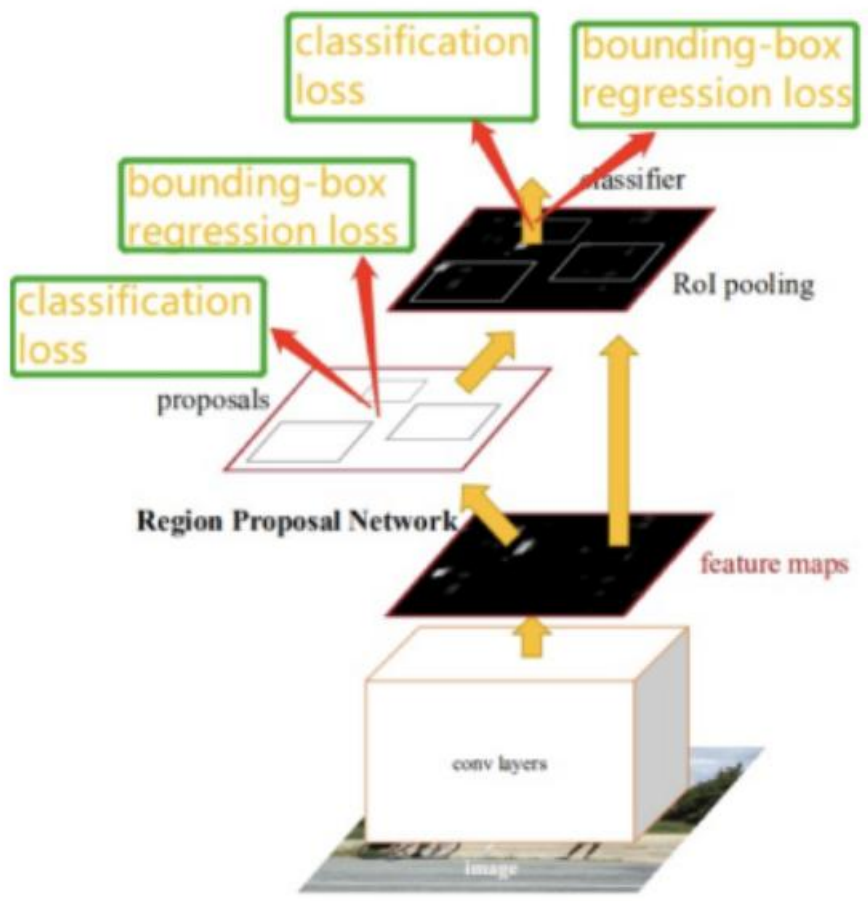


RPN本身是一个全卷积网络，通过训练生成高质量的目标候选框

2.9 说一下faster rcnn的损失函数

Faster R-CNN可以简单地看成是rpn+Fast R-CNN的模型，rpn基于二分类的分类Loss和回归loss，fast r-cnn的多类分类和回归Loss，共产生4个损失函数。

分类采用softmax loss，回归采用smooth loss。



Faster rcnn 共有4个损失函数，rpn的分类和回归Loss，后续物体识别产生的分类和回归loss。

2.10 Faster rcnn怎么筛选正负anchor

Faster rcnn 在生成目标候选框的过程中,会产生大量的非目标区域候选框,通过计算候选框与标注框的iou,来标定正负候选框,即为筛选正负anchor。

Rpn 正负样本标记:

- 正样本: 与ground truth (GT) 的iou大于0.7的anchor, 注意一个GT可能分配正标签给多个anchor
- 负样本: 与ground truth (GT) 的iou小于0.3的anchor, 非正非负的anchor对训练目标没有作用
- 训练时: 正样本 128个, 负样本128个, 比例 1: 1

后续分类的正负样本标记:

- 正样本: 与ground truth (GT) 的iou大于0.5的anchor

- 负样本：与ground truth (GT) 的iou小于0.5的anchor
- 训练时：正样本 32个，负样本96个，比例 1: 3

Faster rcnn通过生成的anchor与gt的iou大小，标定正负样本

2.11 为什么使用focal loss

在目标检测过程中，常出现类别不平衡问题，focal loss从损失函数的角度出发，根据置信度动态调整交叉熵loss，当预测正确的置信度增加时，loss的权重系数会逐渐衰减至0，大量容易的例子对loss贡献很低，这样模型训练的loss更关注难例。

以二分类介绍focal loss，二分类的cross entropy (CE) loss，定义如下：

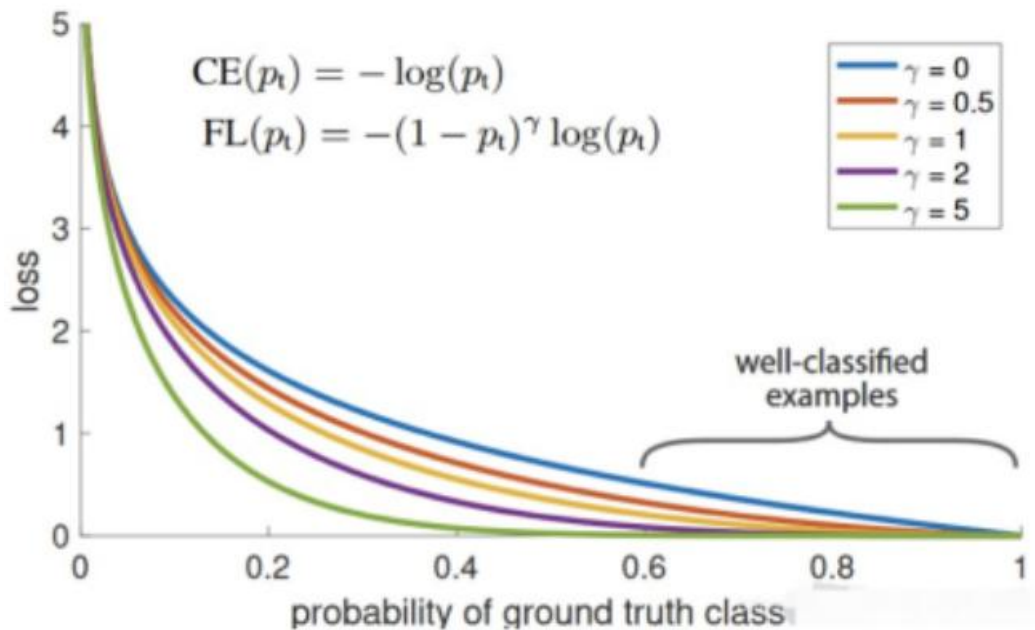
$$CE(p, y) = \begin{cases} -\log(p) & \text{if } y = 1 \\ -\log(1 - p) & \text{otherwise.} \end{cases}$$

其中 $y \in \{-1, 1\}$ 为真实标签，1表示为正例，-1表示为负例， $p \in [0, 1]$ 为模型预测为正例的概率值，进一步可以定义：

$$p_t = \begin{cases} p & \text{if } y = 1 \\ 1 - p & \text{otherwise,} \end{cases}$$

这样CE就可以简写为： $CE(p, y) = CE(p_t) = -\log p_t$

如下图所示，蓝色的曲线表示CE，如果定义 $p_t > 0.5$ 的易分类样本，从曲线可以看到，这部分样本的loss值依然不低，而且这部分例子要占很大比例，加起来后将影响难分类样本的loss，这就是CE loss用于目标检测模型训练所存在的问题



为了解决CE的问题，FL在CE基础上增加一个调节因子 $(1-p_t)^\gamma$ ，FL定义如下

$$FL(p_t) = -(1 - p_t)^\gamma \log p_t$$

当 p_t 很小时，调节因子值接近1，loss不受影响，而当 p_t 趋近于1时，调节因子接近0，这样已经能正确分类的样例loss大大降低。此时若一个样本的 p_t 为0.9，其对应的CE loss是FL的100倍，可见FL相比CE可以大大降低简单例子的loss，使模型训练更关注于难例

Focal loss通过减少易分类样本的权重，使模型在训练时更专注于难分类的样本，提高模型性能。

2.12 目标检测one-stage模型有哪些

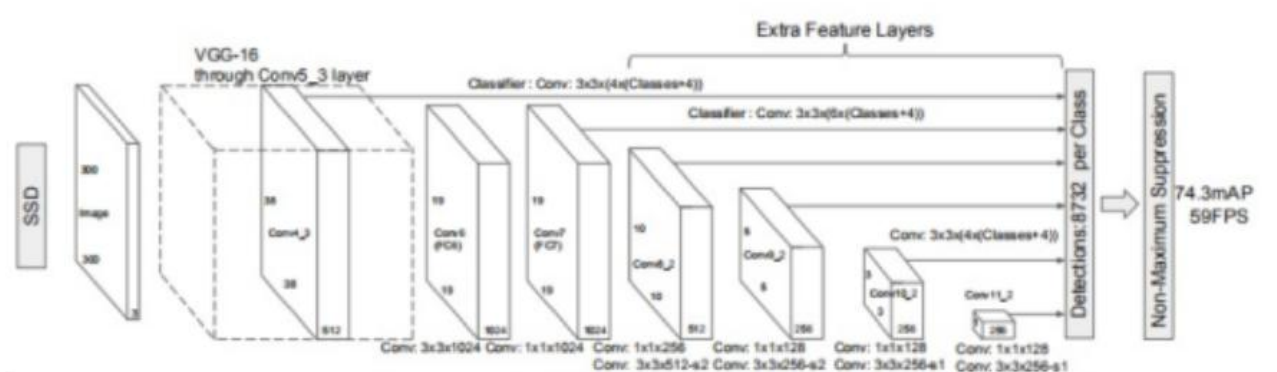
One-stage模型不用单独设计算法产生目标的候选框，直接将目标边框检测转化为回归问题处理，把目标的定位和分类合并为一个问题

One-stage 模型有yolo系列、ssd系列等

2.13 SSD如何生成目标框

SSD在多尺度的feature map上预设anchor。

在base network之后加了几层卷积，这些卷积层会逐渐减小feature map的size，然后在不同size的feature map上进行预设anchor



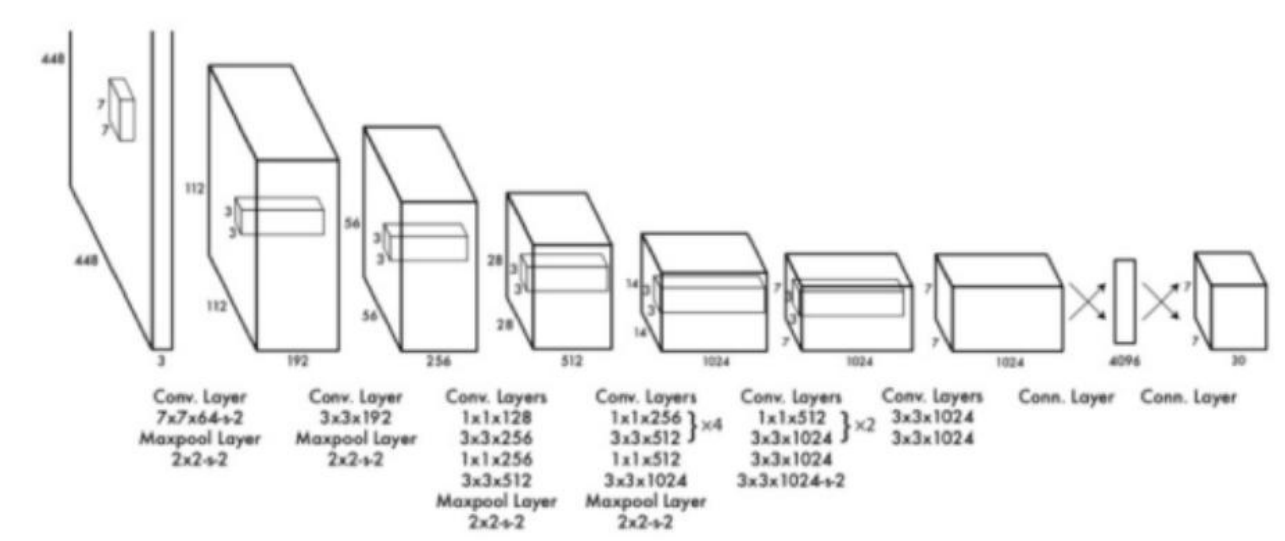
最终anchor的数量为：

$$38 \times 38 \times 4 + 19 \times 19 \times 6 + 10 \times 10 \times 6 + 5 \times 5 \times 6 + 3 \times 3 \times 4 + 1 \times 1 \times 4 = 8732$$

SSD在基础网络之后的多尺度的feature map上每个位置预设anchor

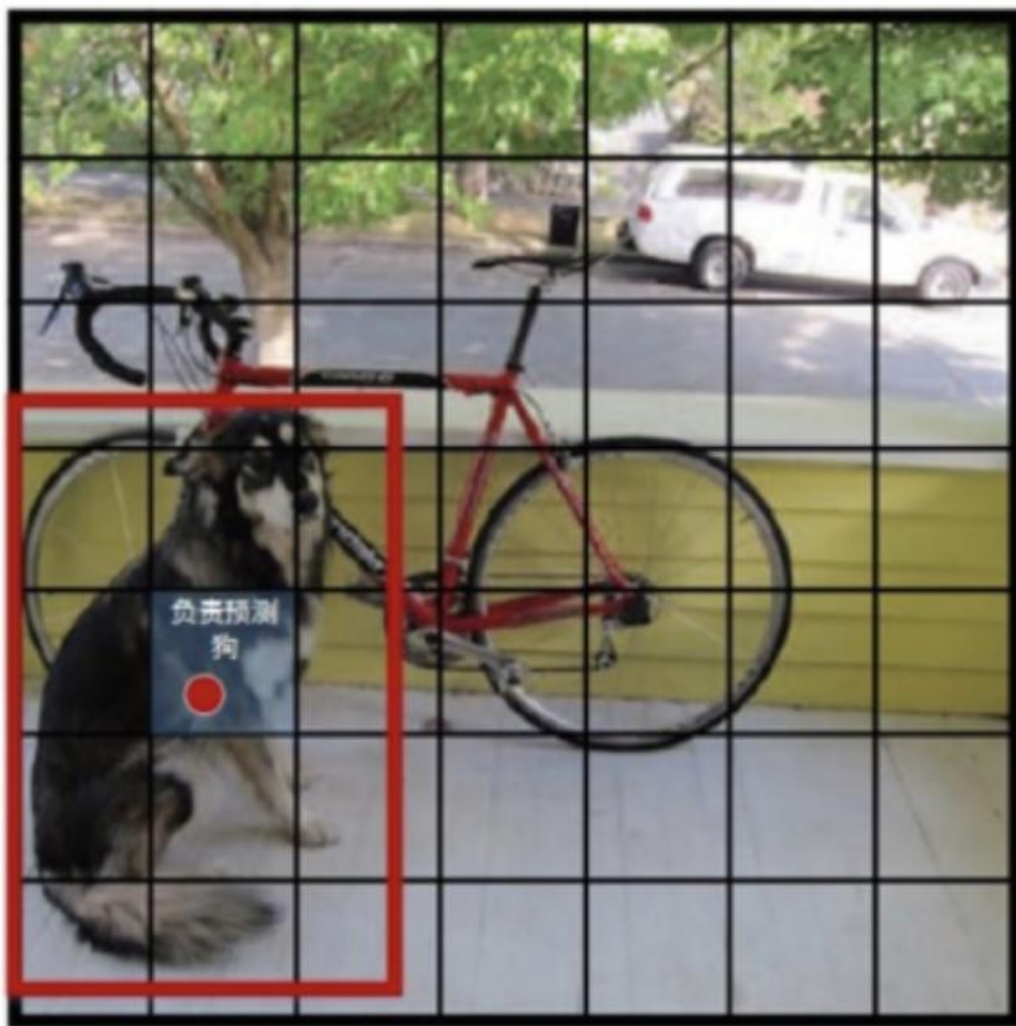
2.14 Yolo v1如何生成目标框

YOLO检测网络包括24个卷积层和2个全连接层，最后一层用来预测目标的位置和类别概率。



生成目标框：

YOLO将输入图像分成SxS个格子，每个格子负责检测‘落入’该格子的物体。若某个物体的中心位置的坐标落入到某个格子，那么这个格子就负责检测出这个物体。如下图所示，图中物体狗的中心点（红色原点）落入第5行、第2列的格子内，所以这个格子负责预测图像中的物体狗。



l anchor数量: $772=98$

yolo将输入图片分成SS个格子，每个格子生成B个bounding box，并且SS和网络最后一层的size一致。

2.15 语义分割模型

使用全卷积神经网络对自然图像进行端到端分割，语义分割才有了重大突破，一般语义分割模型是指加入CNN结构的模型。

语义分割模型有FCN，PSPNet，SegNet，U-Net，DeepLab系列，Mask-RCNN等模型

2.16 说一下DeepLab v1 v2 v3的区别

DeepLab v1是结合了深度卷积神经网络(DCNNs)和概率图模型(DenseCRFs)的方法，采用空洞卷积扩展感受野，获取更多的上下文信息。

Deeplab v2 采用ASPP (atrous spatial pyramid pooling) 多尺度获得更好的分割效果，合并深度卷积网络和概率图模型方法，增强对物体边界的定位，基础层由VGG16转为ResNet。

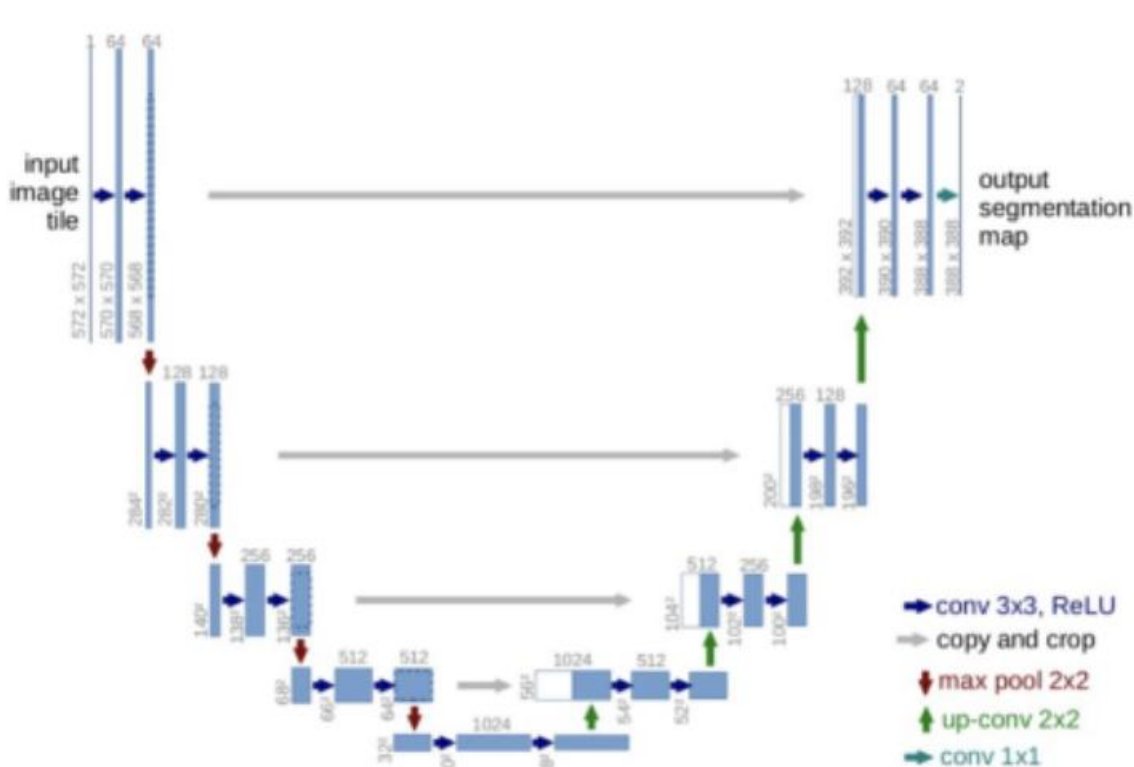
Deeplab v3 改进了ASPP，由不同采样率的空洞卷积和BN层组成，以级联或并行的方式布局模块，去除了条件随机场CRF。

- | Deeplab v1: 空洞卷积+crf
- | Deeplab v2: aspp多尺度融合
- | Deeplab v3: 完善空间金字塔池化模块

2.17 介绍一下Unet

U-net由收缩路径（contracting path）和扩张路径（expanding path）组成。

收缩路径用于获取上下文信息（context），扩张路径用于精确的定位（localization），且两条路径相互对称。



| U-net通过下采样+上采样完成特征编码与解码

| U-net 在channel维度进行多尺度特征融合

| U-net如何实现语义分割？

2.18 Canny边缘检测的流程

Canny 的目标是找到一个最优的边缘检测算法，最优边缘检测的含义是：

- 好的检测- 算法能够尽可能多地标识出图像中的实际边缘。

- 好的定位- 标识出的边缘要尽可能与实际图像中的实际边缘尽可能接近。
- 最小响应- 图像中的边缘只能标识一次，并且可能存在的图像噪声不应标识为边缘。

Canny边缘检测的流程是：

- **图像降噪。**我们知道梯度算子可以用于增强图像，本质上是通过增强边缘轮廓来实现的，也就是说可以检测到边缘的。但是，它们受噪声的影响都很大。那么，我们第一步就是想到要先去除噪声，因为噪声就是灰度变化很大的地方，所以容易被识别为伪边缘。
- **计算图像梯度，得到可能边缘。**计算图像梯度能够得到图像的边缘，因为梯度是灰度变化明显的地方，而边缘也是灰度变化明显的地方。当然这一步只能得到可能的边缘。因为灰度变化的地方可能是边缘，也可能不是边缘。这一步就有了所有可能是边缘的集合。
- **非极大值抑制。**通常灰度变化的地方都比较集中，将局部范围内的梯度方向上，灰度变化最大的保留下来，其它的不保留，这样可以剔除掉一大部分的点。将多个像素宽的边缘变成一个单像素宽的边缘。即“胖边缘”变成“瘦边缘”。
- **双阈值筛选。**通过非极大值抑制后，仍然有很多的可能边缘点，进一步的设置一个双阈值，即低阈值（low），高阈值（high）。灰度变化大于high的，设置为强边缘像素，低于low的，剔除。在low和high之间的设置为弱边缘。进一步判断，如果其领域内有强边缘像素，保留，如果没有，剔除。

2.19 对霍夫变换的理解

霍夫变换常用来提取图像中的直线和圆等几何形状。它通过一种投票算法检测具有特定形状的物体。该过程在一个参数空间中通过计算累计结果的局部最大值得到一个符合该特定形状的集合作为霍夫变换结果。

针对每个像素点，使得theta从-90度到180度，使用极坐标 $p = x\cos(\theta) + y\sin(\theta)$ 计算得到共270组 (p, θ) 代表着霍夫空间的270条直线。将这270组值存储到H中。

如果一组点共线，则这组点中的每个值，都会使得H (p, θ) 加1。

因此找到H (p, θ) 值最大的直线，就是共线的点最多的直线，H (p, θ) 值次大的，是共线点次多的直线。可以根据一定的阈值，将比较明显的线全部找出来

2.20 说一下对HOG特征的理解

HOG (Histogram of Oriented Gradients的简写) 特征检测算法，一种解决人体目标检测的图像描述子，是一种用于表征图像局部梯度方向和梯度强度分布特性的描述符。其主要思想是：在边缘具体位置未知的情

况下，边缘方向的分布也可以很好的表示行人目标的外形轮廓。

整体流程简单描述如下：

- **将输入图像（你要检测的目标或者扫描窗口）灰度化**，即将彩色图转换为灰度图
- **颜色空间归一化**：采用Gamma校正法对输入图像进行颜色空间的标准化（归一化），目的是调节图像的对比度，降低图像局部的阴影和光照变化所造成的影响，同时可以抑制噪音的干扰
- **梯度计算**：计算图像每个像素的梯度（包括大小和方向）；主要是为了捕获轮廓信息，同时进一步弱化光照的干扰
- **梯度方向直方图**：将图像划分成小cells（例如8*8像素/cell），统计每个cell的梯度直方图（不同梯度的个数），即可形成每个cell的描述符
- **重叠直方图归一化**：将每几个cell组成一个block（例如3*3个cell/block），一个block内所有cell的特征descriptor串联起来便得到该block的HOG特征描述符。
- **HOG特征**：将图像image内的所有block的HOG特征描述符串联起来就可以得到该image（你要检测的目标）的HOG特征描述符，就得到最终的可供分类使用的特征向量了

HOG的主要思想是：在一副图像中，局部目标的表象和形状（appearance and shape）能够被梯度或边缘的方向密度分布（即梯度的统计信息，而梯度主要位于边缘的地方）很好地描述。HOG特征检测算法的几个步骤：颜色空间归一化→梯度计算→梯度方向直方图→重叠块直方图归一化→HOG特征

2.21 常用的图像局部特征和全局特征

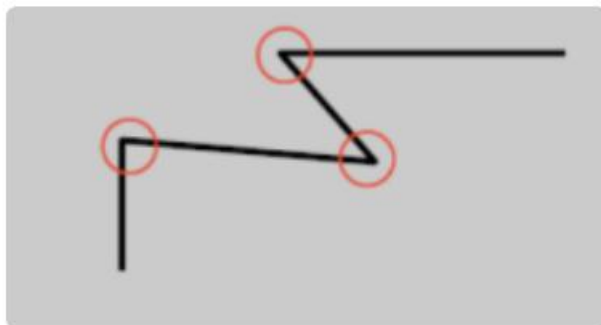
首先明确全局特征和局部特征的意义：

全局特征指图像的整体属性，包括颜色特征、纹理特征、形状特征、直方图等。容易受到干扰，如光照、旋转、噪声等。图像的局部特征反映图像局部的特殊性，局部特征数量丰富，特征间相关度小，不容易受到部分遮挡、光照等噪声的干扰，因为不会因为部分特征的消失而影响其他特征的检测和匹配。典型的局部图像特征生成应包括图像极值点检测和描述两个阶段。好的局部图像特征应具有特征检测重复率高、速度快，特征描述对光照、旋转、视点变化等图像变换具有鲁棒性，特征描述符维度低，易于实现快速匹配等特点。

常用的全局特征就是颜色，纹理，形状，直方图等，局部特征有关键点，角点等

2.22 图像的角点是什么？为什么使用角点作为特征？

首先明确角点是图像中**极值点**，在某方面属性特别突出的点，是在某些属性上强度最大或者最小的孤立点、线段的终点。对于图像而言，如图所示圆圈内的部分，即为图像的角点，其是物体轮廓线的连接点。



描述方法有：

- **一阶导数**(即灰度的梯度)的局部最大所对应的像素点；
- 两条及两条以上边缘的**交点**；
- 图像中梯度值和梯度方向的**变化速率都很高的点**；

角点处的一阶导数最大，二阶导数为零，指示物体边缘变化不连续的方向

2.23 常用的颜色空间

常用的图像空间有：

- **RGB**:是图像采集设备采集图像最常用的一种，由三个分量组成，分别为红色分量(R)，绿色分量(G)和蓝色分量(B)。
- **HSI/HSV**: 分别为色调分量(H)、饱和度分量(S)和亮度分量(I)。色调H分量容是一种颜色“质”的反映，每一种颜色光都有相对应的波长及相应的色调值。饱和度S分量表示一种颜色的纯度。强度I分量反映了光线的强弱变化，是颜色“量”的反映。由于人的视觉对亮度的敏感程度远强于对颜色浓淡的敏感程度，为了便于色彩处理和识别，人的视觉系统经常采用HSI色彩空间，它比RGB色彩空间更符合人的视觉特性.在图像处理和计算机视觉中大量算法都可在HSI色彩空间中方便地使用
- **CMYK**: 应用于印刷工业,印刷业通过青(C)、品(M)、黄(Y)三原色油墨的不同网点面积率的叠印来表现丰富多彩的颜色和阶调，这便是三原色的CMY颜色空间。
- **YUV**: 一种颜色编码方法，常使用在各个视频处理组件中。 YUV在对照片或视频编码时，考虑到人类的感知能力，允许降低色度的带宽。YUV是编译true-color颜色空间（color space）的种类，“Y”表示明亮度，“U”和“V”则是色度、浓度。

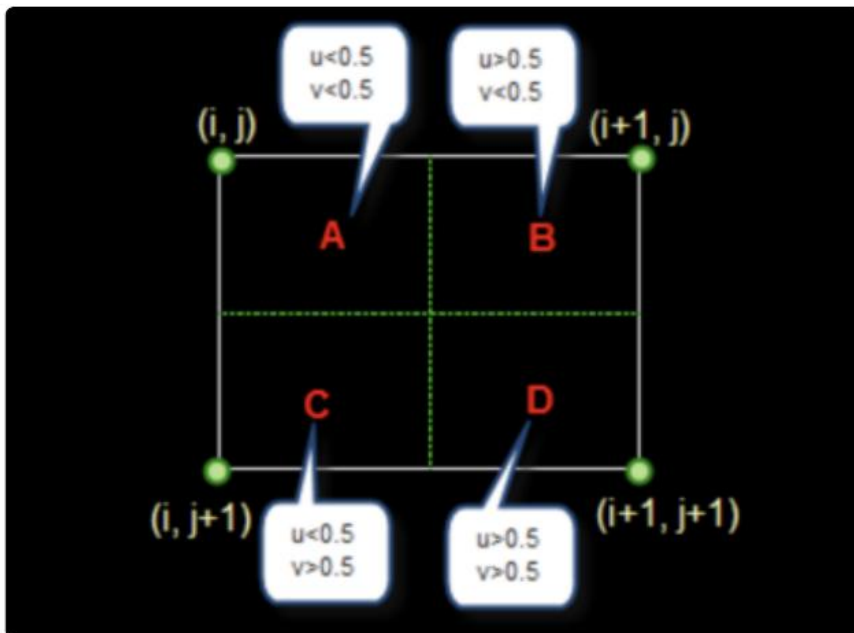
2.24 图像的插值方法有哪些？

在做数字图像处理时，经常会碰到小数象素坐标的取值问题，这时就需要依据邻近象素的值来对该坐标进行插值。比如：做地图投影转换，对目标图像的一个象素进行坐标变换到源图像上对应的点时，变换出来的对应的坐标是一个小数，这时就需要插值。常见的插值方法有：

- **最近邻法**：

这是最简单的一种插值方法，不需要计算，在待求象素的四邻象素中，将距离待求象素最近的邻象素灰度

赋给待求像素。设 $i+u, j+v$ 为正整数， u, v 为大于零小于1的小数，下同)为待求像素坐标，则待求像素灰度的值 $f(i+u, j+v)$ 如下图所示

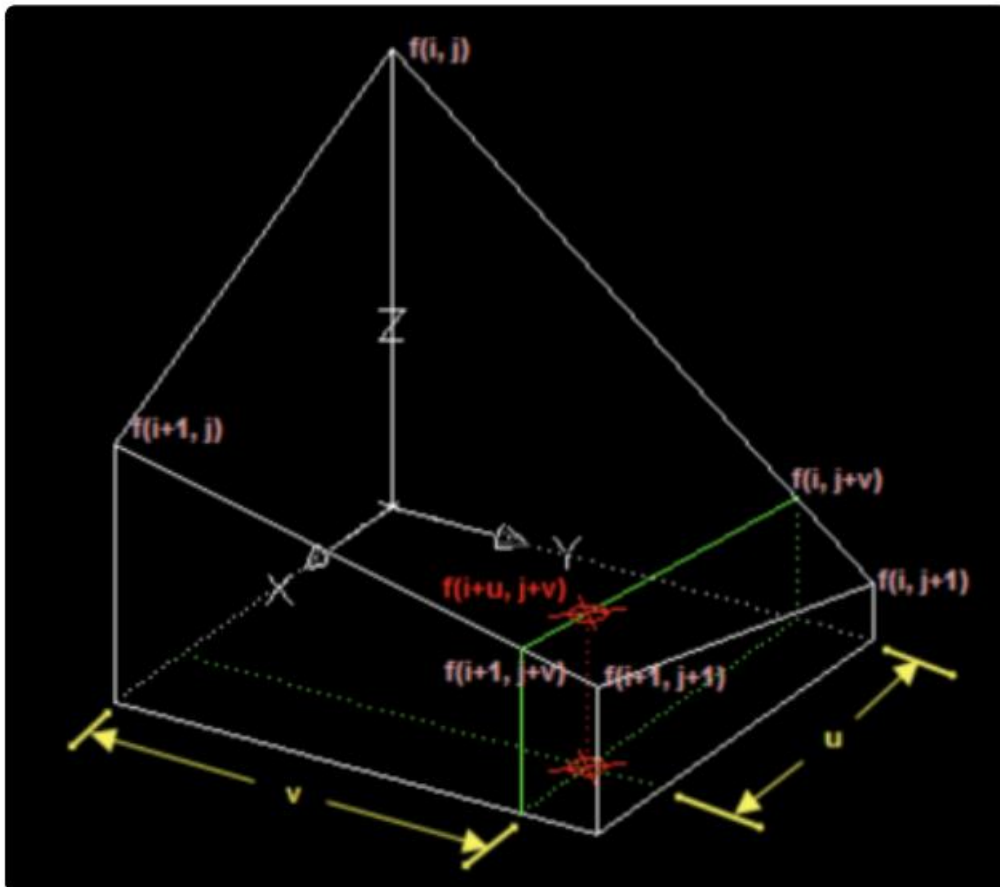


如果落在A区，即，则将左上角像素的灰度值赋给待求像素，同理，落在B区则赋予右上角的像素灰度值，落在C区则赋予左下角像素的灰度值，落在D区则赋予右下角像素的灰度值。

最近邻法计算量较小，但可能会造成插值生成的图像灰度上的不连续，在灰度变化的地方可能出现明显的锯齿状。

- 双线性内插法

双线性内插法是利用待求像素四个邻像素的灰度在两个方向上作线性内插，如下图所示：



对于 $(i, j + v)$, $f(i, j)$ 到 $f(i, j + 1)$ 的灰度变化为线性关系，则有：

$$f(i, j + v) = [f(i, j + 1) - f(i, j)] * v + f(i, j)$$

同理对于 $(i + 1, j + v)$ 则有：

$$f(i + 1, j + v) = [f(i + 1, j + 1) - f(i + 1, j)] * v + f(i + 1, j)$$

从 $f(i, j + v)$ 到 $f(i + 1, j + v)$ 的灰度变化也为线性关系，由此可推导出待求像素灰度的计算式如下：

$$f(i + u, j + v) = (1 - u) * (1 - v) * f(i, j) + (1 - u) * v * f(i, j + 1) + u * (1 - v) * f(i + 1, j) + u * v * f(i + 1, j + 1)$$

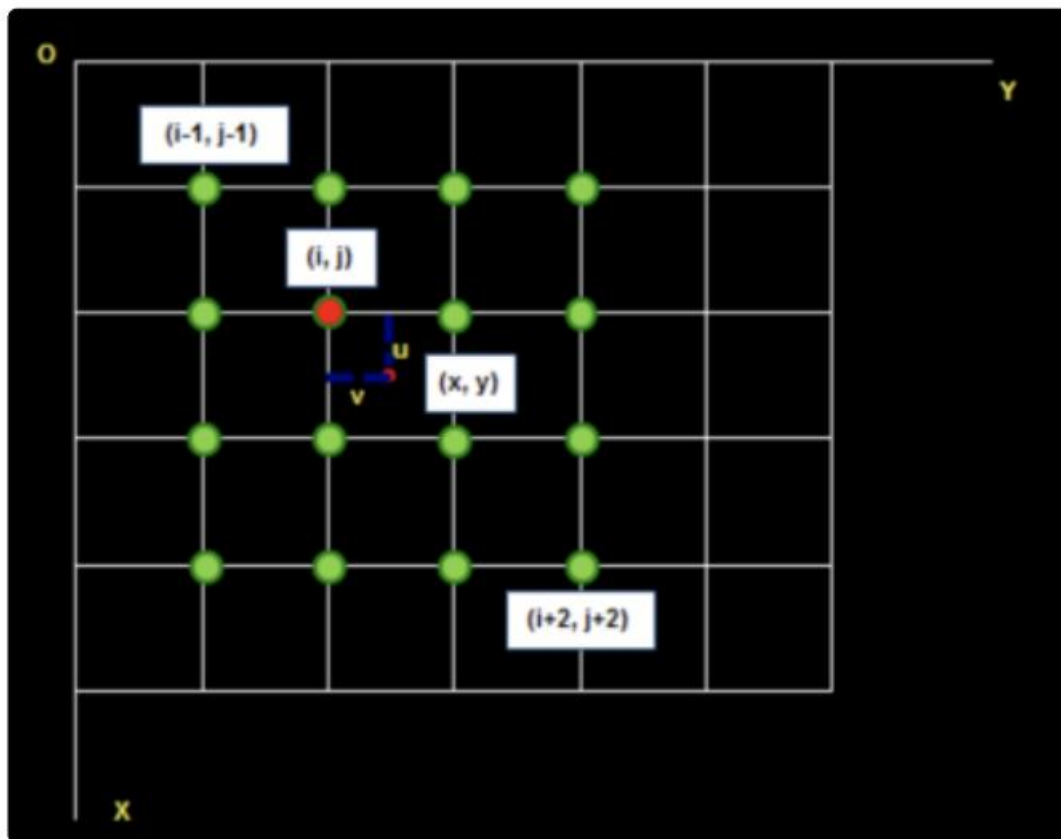
双线性内插法的计算比最邻近点法复杂，计算量较大，但没有灰度不连续的缺点，结果基本令人满意。它具有低通滤波性质，使高频分量受损，图像轮廓可能会有一点模糊。

● 三次内插法

该方法利用三次多项式 $S(x)$ 求逼近理论上最佳插值函数 $\sin(x)/x$ ，其数学表达式为：

$$S(x) = \begin{cases} 1 - 2|x|^2 + |x|^3 & 0 \leq |x| < 1 \\ 4 - 8|x| + 5|x|^2 - |x|^3 & 1 \leq |x| < 2 \\ 0 & |x| \geq 2 \end{cases}$$

待求像素(x, y)的灰度值由其周围16个灰度值加权内插得到，如下图：



待求像素的灰度计算式如下：

$$f(x, y) = f(i + u, j + v) = ABC$$

其中：

$$A = \begin{pmatrix} S(1+v) \\ S(v) \\ S(1-v) \\ S(2-v) \end{pmatrix}^T$$

$$B = \begin{pmatrix} f(i-1, j-1) & f(i-1, j) & f(i-1, j+1) & f(i-1, j+2) \\ f(i, j-1) & f(i, j) & f(i, j+1) & f(i, j+2) \\ f(i+1, j-1) & f(i+1, j) & f(i+1, j+1) & f(i+1, j+2) \\ f(i+2, j-1) & f(i+2, j) & f(i+2, j+1) & f(i+2, j+2) \end{pmatrix}$$

$$C = \begin{pmatrix} S(1+u) \\ S(u) \\ S(1-u) \\ S(2-u) \end{pmatrix}$$

三次曲线插值方法计算量较大，但插值后的图像效果最好。

2.25 常用的滤波方法，写出均值滤波的实现

图像滤波既可以在空域进行，也可以在频域进行。图像滤波可以更改或者增强图像。常用的滤波器是：

- **高斯滤波**：高斯滤波器是响应脉冲为高斯形状的滤波器，对于图像来说，高斯滤波器是利用高斯核的一个2维的卷积算子，用于图像模糊化（平滑**，去除细节和噪声）**
- **均值滤波**：均值滤波器很简单，就是像素的值用窗内邻居的平均值来代替。主要应用于平滑噪声，但是跟上面的高斯滤波器比，它的平滑手段显然更粗暴些，邻域的值不管远近一律平等，高斯滤波器还至少根据远近有些区分。
- **中值滤波**：中值滤波器就是用窗内的中间值代替该像素的值，也多用于减少噪声，特别对于椒盐噪声有很好的效果均值滤波的实现：

Python | 复制代码

```
1 def means_filter(input_image, filter_size):
2
3     # 创建原图像的副本
4
5     input_image_cp = np.copy(input_image)
6
7     # 创建滤波器
8
9     filter_template = np.ones((filter_size, filter_size))
10
11     # 对图像进行padding
12
13     pad_num = int((filter_size - 1) / 2)
```

Python | 复制代码

```
1 input_image_cp = np.pad(input_image_cp, (pad_num, pad_num), mode="constant", constant_values=0)
2
3 # 获取图像的大小
4
5 m, n = input_image_cp.shape
6
7 output_image = np.copy(input_image_cp)
8
9 # 遍历图像进行滤波
10
11 for i in range(pad_num, m - pad_num):
12
13     for j in range(pad_num, n - pad_num):
14
15         output_image[i, j] = np.sum(filter_template * input_image_cp[i - pad_num:i + pad_num + 1, j - pad_num:j + pad_num + 1]) / (filter_size ** 2)
16
17     # 输出滤波后结果
18
19 output_image = output_image[pad_num:m - pad_num, pad_num:n - pad_num]
20
21 return output_image
```

2.26 Graph-cut的基本原理和应用

GraphCut方法图像分割问题与图的最小割（min cut）问题相关联。

首先用一个无向图 G 表示要分割的图像， V 和 E 分别是顶点（vertex）和边（edge）的集合。此处的Graph和普通的Graph稍有不同。普通的图由顶点和边构成，如果边的有方向的，这样的图被则称为有向图，否则为无向图，且边是有权值的，不同的边可以有不同的权值，分别代表不同的物理意义。而Graph Cuts图是在普通图的基础上多了2个顶点，这2个顶点分别用符号“S”和“T”表示，统称为终端顶点。其它所有的顶点都必须和这2个顶点相连形成边集合中的一部分。

根据待分割的图像，确定图的节点与边，即图的形状已确定下来，是给图中所有边赋值相应的权值，然后找到权值和最小的边的组合，就完成了图像分割。

2.27 写一个图像的resize方法

实现图像的缩放，要求不能直接使用opencv等图像处理工具：

获取要处理的图像及图像缩放后的大小

确定缩放比例

遍历缩放后的结果，利用双线性插值法完成图像的缩放

```
Python | 复制代码

1 def image_Resize(src, shape):
2
3     # 获取原图像和图像缩放后的大小。
4
5     height, width, channels = src.shape
6
7     dst_width, dst_height = shape
8
9     # 若缩放后大小与原图像相同，则直接放回原图像
10
11     if ((dst_height == height) and (dst_width == width)):
12
13         return src
14
15     # 创建全0数组存放缩放后图像
16
17     dst_Image = np.zeros((dst_height, dst_width, channels), np.uint8)
18
19     # 缩放比例。
20
21     scale_x = float(width)/dst_width
22
23     scale_y = float(height)/dst_height
24
25     # 遍历
26
27     for k in range(channels):
28
29         for dst_y in range(dst_height):
30
31             for dst_x in range(dst_width):
32
33                 # Original coords.
34
35                 src_x = (dst_x + 0.5) * scale_x - 0.5
36
37                 src_y = (dst_y + 0.5) * scale_y - 0.5
38
39                 # 双线性插值
40
41                 src_x_0 = int(np.floor(src_x))
42
43                 src_y_0 = int(np.floor(src_y))
44
45                 src_x_1 = min(src_x_0 + 1, width - 1)
```

```
46
47     src_y_1 = min(src_y_0 + 1, height - 1)
48
49     value0 = (src_x_1 - src_x) * src[src_y_0, src_x_0, k] +
50             (src_x - src_x_0) * src[src_y_0, src_x_1, k]
51
52     value1 = (src_x_1 - src_x) * src[src_y_1, src_x_0, k] +
53             (src_x - src_x_0) * src[src_y_1, src_x_1, k]
54
55     # 缩放后结果
56
57     dst_Image[dst_y, dst_x, k] = int((src_y_1 - src_y) * value0 +
58                                     (src_y - src_y_0) * value1)
59
60     return dst_Image
```

2.28 图像中直方图的应用

直方图就是对数据进行统计的一种方法，并且将统计值组织到一系列实现定义好的 bin 当中。其中，bin 为直方图中经常用到的一个概念，可以译为“直条”或“组距”，其数值是从数据中计算出的特征统计量，这些数据可以是诸如梯度、方向、色彩或任何其他特征。且无论如何，直方图获得的是数据分布的统计图。通常直方图的维数要低于原始数据。

图像直方图 (Image Histogram) 是用以表示数字图像中亮度分布的直方图，标绘了图像中每个亮度值的像素数。这种直方图中，横坐标的左侧为纯黑、较暗的区域，而右侧为较亮、纯白的区域。因此一张较暗图片的直方图中的数据多集中于左侧和中间部分，而整体明亮、只有少量阴影的图像则相反。CV 领域常借助图像直方图来实现图像的二值化。

直方图的性质：

- 直方图反映了图像中的灰度分布规律。它描述每个灰度级具有的像元个数，但不包含这些像元在图像中的位置信息。
- 任何一幅特定的图像都有唯一的直方图与之对应，但不同的图像可以有相同的直方图。
- 如果一幅图像有两个不相连的区域组成，并且每个区域的直方图已知，则整幅图像的直方图是该两个区域的直方图之和。

应用方式如下所示：

- 每幅图像都可做出其灰度直方图。

- 根据直方图的形态可以大致推断图像质量的好坏。由于图像包含有大量的像元，其像元灰度值的分布应符合概率统计分布规律。假定像元的灰度值是随机分布的，那么其直方图应该是正态分布。
- 图像的灰度值是离散变量，因此直方图表示的是离散的概率分布。若以各灰度级的像元数占总像元数的比例值为纵坐标轴做出图像的直方图，将直方图中各条形的最高点连成一条外轮廓线，纵坐标的比例值即为某灰度级出现的概率密度，轮廓线可近似看成图像相应的连续函数的概率分布曲线

2.29 SIFT/SURF的特征提取方法，是如何保持尺度不变性的？

主要考察SIFT/SURF特征的理解，SIFT算法的实质是在不同的尺度空间上查找关键点(特征点)，并计算出关键点的方向。SIFT所查找到的关键点是一些十分突出，不会因光照，仿射变换和噪音等因素而变化的点，如角点、边缘点、暗区的亮点及亮区的暗点等。

- **尺度空间极值检测**：搜索所有尺度上的图像位置。通过高斯差分函数来识别潜在的对于尺度和旋转不变的关键点。
- **关键点定位**：在每个候选的位置上，通过一个拟合精细的模型来确定位置和尺度。关键点的选择依据于它们的稳定程度。
- **关键点方向确定**：基于图像局部的梯度方向，分配给每个关键点位置一个或多个方向。所有后面的对图像数据的操作都相对于关键点的方向、尺度和位置进行变换，从而保证了对于这些变换的不变性。
- **关键点描述**：在每个关键点周围的邻域内，在选定的尺度上测量图像局部的梯度。这些梯度作为关键点的描述符，它允许比较大的局部形状的变形或光照变化。

SURF 算法，是SIFT算法的增强版，它的计算量小，运算速度快，提取的特征与SIFT几乎相同。

SIFT/SURF特征:尺度不变特征转换，在空间尺度中寻找极值，提取位置，尺度、旋转不变量。流程是：尺度空间极值检测—>关键点定位—>关键点方向确定—>关键点描述

十、面试时注意事项和人事面

1. 请介绍一下你自己

回答提示：这是面试官100%会问的问题，一般人回答这个问题过于平常，只说姓名、年龄、爱好、所学专业等，如果你用一分钟来重复你的简历，那么，你的印象加分没有了！不妨坦诚自信地展现自我，重点突出与应聘职位相吻合的优势。你的相关能力和素质是企业最感兴趣的信息。因为，在许多情况下，在听取你的介绍时，面试官也会抓住他感兴趣的点深入询问。所以，在进行表述时，要力求以真实为基础，顾及

表达的逻辑性和条理性，避免冗长而没有重点的叙述。一定要在最短的时间内激发起面试官对你的好感。

回答范例：

- 1) 我叫 XX，今年 X 岁，XXXX 年毕业于 XX 大学。有 3 年的开发工作经验，我对技术有深厚的兴趣，专业知识面宽，责任心强，思路清晰，沟通力能好，精通.Net 技术体系，熟悉 MVC。平常有时间看看博客，并且自己也喜欢在CSDN上写技术类的文章，与博友一起讨论。谢谢！
- 2) 如果跟人事自我介绍：您好！我叫***，（如果学校和专业好的话可以提一下，一般的话就算了。）我之前在 XX 公司做过两年 JAVA 开发工程师，在这两年当中大概接触过什么样类型的项目，（比如商城，金融等）今天看了咱们公司招聘的岗位需求跟我之前的工作内容挺相符的，而且咱们公司的业务也是我喜欢的类型，所以我今天过来面试。希望可以加入到咱们公司当中来。
- 3) 如果是跟技术自我介绍：您好！我叫***，（如果学校和专业好的话可以提一下，一般的话就算了。）我之前在 XX公司做过两年 JAVA 开发工程师，在这两年当中大概接触过什么样类型的项目，（比如商城，金融等）；在做这些项目的当中常用的一些技术点包括 XX 技术，（另外除了常用的技术还会一些其他的专业技能），今天看了咱们公司招聘的岗位需求跟我之前的工作内容挺相符的，所以我今天过来咱们公司面试，希望可以加入到咱们这个大家庭当中来

2. 为什么来XX（北京或深圳）找工作？

面试官对异地求职者90%都会问的问题，主要考察你是否稳定，个人经验能力之外，排在第一位的就是稳定性，如果不够稳定，那么其余都是空谈。

回答范例：我来自河南，河南是一个农业大省，IT 行业还不是很发达，我是学计算机专业的，也很喜欢这个行业，深圳在国内 IT 行业发展是最快的，所以我想来这里谋求发展，学习更多的新技术，能够带来自我的提升。

注意：不要说以前公司有多么不好。也不要说哪个哥们混的很不错，羡慕才来深圳。因为企业招人想要的都是能够长期工作的人，可能哪个哥们哪天在别的地方又混的更好了，你是不是还要跳槽？所以，只要说来学习更多新技术和管理经验就够了。

3. 你为什么离开原来的公司？

回答提示：最重要的是：应聘者要使找招聘单位相信，应聘者在过往的单位的“离职原因”在此家招聘单位里不存在。避免把“离职原因”说得太详细、太具体。不能掺杂主观的负面感受，如“太辛苦”、“人际关系复杂”、“管理太混乱”、“公司不重视人才”、“公司排斥我们某某的员工”等。但也不能躲闪、回避，如“想换换环境”、“个人原因”等。不能涉及自己负面的人格特征，如不诚实、懒惰、缺乏责任感、不随和等。尽量使解释的理由为应聘者个人形象添彩。

回答范例：如“我离职是因为这家公司倒闭；我在公司工作了三年多，有较深的感情；从去年始，由于市场形势突变，公司的局面急转直下；到眼下这一步我觉得很遗憾，但还要面对现实，重新寻找能发挥我能力的舞台。”同一个面试问题并非只有一个答案，而同一个答案并不是在任何面试场合都有效，关键在应聘者掌握了规律后，对面试的具体情况进行把握，有意识地揣摩面试官提出问题的心理背景，然后投其所好。

分析：除非是薪资太低，或者是最初的工作，否则不要用薪资作为理由。“求发展”也被考官听得太多，离职理由要根据每个人的真实离职理由来设计，但是在回答时一定要表现得真诚。离职原因没有定式，要根据自己的情况以及应聘公司的情况灵活的去说，大概方向就是：合同到期，距离，技术，发展，公司经营不好，其他。

4. 你最大的缺点是什么？

被面试官问的概率很大，也是 HR 的杀手锏和狠招，这个问题最难回答，通常面试官不希望听到求职直接回答的缺点是什么，如果求职者说自己小心眼、脾气大、工作效率低，企业肯定不会录用你。不要自作聪明地回答“我最大的缺点就是过于追求完美”，有的人以为这样回答会显得自己比较出色，但事实上，他已经岌岌可危了。面试官喜欢求职者从自己的优点说起，中间加一些小缺点，最后再把问题转回到优点上，突出优点的部分，面试官喜欢聪明的求职者。

回答范例：这个问题好难回答啊！我想想……（亲和力表现，也缓解了自己的紧张情绪）

我的缺点是比较执着，比如在技术方面比较爱钻研，有的时候会为一个技术问题加班到深夜。还有就是，工作比较按部就班，总是按照项目经管的要求完成任务。另外的缺点是，总在自己的工作范围内有创新意识，并没有扩展给其他同事。这些问题我想我可以进入公司后以最短的时间来解决，我的学习能力很强，我相信可以很快融入公司的企业文化，进入工作状态。我想就这些吧。

5. 你对薪资的要求？

如果你对薪酬的要求太低，那显然贬低自己的能力；如果你对薪酬的要求太高，那又会显得你分量过重，公司受用不起。一些雇主通常都事先对求职的职位定下开支预算，因而他们第一次提出的价钱往往是他们所能给予的最高价钱，他们问你只不过想证实一下这笔钱是否足以引起你对该工作的兴趣。

回答范例一：我对工资没有硬性要求，我相信贵公司在处理我的问题上会友善合理。我注重的是找对工作的机会，所以只要条件公平，我则不会计较太多。如果你必须自己说出具体数目，那就不要说一个宽泛的范围，不要说10000-13000之间，那样你将只会得到最低限底的数字，也就是10000。最好给出一个具体的数字。你可以说我上家给的薪资是14K，个人期望薪资有一定的涨幅。

6. 什么时候能入职?

大多数企业会关心就职时间，最好是回答“如果被录用的话，到职日可按公司的规定上班”，如果还未辞去上一个工作，但上班时间又太近，似乎有些强人所难，因为交接至少要一个月的时间，应进一步说明原因，录取公司应该会通融的。

7. 介绍一个你认为最熟悉的项目（项目经理）

这个问题在技术面试时常被问到，问这个问题的意图是想考察你的成长路径和编程习惯，因为，你最熟悉的项目往往是 你成长最快的项目，那个成长最快的项目往往会给你今后的编程习惯留下很多痕迹。所以，通过你对熟悉项目的描述，有经验的他会很快锁定你技术成长中的缺陷和闪光点，从而判断是否能够“为我所用”。

你最好拿出一个自己最擅长技术的那个项目进行介绍，他听完你的介绍后，会接下来进行提问，这样他所有问的问题，你都成竹在胸了。

切忌拿自己参与很少的项目来介绍，一旦他深入的询问很可能你会答非所问，反而造成更严重的影响。你大强以和他谈 谈在那个项目中获得的经验，这样会引起此君的共鸣，有可能的话，说出一些你自己的小技巧，他会很高兴，同时这场 面试也会很轻松，拿到 Offer 基本没问题了。

8. 如果公司录用你，你将怎样开展工作?

很多企业在招聘开发人员时很看重是否能够尽快上手，所以回答这个问题时要“实打实”的回答，在回答中最好强调能够“尽快”投入开发工作中，这样领导就放心了，会觉得你不是一个只会盲目工作的人，而是一个按部就班，稳打稳扎的人。

回答范例：我对咱们公司的大体情况只有一个大概了解，在这个职位的工作性质仅仅是我自己的一个理解。作为这个职位而言，我想我首先要对本公司的主营业务要有一个了解，了解公司的业务组成部分、业务的发展方向、我们面向的客户性质等。第二我要了解所属部门在公司中的地位，以及部门的工作目标，从而确定自身的工作努力方向。第三，了解我参与项目的开发方式，架构方式，紧密配合领导工作，尽快投入具体的开发工作中。这就是我开展工作的计划。

9. 你还有什么问题问我吗?

这个问题看上去可有可无，其实很关键，面试官不喜欢说“没有问题”的人，没有问题就是自寻死路，没有问题传达出你 对公司缺乏兴趣，而只是来寻找一笔薪水。其实在面试过程中谦虚礼貌的问面试官怎么称呼，

该部门工作中的信息，如项目情况，开发技术再或者说贵公司的晋升机制是什么样的等。表现出一种很积极主动的状态是非常讨巧的。也可以更多的了解到自己来的工作环境。企业很欢迎这样的求职者，因为体现出你对学习的热情和对公司的忠诚度以及你的上进心。这个要分析面试官的身份，如果是普通技术面试官，可以问问对方目前项目的规划，使用的技术情况，还有就是您在这边工作多久了，是否能分享一些您在这边的工作体会。如果是 HR 的话，可以问问公司的企业文化，工资评定是否跟绩效挂钩，公司是否定期组织员工活动、团建等等，如果是公司 CEO 或者是 CTO 的话，可以问问团队的组成和公司前景之类的。或者告诉面试官关于公司的部分情况通过查看公司网站，还有面试基本上了解了，比较喜欢咱们公司这个平台和氛围，希望有机会能加入一起共事，如果您这边觉得我合适的话，最快什么时间能给我回复。

10. 社保问题

如果上家公司写的是外地公司的话，之前是否上过社保深圳这边是查不到的，而且都要重新上。如果上家写的深圳公司，之前确实没有上过的话，就说之前公司规模小，五险一金一直 没有上，但是签了自动放弃的协议，公司也给一部分补偿金，金额大概 1000 左右。如果之前你在深圳上过一段时间的社保，跟现在写的工作经历对不上的话，就说之前公司不给上，有一段时间自己通过关系找了一家公司挂靠了一段时间。面试的时候如果HR问你之前在深圳是否上过社保，只要说是或者否就可以，不用主动说太多。对方问的话再解释。

11. 上家公司薪资构成，及到手多少钱？

网上找税收计算器，把上家薪资输入自动算出扣税多少，再加上社保补偿金，就是你最后到手的工资，不用跟面试官说的特详细，只要说个大概就好。

12. 你有没有在工作中遇到过你无论如何都难以解决的问题？你的态度如何？

工作中难免会有些问题难以被彻底的解决，在这样的情况下，我会思考这样几个问题：我是否已经尽了全力，是否还有什么方法或什么人能够帮助我解决问题？我是否已经全面的思考过问题的方方面面，还是钻在死胡同里出不来？我是否能在目前情况下改善问题，哪怕只是较小程度的改善？有时一些小小的改善积累起来可能最终就能够解决问题。我认为遇到问题的态度切忌慌乱，往往欲速则不达，因此要冷静下来思考。另外就是切记不停的抱怨，不停的抱怨不但会给人留下负面印象，甚至因为消极的态度错过了解决问题的最后机会。

点评：该求职者首先表现了他不会轻易放弃解决问题的态度，懂得从各个角度去思考问题，并且会努力到最后一刻。在表述自己遇到问题的态度方面，该求职者也表现的非常完美，冷静、不抱怨并积极处理问题是所有面试官都会欣赏的态度。

13. 你对加班的看法？

实际上好多公司问这个问题，并不证明一定要加班，只是想测试你是否愿意为公司奉献。回答范例：如果是工作需要我 会议不容辞加班，我现在单身，没有任何家庭负担，可以全身心的投入工作。但同时，我也会提高工作效率，减少不必要的加班。

14. 你朋友对你的评价？

回答提示： 想从侧面了解一下你的性格及与人相处的问题。

回答样本一：我的朋友都说我是一个可以信赖的人。因为，我一旦答应别人的事情，就一定会做到。如果我做不到，我就不会轻易许诺。

回答样本二：我觉的我是一个比较随和的人，与不同的人都可以友好相处。在我与人相处时，我总是能站在别人的角度考虑问题。

15. 谈谈你对跳槽的看法？

回答提示：①正常的“跳槽”能促进人才合理流动，应该支持。②频繁的跳槽对单位和个人双方都不利，应该反对。

16. 工作中你难以和同事、上司相处，你该怎么办？

回答提示：

①我会服从领导的指挥，配合同事的工作。

②我会从自身找原因，仔细分析是不是自己工作做得不好让领导不满意，同事看不惯。还要看看是不是为人处世方面做得不好，如果是这样的话我会努力改正。

③如果我找不到原因，我会找机会跟他们沟通，请他们指出我的不足，有问题就及时改正。

④作为优秀的员工，应该时刻以大局为重，即使在一段时间内，领导和同事对我不理解，我也会做好本职工作，虚心向 他们学习，我相信，他们会看见我在努力，总有一天会对我微笑的。

17. 假设你在某单位工作，成绩比较突出，得到领导的肯定，但同时你发现同事越来越孤立你，你怎么看这个问题？你准备怎么办？

回答提示：

①成绩比较突出，得到领导的肯定是件好事情，以后更加努力。

②检讨一下自己，是不是对工作的热心度超过同事间交往的热心了，加强同事间的交往及共同的兴趣爱好。

③ 工作中，切勿伤害别人的自尊心。

④ 不再领导前拨弄是非。

18. 最能概括你自己的三个词是什么？

回答提示：我经常用的三个词：适应能力强，有责任心和做事有始终，结合具体例子向主考官解释，

19. 你的业余爱好是什么？

回答提示：找一些富于团体合作精神的，这里有一个真实的故事：有人被否决掉，因为他的爱好是深海潜水。主考官说：因为这是一项单人活动，我不敢肯定他能否适应团体工作。多说团体合作，爱浏览技术网站、博客、书籍。最好提前想好最近在看那本书，看到了哪个部分。

20. 说说你对行业、技术发展趋势的看法

回答提示：企业对这个问题的兴趣，只有有备而来的求职者能够过关。求职者可以直接在网上查找对你所申请的行业部门的信息，只有深入了解才能产生独特的见解。企业认为最聪明的求职者是对所面试的公司预先了解很多，包括公司各个部门，发展情况，在面试回答问题的时候可以提到所了解的情况，企业欢迎进入企业的人是“知己”，而不是“盲人”。

21. 说你的家庭？

回答提示：企业面试时询问家庭问题不是非要知道求职者家庭的情况，探究隐私，企业不喜欢探究个人隐私，而是要了解家庭背景对求职者的塑造和影响。企业希望听到的重点也在于家庭对求职者的积极影响。企业最喜欢听到的是：我很爱我的家庭，我的家庭一向很和睦，虽然我的父亲和母亲都是普通人，但是从小，我就看到我父亲起早贪黑，每天工作特别勤劳，他的行动无形中培养了我认真负责的态度和勤劳的精神。我母亲为人善良，对人热情，特别乐于助人，所以在单位人缘很好，她的一言一行也一直在教导我做人的道理。企业相信，和睦的家庭关系对一个人的成长有潜移默化的影响。

22. 就你申请的这个职位，你认为你还欠缺什么？

回答提示：企业喜欢问求职者弱点，但精明的求职者一般不直接回答。他们希望看到这样的求职者：继续重复自己的优势，然后说：“对于这个职位和我的能力来说，我相信自己是可以胜任的，只是缺乏经验，这个问题我想我可以进入公司以后以最短的时间来解决，我的学习能力很强，我相信可以很快融入公司的企业文化，进入工作状态。”企业喜欢能够巧妙地躲过难题的求职者。

23. 你欣赏哪种性+格的人？

回答提示：诚实、不死板而且容易相处的人、有“实际行动”的人。

24. 你为什么愿意到我们公司来工作？

回答提示：对于这个问题，你要格外小心，如果你已经对该单位作了研究，你可以回答一些详细的原因，像“公司本身的高技术开发环境很吸引我。”、“我同公司出生在同样的时代，我希望能够进入一家与我共同成长的公司。”、“你们公司一直都稳定发展，在近几年来在市场上很有竞争力。”、“我认为贵公司能够给我提供一个与众不同的发展道路。”这都显示出你 已经做了一些调查，也说明你对自己的未来有了较为具体的远景规划。

25. 你和别人发生过争执吗？你是怎样解决的？

回答提示：这是面试中最险恶的问题，其实是考官布下的一个陷阱，千万不要说任何人的过错，应知成功解决矛盾是一个协作团体中成员所必备的能力。假如你工作在一个服务行业，这个问题简直成了最重要的一个环节。你是否能获得这份工作，将取决于这个问题的回答。考官希望看到你是成熟且乐于奉献的。他们通过这个问题了解你的成熟度和处世能力。在没有外界干涉的情况下，通过妥协的方式来解决才是正确答案

26. 你做过的哪件事最令自己感到骄傲？

回答提示：这是考官给你的一个机会，让你展示自己把握命运的能力。这会体现你潜在的领导能力以及你被提升的可能性。假如你应聘于一个服务性质的单位，你很可能被邀请去午餐。记住：你的前途取决于你的知识、你的社交能力和综合表现。

27. 对这项工作，你有哪些可预见的困难？

回答提示：①不宜直接说出具体的困难，否则可能令对方怀疑应聘者不行。②可以尝试迂回战术，说出应聘者对困难所持有的态度——工作中出现一些困难是正常的，也是难免的，但是只要有坚忍不拔的毅力、良好的合作精神以及事前周密而充分的准备，任何困难都是可以克服。

分析：一般问这个问题，面试者的希望就比较大了，因为已经在谈工作细节，但常规思路中的回答，又被面试官“骗”了。当面试官询问这个问题的时候，有两个目的。第一，看看应聘者是不是在行，说出的困难是不是在这个职位中一般都不可避免的问题。第二，是想看一下应聘者解决困难的手法对不对，及公司能否提供这样的资源。而不是想了解应聘者对困难 的态度。

28. 与上级意见不一致，你将怎么办？

回答提示：①一般可以这样回答“我会给上级以必要的解释和提醒，在这种情况下，我会服从上级的意见。”②如果面试你的是总经理，而你所应聘的职位另有一位经理，且这位经理当时不在场，可以这样回答：“对于非原则性问题，我会服从上级的意见，对于涉及公司利益的重大问题，我希望能向更高层领导反映。”

分析：这个问题的标准答案是思路①，如果用②的回答，必死无疑。你没有摸清楚改公司的内部情况，先想打小报告，这样的人没有人敢要。

29. 你希望与什么样的上级共事？

回答提示：①通过应聘者对上级的“希望”可以判断出应聘者对自我要求的意识，这既上一个陷阱，又是一次机会。②最好回避对上级具体的希望，多谈对自己的要求。③如“做为刚步入社会的新人，我应该多要求自己尽快熟悉环境、适应环境，而不应该对环境提出什么要求，只要能发挥我的专长就可以了。”

分析：这个问题比较好的回答是，希望我的上级能够在工作中对我多指导，对我工作中的错误能够立即指出。总之，从上级指导这个方面谈，不会有大的纰漏。

30. 为什么我们要在众多的面试者中选择你？

回答提示：根据我对贵公司的了解，以及我在这份工作上所累积的专业、经验及人脉，相信正是贵公司所找寻的人才。而我在工作态度、EQ上，也有圆融、成熟的一面，和主管、同事都能合作愉快。

分析：别过度吹嘘自己的能力，或信口开河地乱开支票，例如一定会为该公司带来多少钱的业务等，这样很容易给人一种爱说大话、不切实际的感觉。技术的符合度，经验，人脉，积极乐观，容易相处，擅长求同存异，以团队为主。

31. 怎样看待学历和能力？

回答提示：学历我想只要是大学专科的学历，就表明觉得我具备了根本的学习能力。剩下的，你是学士也好，还是博士也好，对于这一点的讨论，不是看你学了多少知识，而是看你在这个领域上发挥了什么，也就是所说的能力问题。一个人工作能力的高低直接决定其职场命运，而学历的高低只是进入一个企业的敲门砖。

32. 工作中学习到了些什么？

回答提示：这是针对转职者提出的问题，建议此时可以配合面试工作的特点作为主要依据来回答，如业务工作需要与人沟通，便可举出之前工作与人沟通的例子，经历了哪些困难，学习到哪些经验，把握这些要点做陈述，就可以轻易过关了。

33. 你希望你的上级是什么样子的？

我希望我的上级能够在工作中对我多指导，对我工作中的错误和问题能够及时指出。

34. 除了本公司外，还应聘了哪些公司？

回答提示：很奇怪，这是相当多公司会问的问题，其用意是要概略知道应徵者的求职志向，所以这并非绝对负面答案，就算不便说出公司名称，也应回答“类似的公司（互联网）”，如果应聘的其他公司是不同业界，容易让人产生无法信任的感觉。

35. 面试注意事项

- 1) 在面试官面前千万不要抖脚，手脚不要动来动去，不能有小动作。
- 2) 在面试过程中，千万不要跟面试官去争论，说话太冲，太能说、抢话说、乱说都不好，遇到难题，先思考一下，切记心浮气燥，表达时口气温和，谦虚。
- 3) 如果面试过程中都不错，谈的也很好，之后却没有给 Offer，完全是自己意料之外的情况，这个很有可能，或许是因为公司有了其他的人选，不用介意，更不要沮丧。
- 4) 在面试过程中，切忌问关于公司计划、行业机密等相关的东西，不要打探公司的内幕，机密敏感性的问题不要问东问西。
- 5) 千万要注意仪容仪表，要有礼貌，最好不要有口吃，口头表达，逻辑思维很重要，不要让面试官觉得你很幼稚，太过小孩子气，显的不够稳重踏实。
- 6) 在去面试之前，要熟悉自己的简历，特别是工作经历，准备好关于一些离职原因、职业规划方面的问题的回答方式。在面试过程中，90%的面试官会让做自我介绍，所以提前要准备一下，说出的内容既要和简历相符，又要有重点有突出的地方，不能像背简历一样。
- 7) 面试完后，如果等待的时间较长，没有回应，就可能没有什么希望了，自己可以打电话去了解情况。
- 8) 在面试过程中，谈到薪资的时候，如果没有说明是税后工资就是税前，假如是税前6000，这里面就包括了公司给交的公积金，还有其他五险要交的费用，拿到手差不多4500左右。